전역 조명

리얼타임 렌더링 4판 · 11장 Global Illumination

1차시 : 복습 ~ 일반적인 전역 조명 모델

2차시 : 주변 폐색 (Ambient Occlusion)

3차시 : 방향성 폐색 (Directional Occlusion) ,확산 전역 조명

4차시 : 반사 전역 조명, 기타 & 최신 엔진 UseCase

오늘의 순서 - 1차시

00 서론 — 컴퓨터 그래픽스처럼 보이지 않기

01 복습, GI와는 무엇이 다른가 — 7·9·10장

02 렌더링 방정식과 전역 조명 알고리즘 — 11.1

03 일반적인 전역 조명 알고리즘 — 11.2

서론

“컴퓨터 그래픽스처럼 보이면, 좋은 컴퓨터 그래픽스가 아니다.”

— 제레미 번, 『디지털 라이팅 & 렌더링』

CG를 다루는 고전적인 교과서 포지션의 서적에서 인용한 문구라고 하고
프로그래밍지식보다는 그냥 사실적인 조명, 그림자, 텍스쳐, 라이팅, 렌더링 기법을 다루는 책이라고 합니다.

3D 아티스트를 위한 지식과 TA, 그리고 비주얼 프로그래머에게 도움이 되는 서적으로 보이는 내용들이 많이 있음
(물론 게임에서 사용하는 지식은 일부가 될 것 같습니다. 아 슬프구나)

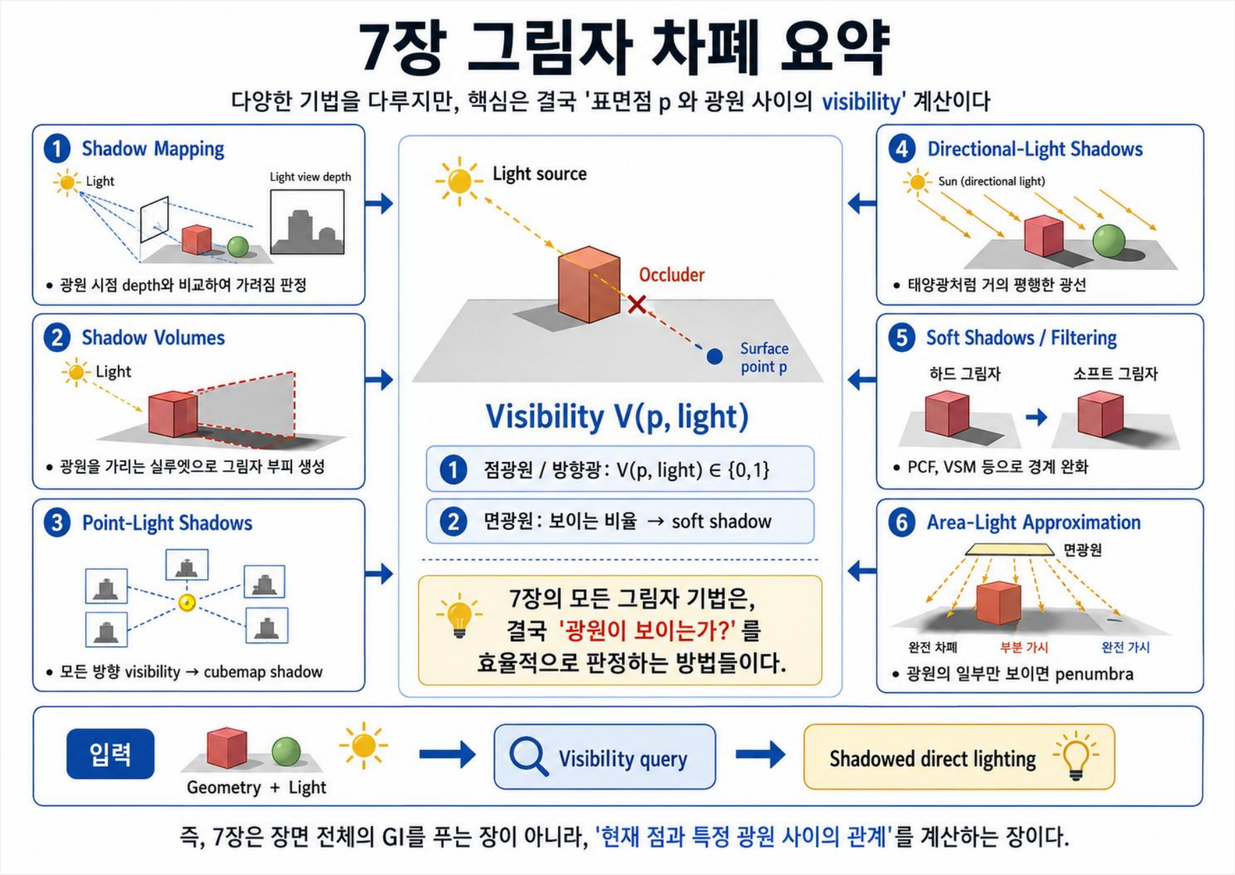
리얼타임 렌더링 4판 · 11장 전역 조명 · 1차시

복습 — 7·9·10

본격적인 GI파트를 다루기 전에
그림자, PBR, Local Illumination 파트에서 다룬 내용과의 연관점, 차이점을 먼저 확인하고
앞으로 보게될 GI파트에서 해결하고자 하는 문제가 무엇인지 확인합시다.

7장 • 그림자

Shadow Map, Shadow Volume등으로 다른 오브젝트가 드리우는 그림자를 계산했지만,
이때 계산한 그림자는 직접광에 대한 가시성만을 판단하는 그림자였습니다.



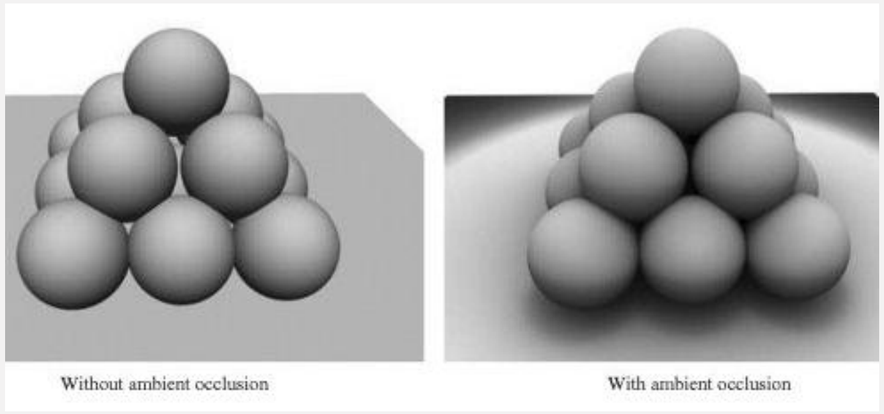
고마워요 GPT!

7장 • 현실의 빛과 어긋나는 지점

7장에서 계산한 것

지정한 광원 방향의 가시성

광원은 점(Point)거나 방향(Directional)이였고,
판정은 이진 값으로 처리되었습니다 (가려짐/안가려짐)
광원이 여러 개일 경우, 각 광원마다의 가시성을 판단했습니다.



광원의 위치 (위쪽) 기준으로 직접광의 차폐만을 고려했을 때와 간접광의 차폐를 비교한 모습

실제로 일어나는 일

다른 물체가 반사하는 빛도 차폐 될 수 있다

현실세계는 직접적인 광원 (Light Source)외에도, 대부분의 물체가 빛을 반사합니다.
그렇기 때문에, 동일하게 직접광을 받는 상황에도 직접광이 없는 방향의 차폐가
그림자를 만들게 됩니다.

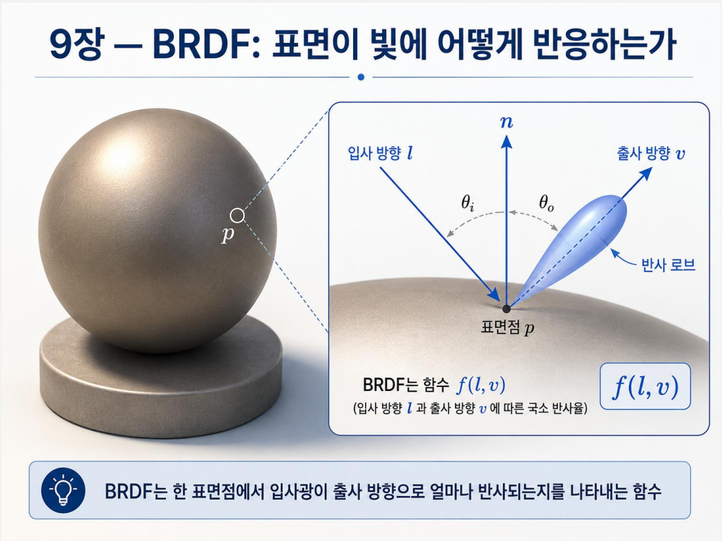
9장 · BRDF

양방향 반사도 분포 함수

$f(l, v)$

방향 l에서 들어온 빛이 방향 v로 얼마나 나가는가에 대한 함수
재질에 따라 특수한 함수를 사용하거나, 미세 표면을 고려하는 등의 내용이 있었지만
재질과 입사/출사 방향이 동일한 경우 언제나 같은 값을 냄.

입사 방향 l에서 광량이 얼마나 들어오는지에 대해 다룬건 아니고, 단순히 재질(material)의 특성을 다룬 파트라고 이해



9장 · BRDF

반사율 방정식 · 9장

$$L_o(p, v) = \int_{l \in \Omega} f(l, v)\, L_i(p, l)\, (n \cdot l)^+ dl$$

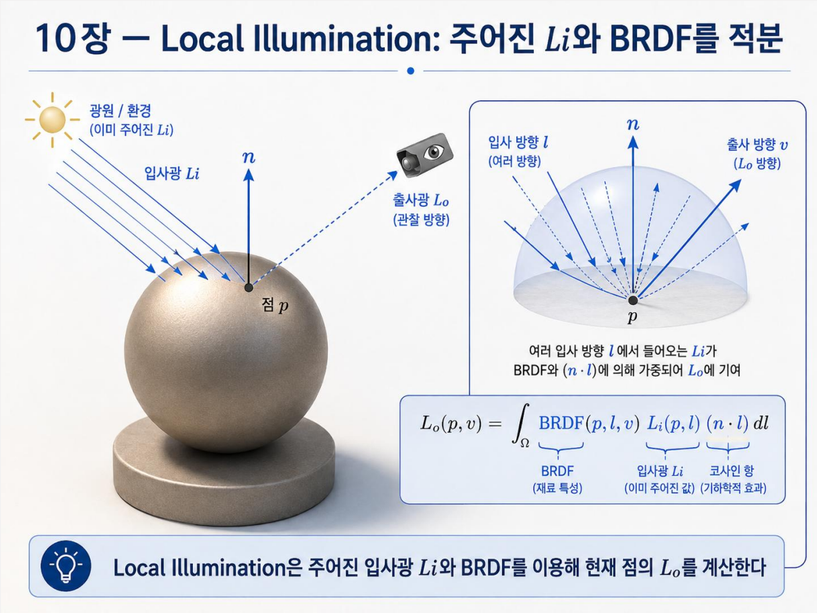
9장에서 다룬 항은 반사율 방정식을 기준으로 위에서 푸른빛으로 강조한 항이 됩니다.
빛이 어디서 들어오는지에 대해서는 다루지 않았습니다.

10장 · Local Illumination

10장에서 다룬 것은 조명 항

$$L_o(p, v) = \int_{l \in \Omega} f(l, v)\, L_i(p, l)\, (n \cdot l)^+ dl$$

10장은 L_i를 알고 있는 값 (프로그램기준으로는 알 수 있는 값)으로 놓고 반사율 방정식을 푸는 방법에 대해 다루었습니다.
점광원/방향광만을 고려하는 경우에는 적분은 유한한 합으로 나눌 수 있고
환경 조명인 경우에는 미리 적분해둔 텍스쳐 (또는 SH등)를 읽어 옵니다.
L_i가 유한하거나, 장면의 변화/상태와 무관하게 주어질 수 있다는 가정이 지역 조명의 핵심 구분 방법입니다.



10장 • 환경 조명이 저장한 것

환경 맵

무한히 먼 곳에서 오는 입사 광도를 방향의 함수로 저장합니다.
위치는 인자가 아닙니다.

방사 조도 맵

코사인 가중 적분을 미리 끝내 둔 확산 반사 전용 조명입니다.
실행 시에는 법선으로 한 번 조회합니다.

구면 조화

Image-Based-Light 대신 낮은 주파수 조명을 아홉 개 남짓한 계수로 압축합니다.
저장과 보간이 싸고 회전도 쉽습니다.

셋 다 “빛이 어디서 오는가”를 장면과 무관한 함수 로 고정합니다.
물체가 움직여도, 옆에 빨간 벽을 세워도 이 함수는 변하지 않습니다.

11장 · 다룰 내용

항목	지역 조명 모델의 가정	실제 장면
광원까지의 거리	무한히 멀다 — 방향만 남는다	유한하고, 위치마다 방향이 다르다
물체 간 차폐	지정한 광원 방향에만 그림자	모든 입사 방향에서 간접광이 차폐 될 수 있음
물체 간 상호 반사	빛은 물체 표면에서 한 번만 튕긴다	벽의 색이 바닥으로 번진다 (간접광)
조명	조명은 고정된 텍스처거나 punctual light	물체가 움직이면 조명도 함께 바뀐다

렌더링 방정식과 전역 조명

9, 10장에서 다루지 않은 렌더링 방정식과 전역 조명에서 필요한 광선 투사함수에 대해 다룬다는 내용 다듬어서 쓰기

11.1 The Rendering Equation

반사율 방정식에서 렌더링 방정식으로

9·10장 — 반사율 방정식

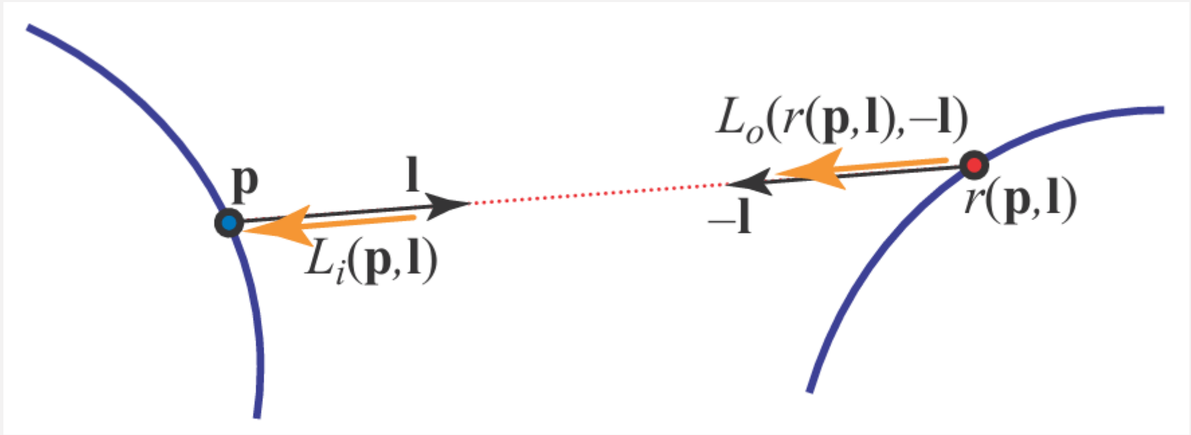
$$L_o(p, v) = \int_{l \in \Omega} f(l, v)\, L_i(p, l)\, (n \cdot l)^+ dl$$

11장 — 렌더링 방정식

$$L_o(p, v) = L_e(p, v) + \int_{l \in \Omega} f(l, v)\, L_o(r(p, l), -l)\, (n \cdot l)^+ dl$$

9장과 10장에서 다룬 반사율 방정식은, 1986년 Kajiya가 제시한 렌더링 수식의 제한된 특수사례.
빛이 들어와서 반사한다고 했지만, 현실의 물체는 스스로 빛을 내는 경우도 있으며 (빨간 항)
물체에 들어오는 빛은 어디 허공에서 생성되는게 아니라 어딘가에서 방출되어 옵니다. (파란항)

광선 투사 함수 $r(p, l)$



r(p, l)은 점 p에서 방향 l로 광선을 쏘았을때 가장 먼저 만나는 표면 위의 점을 돌려줍니다.
기존의 반사방정식에서 다룬 빛이, 어느 지점에서 얼마나 오는지 알기 위해서는 r(p, l)지점을 찾아야겠죠?

r(p, l)지점에서 p방향으로 오는 빛(Li)은 Lo(r(p,l),–l) 이 됩니다.

r인이유 : ray casting function입니다

- 반투명한 물체에 대해서는 다루지 않습니다. BRDF함수부터가 빛의 반사를 가정한 함수지 빛의 투과를 고려하지 않았습니다.
- 그렇기 때문에 ray cast하는 방향에서 오는 빛은 언제나 단일 지점임을 보장할 수 있습니다.
- 반투명한 물체는 14장에서 다룰 예정이니 잠시동안은 불투명한 세상을 상상합시다.

렌더링 방정식이 의미하는것

$$L_o(p, v) = L_e(p, v) + \int_{l \in \Omega} f(l, v)\, L_o(r(p, l), -l)\, (n \cdot l)^+ dl$$

$L_e(p, v)$ 는 이미 10장에서 광원의 방출을 다루는 방법을 보았으므로 생략합니다.
그런데 $L_o(p, v)$ 값을 구하려면 반구 전체 방향으로 ray cast를 시도해서 $L_o(r(p, l), -l)$ 를 구해야합니다.

그렇다면 각 L_o 값들을 구하기 위해서 다시 해당 지점들에서 반구 전체 방향으로 ray cast를 시도하고...
모든 빛이 흡수될때까지 재귀적으로 계산해야 합니다

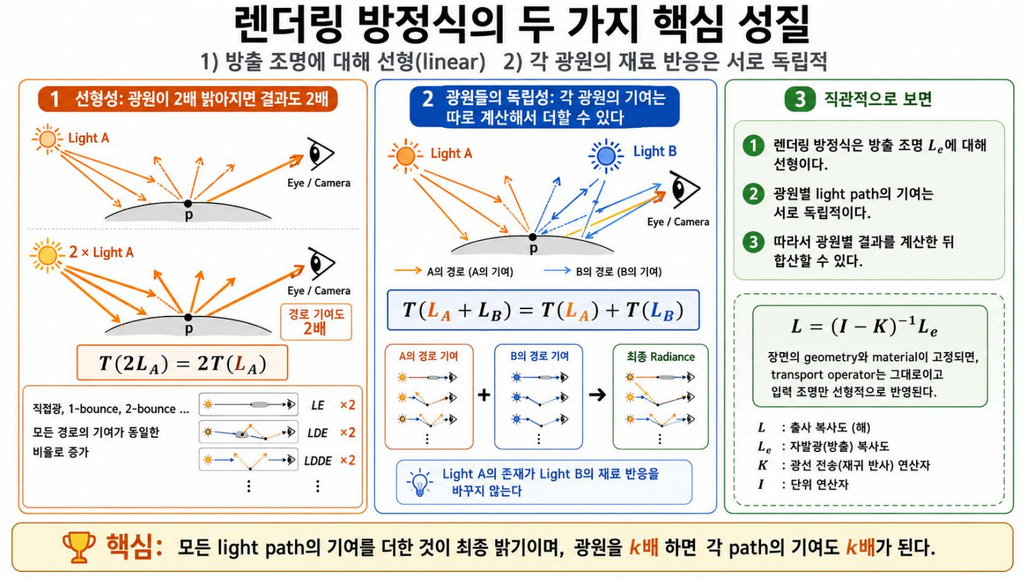
저 L_o 계산하는게 해석적으로 풀리진 않을 것 같은데 아마 없겠죠? AI 피셜로는 제2종 프레드홀듬 적분 방정식이라는데 모르겠음

렌더링 방정식의 성질

모든 광원의 emission이 2배가 되면, 음영 처리 결과가 2배 밝아지고
각 Light Source에 대한 재료의 반응은 다른 Light Source와 독립적입니다.

Light는 기본적으로 Additive하게 작동하기 때문에 (가산혼합)
광원별 결과를 계산 한 뒤 합산할 수 있음을 보일 수 있습니다.

PS 하셨던 분들은 기대값의 선형성 태그라던가, 그런곳에서 본 선형성이라는 단어가 와닿을듯



실시간 렌더링에서 주로 지역 조명만 사용하는 이유

렌더링 방정식을 프레임마다 전부 풀 수는 없다

목표로 하는 픽셀 하나를 렌더링하기 위해 씬의 모든 지점에 대해 BRDF함수를 평가하고 적분하는 것은 실시간 내에 처리하기 어렵습니다.
정밀도를 낮추기 위해 반구를 샘플링 하는 비율을 낮추고, 낮은 광량을 가진 Light Pass계산을 생략하더라도
런타임 (60프레임 기준 16ms)환경에서 작동하기에는 현실적인 제약이 아닙니다.

GI는 전통적인 단일 Rasterization Pass의 Fragment Shader만으로는 완전한 GI를 계산하기 어렵다.

Rasterization 파이프라인의 Fragment Shader는 현재 rasterize된 surface point를 독립적으로 shading하는 데 최적화되어 있습니다.
반면 Global Illumination은 현재 점의 조명을 계산하기 위해 장면의 다른 surface point들의 조명·가시성 정보가 필요합니다.
이러한 지점들은 현재 화면에 fragment를 생성하지 않거나, 같은 pass에서 아직 shading 결과가 존재하지 않을 수 있으므로,
단일 Fragment Shader pass만으로 GI의 surface 간 light transport를 해결하기 어렵습니다.

(지역 조명은 Li가 주어진 값이기 때문에, 대부분의 데이터를 fragment 셰이더에서 알 수 있습니다)

- 그림자 처리도 그렇지만, 위와 같은 이유로 런타임 GI는 기본적으로 DrawCall이 내려지기전, 미리 계산하는 과정을 거치게 됩니다.
- 그게 이미 구성된 씬에서 미리 구워두는 방법이든, 런타임에 Shadow Map처럼 다른 정보를 굽게 되든…

Heckbert의 광자 경로 표기법

기호

- L — 광원(light), 경로의 출발점
- E — 눈(eye), 경로의 도착점
- D — 확산 반사(diffuse)
- S — 정반사(specular)

연산자

- $k*$ — 0회 이상 반복
- $k+$ — 1회 이상 반복
- $k?$ — 0회 또는 1회
- $(k|k')$ — 둘 중 하나

는 UseCase들을 보고 기호에 매핑했을때 이해가 더 빠를 것 같기 때문에 지금은 넘깁니다.

전역 조명을 실시간에 넣는 두 가지 전략

전략 1

단순화 — 식의 일부에 강한 가정을 건다

조명이 모든 방향에서 균일하다고 보거나, 반사가 확산뿐이라고 보거나, 빛이 멀리 가지 않는다고 보는등의 제약으로 식을 단순화 합니다.

전략 2

사전 계산 — 변하지 않는 것을 미리 푼다

정적인 기하 구조와 정적인 광원 사이의 빛 전달을 미리 풀어 라이트 맵이나 계수로 저장하고, 실행 시에는 조회와 보간만 합니다.

어떻게 하드웨어에 호환되는 기법으로 단순하게 만들거나 미리 구워두지 (또는 둘다 해서 퉁치기) 같은 고민이 요즘 GI기술의 집합체
11.2장에서는 그나마 덜 다루고, 11.3장부터 적극적으로 위 두 주제를 다룰 예정입니다.

최근 GI 기술에는 최근에 풀었던 값을 캐싱해서 사용하는 방법에 대해서도 다루고, 별도의 하드웨어 유닛의 도움을 받아 병목을 해결하기도 합니다.

일반적인 전역 조명 알고리즘

10장에서는 반사율 수식을 푸는 다양한 방법에 중점을 두고, 들어오는 광도 Li의 분포를 가정하며 그것이 음영에 어떻게 영향을 미치는지 분석했었다.
이번 장에서는 방금 설명한 렌더링 수식을 풀도록 설계된 알고리즘 2개를 제시한다.

한 지점에 도달하는 광도는 다른 지점에서 방출되거나, 반사된 광도가 된다.

퀄리티가 굉장하지만, 기존에 설명했듯 런타임에 계산하기에는 무리가 있다.

그럼에도 해당 과정을 다루는 이유는
1. 정적이거나 / 부분적으로 정적인 씬에서 이런 알고리즘으로 사전처리해 나중에 사용할 수 있도록 저장할 수 있기 때문
2. 실시간 솔루션을 설계할때도 올바른 방법이 무엇인지 비교하거나, 유사한 유형의 추론을 적용할 수 있기 때문

특히 유사한 유형의 추론을 적용한다는 부분에 집중해서 일반적인 전역 조명 알고리즘을 보면 되겠다.
해당 파트에서 나오는 개념과, 이를 구현하기 위해 사용하는 아이디어들은 이후 각기 다른 GI 구현체들의 기본 개념이 될 수 있다.

렌더링 수식을 푸는 두가지 일반적인 방법은 유한 요소 해석(방법) 과 몬테 카를로 방법이 있다.
라디오시티(Radiosity)는 유한 요소 해석을, 다양한 레이트레이싱(RayTracing)은 몬테 카를로 방법을 기반에 두고 있다.

- 유한 요소 해석 : 공간을 작은 요소들로 분할하고, 각 요소별 물리 법칙을 식으로 세워 연립방정식을 푸는 형태. 고체, 유체, 열역학 등에서 사용한다고 함
- 몬테 카를로 방법 : 유명한걸로는 원의 면적을 계산하는것이 있음. RayTracing을 몬테 카를로 방법을 사용하면 임의의 방향으로 Ray를 쏘는것을 상상하면 된다

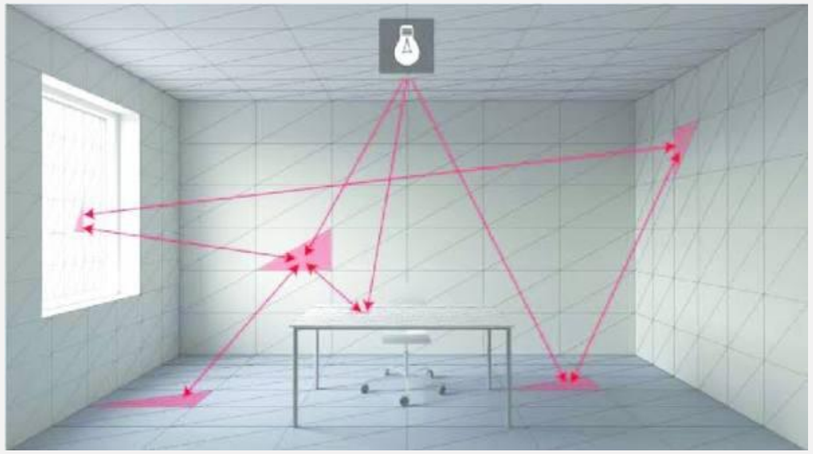
Radiosity

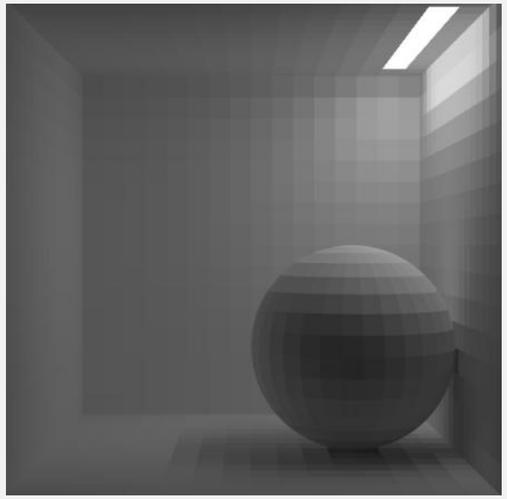
모든 표면이 확산 반사만 한다고 가정하면, 한 점의 밝기는 보는 방향과 무관해집니다. (Lambertian)

방향 의존성이 사라지면 미지수는 방향의 함수가 아니라 위치만의 함수가 됩니다.

장면을 패치로 나누고 패치들 사이의 에너지 교환을 연립방정식으로 풉니다.

열전달 공학에서 넘어온 접근으로, 1984년 코넬대에서 그래픽스에 처음 적용되었습니다.





리얼타임 렌더링 4판 · 11장 전역 조명 · 1차시

Radiosity

패치 i의 방사도

$$B_i = B_{ei} + \rho_i \sum_j F_{ij} B_j$$

F_{ij}는 패치 i를 떠난 에너지 중 패치 j에 도달하는 비율입니다. 재질과 무관한 순수한 기하 문제입니다.

형태 계수 계산이 전체 비용을 지배하고, 패치 수의 제곱에 비례해 늘어납니다.

점진적 세밀화는 가장 밝은 패치부터 순서대로 에너지를 뿌려, 중간 결과를 미리 보여 줍니다.

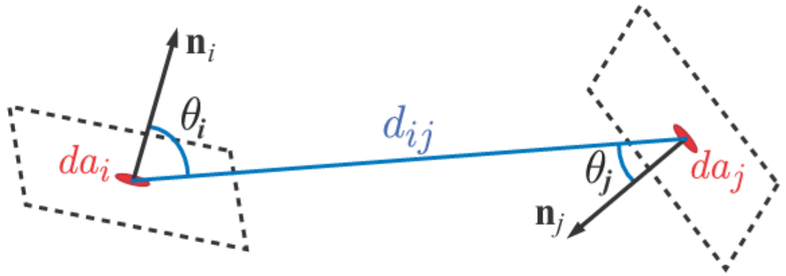
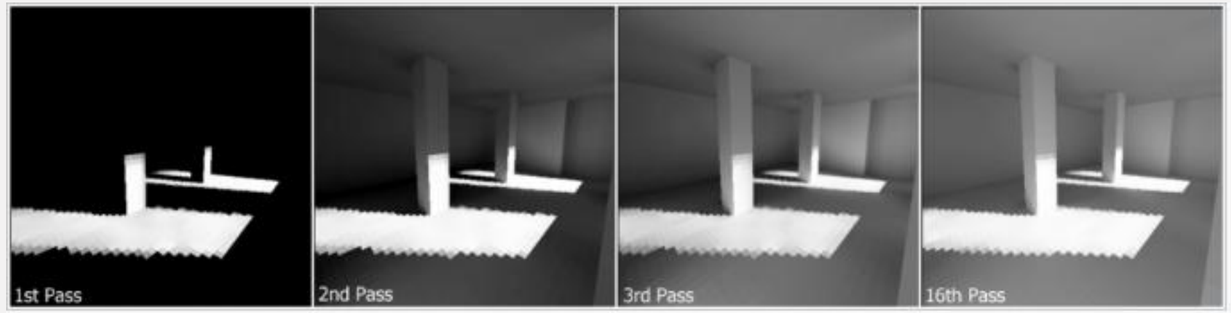


Figure 11.4. The form factor between two surface points.



As the algorithm iterates, light can be seen to flow into the scene, as multiple bounces are computed. Individual patches are visible as squares on the walls and floor.

리얼타임 렌더링 4판 · 11장 전역 조명 · 1차시

Radiosity의 장단점

얻은 것

시점과 무관한 해

한 번 풀어 두면 카메라를 어디에 두어도 그대로 쓸 수 있음.
결과가 정점이나 텍셀에 저장되므로 실행 시 비용이 거의 없음.

잃은 것

정반사와 메시 독립성

방향정보를 손실하고 표면 patch가
받은 빛이 균일하게 반사된다 가정하였기 때문에
거울과 하이라이트를 표현하지 못함 (Diffuse 경로에만 특화)
메시 분할 정도가 품질이라 그림자 경계에서 계단이 드러남.

Ray Tracing

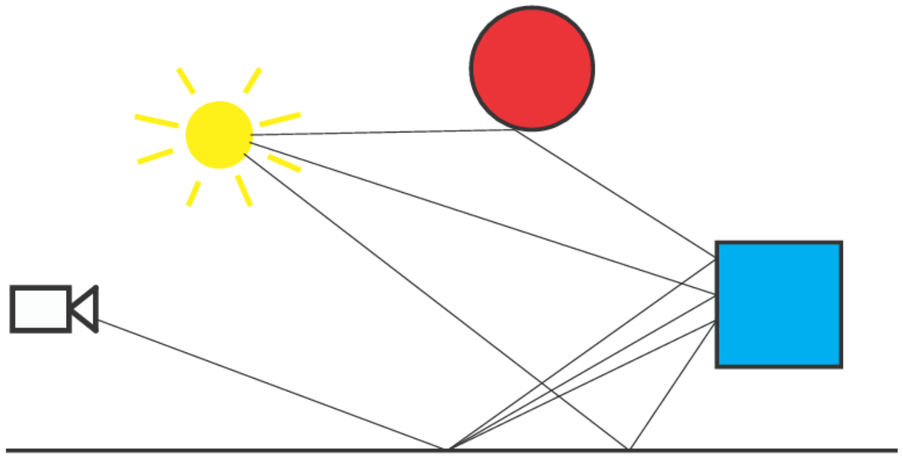


Figure 11.5. Example paths generated by a path tracing algorithm. All three paths pass through the same pixel in the film plane and are used to estimate its brightness. The floor at the bottom of the figure is highly glossy and reflects the rays within a small solid angle. The blue box and red sphere are diffuse and so scatter the rays uniformly around the normal at the point of intersection.

Ray Tracing의 장단점

얻은 것

재질과 상관없이 빛의 반사를 시뮬레이션 할 수 있음

RadioSity는 재질이 가진 glossy함이나, Mirror같은 반사 로브의 특징을 살릴 수 없었으나, Ray Tracing은 BRDF의 형태에 크게 구애받지 않고 사용할 수 있는 기술이 된다.

엄청난 샘플링 비용을 줄이기 위해
대부분의 빛이 방향이 들어오는 방향으로 더 많은 광선을 쏘거나
BRDF 함수의 특성에 따라 샘플링 전략을 변경하는 등의 기술들이 연구되었고, 연구되는 중
이후 GI파트에서도 약간 다루겠지만, 적은량의 샘플링을 어떻게 커버할지도 중요한 주제

잃은 것

샘플링 비용과 노이즈

모든 방향의 Ray를 Trace할 수 없고, trace의 수가 줄어들수록 노이즈의 분산이 커지게 된다.
동일한 방법으로 샘플링한다고 가정했을때 노이즈(표준편차)를 2배 줄이기 위해서는, 4배 더 많은 샘플이 필요하게 된다.

Radiosity의 장점으로 쓴 시점과 무관한 해는 RayTracing의 단점으로 쓰진 않았습니다.
Unity / Unreal에서 static한 오브젝트에 대해 미리 굽는 과정에서는 RayTracing을 사용하고 있고 (거울등의 처리때문에라도), 이때는 광원 (Light Sources)에서 Ray를 쏘아 장면에 반영되는 광량을 계산해, 물체의 표면에 반영합니다.

1차시 마무리

주변 폐색 하다가 어디서 끊어야할지도 모르겠고 준비도 미흡해서 일단 여기서 끊기
앞 파트 발표 시간 얼마나 소요되는지 보고 판단하기

오늘의 순서 - 2차시

00 서론 — 지역 조명은 어디서 티날까?

01 주변 폐색 — 11.3

02 주변 폐색 — 11.3 * 오타 아닙니다. 주변 폐색만 하루종일 합니다

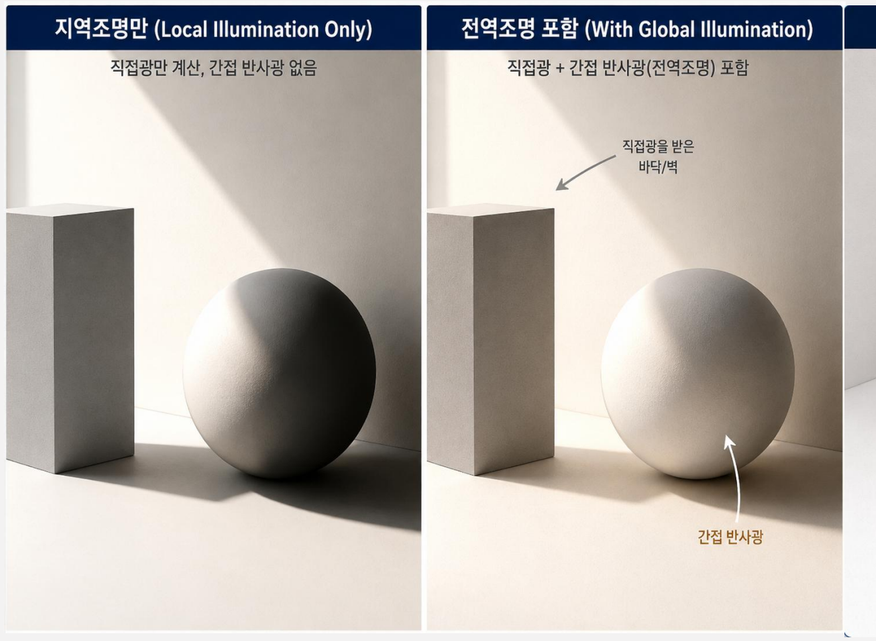
03 방향성 폐색 — 11.4

서론

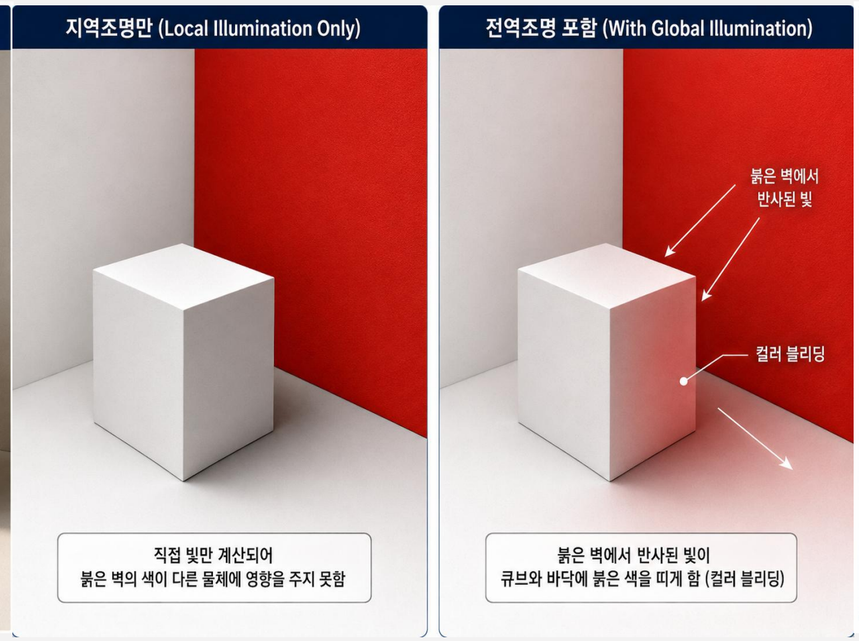
11장 1차시에 다룬 일반적인 전역 조명 알고리즘은 무겁지만,
7장에서 다룬 그림자와 10장에서 다룬 지역조명만으로는 물체 표면에서 반사된 간접광을 표현하지 못합니다.

아래 예시는 간접광이 만드는 대표적인 시각적 효과 3가지를 설명합니다. * 이미지는 AI 생성이라 실제 렌더링 결과와 차이가 있을 수 있음

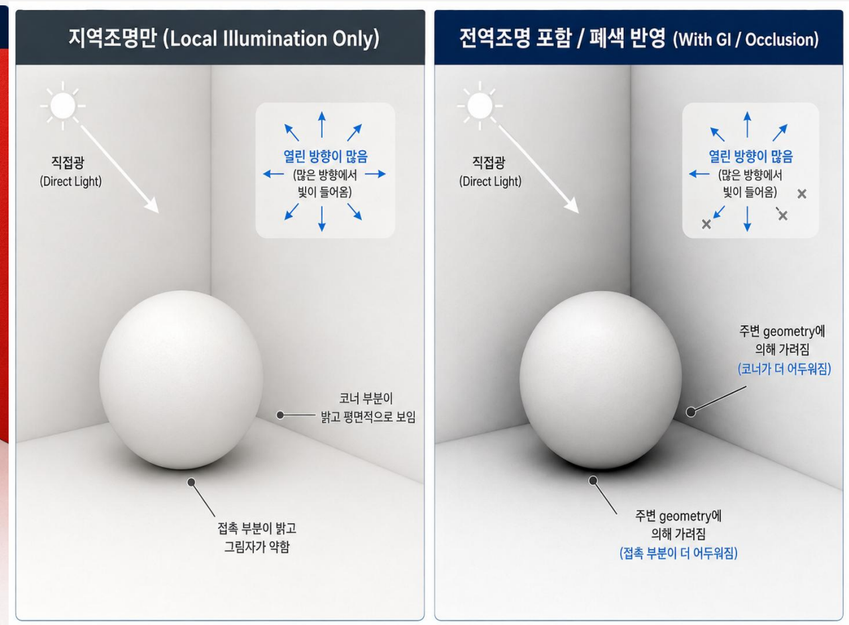
Indirect Fill : 직접광이 닿지 않더라도, 간접광을 닿아 밝아질 수 있음



Color Bleeding : 표면의 색상이 다른 표면의 반사광의 영향을 받음

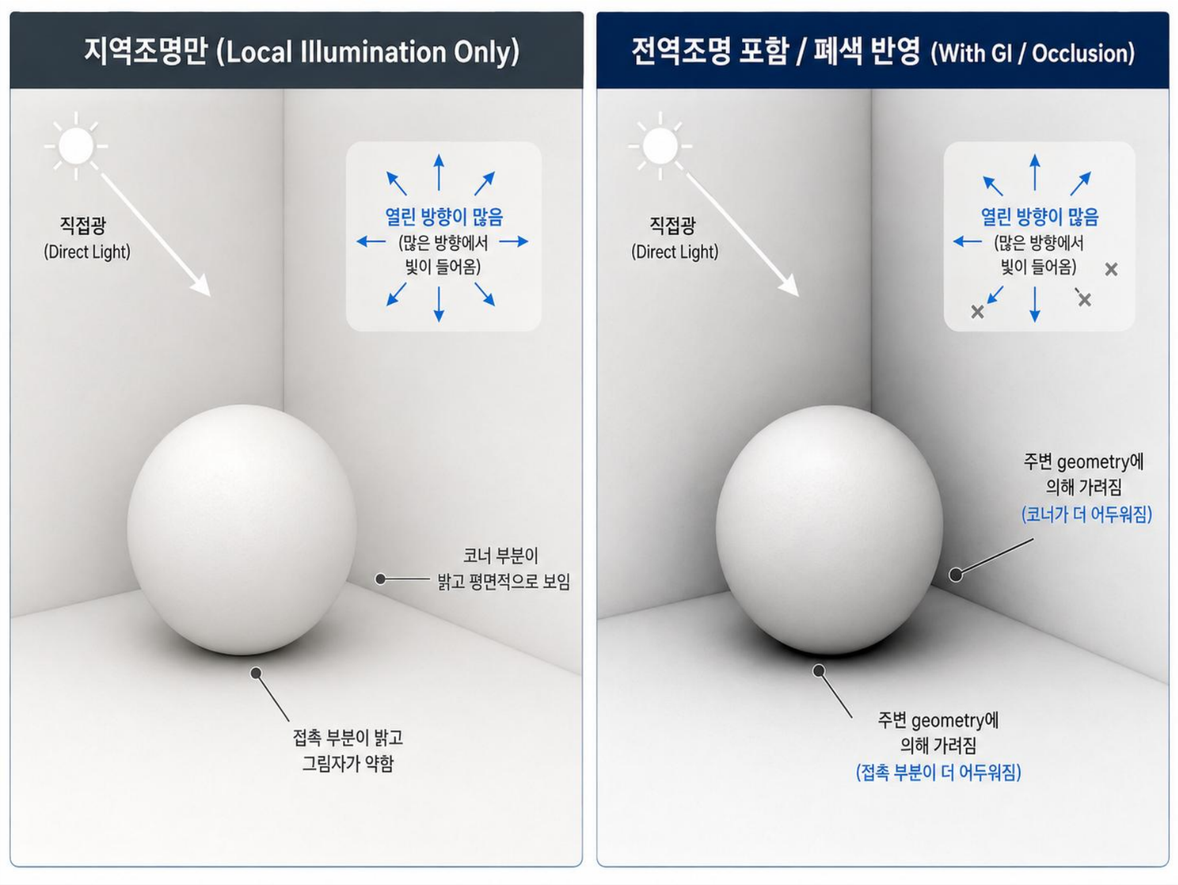


Occlusion : 빛이 들어오는 영역이 좁아지는 경우 상대적으로 어두워짐



서론

11.3 ~ 11.4는 주변광의 차폐를 어떻게 처리할 것인지에 대한 맥락을 담고 있습니다.



주변 폐색 (Ambient Occlusion)

1차시에 다룬 일반적인 전역 조명 알고리즘은 런타임에 사용하기에는 계산 비용이 너무 크고 복잡합니다.

이번 장에서 다룰 주변 폐색 (Ambient Occlusion)은 기본적이면서도 대표적인 GI 효과입니다.
영화 진주만 (2001)에서 비행기에 사용하기 위한 환경 조명의 품질을 개선하기 위해 개발되었다고 합니다.

매번 1900년대, 심하면 1800년대 사람들 따라가느라 자괴감 느껴지고 벅찼는데, GI는 실사용 영역에서는 2000년대 기술들 나와서 좀 감동

이때도 많은 영화의 렌더링에는 RayTracing이 사용되고 있었지만, 진주만은 (그 당시 기준으로는) 씬이 복잡해서 메모리 문제를 해결하기 위한 다른 수단이 필요했습니다.

* RayTracing 기반 렌더링을 위해서는 씬 전체가 메모리에 올라가야만 했습니다. 반사광은 반사되는 위치의 재질 정보에 따라서도 달라지기 때문
* 하지만 단순 차폐 검사는 오브젝트 단위로 미리 구워둘 수 있고, 재질의 변경에도 영향을 받지 않습니다. 왜 그런지는 이제 알아보도록 합시다.



진주만 영화에서 사용된 AO 효과의 예시 (출처)

주변 폐색 이론 (Ambient Occlusion Theory)

주변 폐색 이론의 이론적 배경은 반사율 수식에서 유도할 수 있습니다.
단순화를 위해 Lambertian 표면을 가정합니다.

이러한 표면에서 나가는 radiance L_o 는 surface irradiance E에 비례합니다. *irradiance = 복사조도, 표면이 받은 빛 (에너지)의 양

반사율 방정식

$$L_o(p,v) = \int_{l\in\Omega} f\,(l,v)L_i(p,l)(n\cdot l)^+ dl$$

Lambertian BRDF

$$f(l,v) = \frac{\rho_{ss}}{\pi}$$

Surface irradiance

$$E(p,n) = \int_{\Omega} L_i\,(p,l)(n\cdot l)^+ dl$$

$$L_o(p,v) = \frac{\rho_{ss}}{\pi} E(p,n)$$

* 10장에서 다룬 램버시안

여기에 더해, **단순화를 위해 들어오는 모든 방향 l에 대해, incoming radiance가 일정하다고도 가정**해봅시다. $L_i(p,l) = L_A$

*위 가정을 diffuse illumination(확산 조명)만 존재하는 환경이라고도 할 수 있습니다.

한글본(4판)에서는 *다시 말하지만 단순함을 위해 어쩌구*로 써놨는데
Lambertian 표면과 incoming radiance가 일정하다는건 서로 다른 가정 두개입니다.

$$E(p,n) = \int_{\Omega} L_A\,(n\cdot l)^+ dl$$

$$E(p,n) = L_A \int_{\Omega} (n\cdot l)^+\, dl = \pi L_A$$

* 수식 (11.6)

일정하고 균일한 조명이 Lambertian표면에 들어오면 irradiance (결과적으로는 radiance도) 표면 위치와 법선(normal)에 의존하지 않으며, 오브젝트 전체에 일정합니다.
이는 물체의 형태에 따른 명암이 보이지 않게 되며 flat하게 보이게 됩니다.

주변 폐색 이론 (Ambient Occlusion Theory)

방금 유도한 식은 가시성을 고려하고 있지 않습니다.
반구의 모든 방향에서 동일한 빛이 들어오더라도, 오브젝트 본인에게 가리거나 다른 오브젝트에 의해 빛이 차단될 수 있습니다.

이런 방향에서는 L_A 가 아닌 다른 radiance가 들어오게 됩니다.
여기서 또 **단순화를 위해 가려진 방향에 대한 radiance는 0이라고 가정**합시다.

차단된 오브젝트에서 오는 반사광 / 해당 방향에서 오는 산란된 빛 과 같은 요소를 무시하지만 결과를 단순화 할 수 있습니다.
이는 Cook과 Torrance가 처음 제안한 다음의 식을 얻게 됩니다.

$$E(p,n) = L_A \int_\Omega v\,(p,l)(n \cdot l)^+ dl$$ * 수식 (11.7)

여기서 v(p, l)은 p에서 l방향으로 투사되는 광선이 차단되면 0이고 차단되지 않으면 1인 가시성 함수(visibility function)입니다.

이 가시성 함수의 정규화된 코사인 가중치 적분(normalized, cosine-weighted integral)을 이번 장에서 다룰 주변 폐색(AO)이라고 합니다.

$$k_A(p) = \frac{1}{\pi} \int_\Omega v\,(p,l)(n \cdot l)^+ dl$$ * 수식 (11.8), 주변 폐색

완전히 가려진 표면의 경우 0, 어떤 방향도 가려지지 않은 경우 1이 됩니다.
n·l=cosθ 이기 때문에, 빛이 들어오는 방향에 따라 가중치가 다르게 됩니다 (코사인 가중치)

주변 폐색 이론 (Ambient Occlusion Theory)



주변 폐색 k_A을 정의했기 때문에, 폐색을 고려한 irradiance를 구할 수 있게 됩니다.

$$E(p,n) = L_A \int_\Omega v\,(p,l)(n \cdot l)^+ dl \qquad k_A(p) = \frac{1}{\pi}\int_\Omega v\,(p,l)(n \cdot l)^+ dl$$

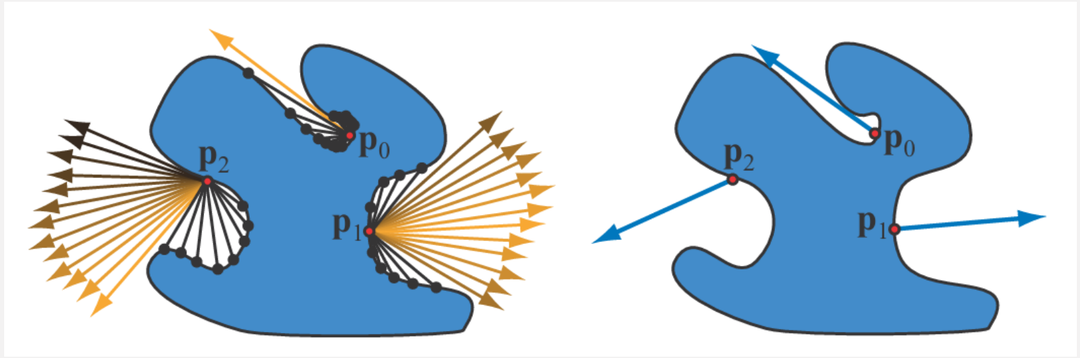
$$E(p,n) = k_A(p)\pi L_A$$ * 수식 (11.9)

이제 표면 위치 p가 바뀌면 주변 폐색 k_A이 변경되기 때문에, irradiance가 위치에 따라 변경되게 됩니다.
이는 그림의 예시에서 볼 수 있듯이 훨씬 현실적인 결과로 이어집니다.

오목한 표면 위치는 k_A값이 낮기 때문에 어두워지게 됩니다. (그림의 p0과 p1 비교)

가시성 함수 v(p, l)이 통합될때 코사인(n·l)값에 따라 가중되어 표면 방향도 영향을 미칩니다.
p1과 p2는 동일한 폐색되지 않은 각도를 가졌음에도 p1이 더 높은 값을 가지게 됩니다.

이후 다룰 내용부터는 간결한 표현을 위해 표면위치 p에 대한 의존성은 명시적으로 표시하지 않습니다.

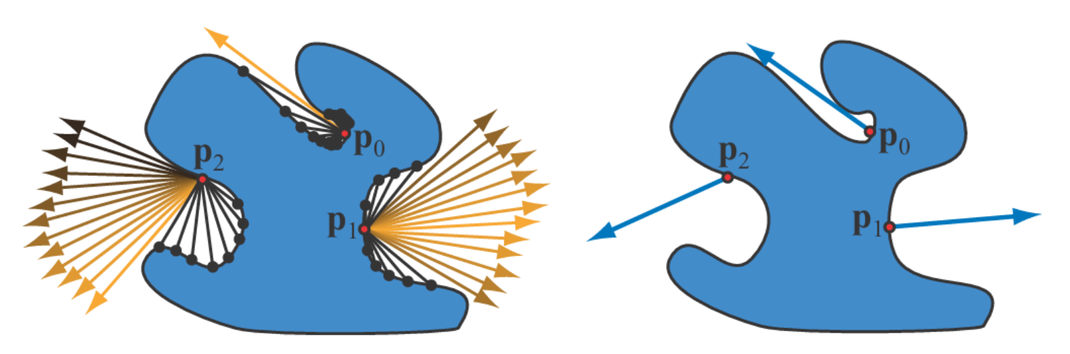


주변 폐색 이론 (Ambient Occlusion Theory)

주변 폐색 k_A 이외에도 Landis는 폐색되지 않은 방향의 코사인 가중 평균을 이용하여
구부러진 법선(bent normal)으로 알려진 폐색되지 않은 평균 방향도 계산하였습니다.

$$n_{bent} = \frac{\int_{l\in\Omega} l\, v(l)(n\cdot l)^{+} dl}{\left\| \int_{l\in\Omega} l\, v(l)(n\cdot l)^{+} dl \right\|}$$ * 수식 (11.10)

Bent normal은 셰이더 처리에서 실제 geometr의 normal 보다 더 정확한 결과를 만들기 위해 사용될 수 있습니다.
(실제 도달 가능한 라이트맵, irradiance volume, 그리고 이후 다룰 GI 결과들의 조회 등…)



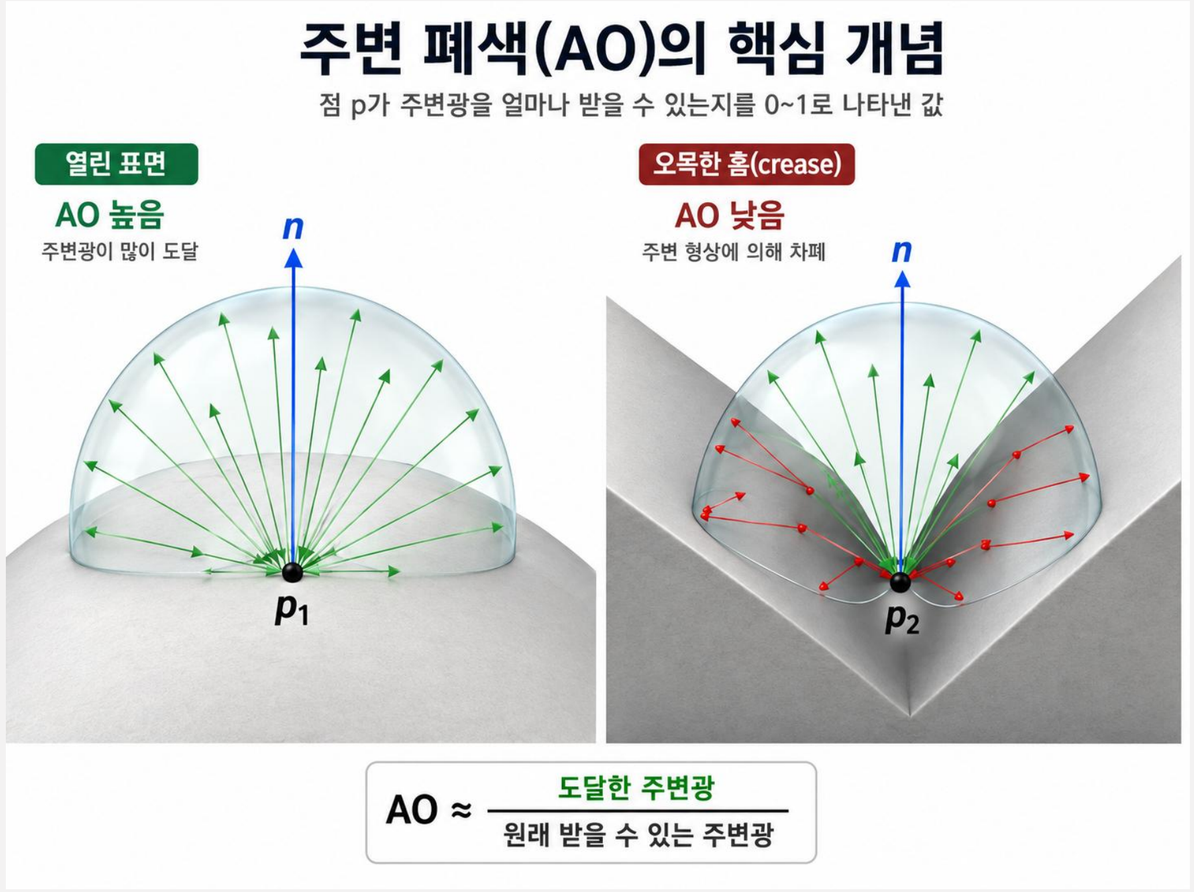
* p0, p1, p2에서 뻗어나가는 파란색 선분의 방향이 bent normal, 실제 물체의 normal 방향과 다를 수 있다

주변 폐색 이론 (Ambient Occlusion Theory) - 요약

주변 폐색은 주변 형상의 영향으로 주변광을 얼마나 받을 수 있는지를 0~1로 나타낸 값이 됩니다.

$E(n) = k_A \pi L_A$ * 수식 (11.9)

아니 이렇게 수식이랑 싸워서 얻은결론이
이거 저거 퉁치고 나면 빛이 얼마나 들어오는지 그냥 scalar하나랑 vector하나 남는다고



퉁치고 넘어간 가정 3개

표면은 Lambertian – (각도에 관계 없이 균일한 outgoing radiance)
Incoming radiance도 균일 (diffuse illumination만 존재)
차폐면 (또는 차폐 방향)에서 오는 radiance는 0



리얼타임 렌더링 4판 · 11장 전역 조명 · 2차시

가시성과 차폐도(Visibility and Obscurance)

한글판은 Obscurance를 모호성이라고 번역했는데, 차폐도가 더 맞는 번역같습니다
뭐가 모호하다는건지 진짜 보면서 너무 모호했어...

주변 폐색 계수를 계산하기 위해서는 가시성 함수 v(l)가 필요했습니다.

$$k_A = \frac{1}{\pi}\int_{\Omega} v\,(l)(n \cdot l)^{+} dl$$

* 수식 (11.8), 주변 폐색

오브젝트의 표면p에서 l방향으로 ray를 쏘아서, 다른 물체에 닿게 되는지를 기반으로 v(l)을 정의하는건 간단하지만,
이는 닫힌 공간에서 결과가 좋지 못합니다.

다양한 오브젝트가 있는 닫힌 방에서는 표면에서 발사한 모든 광선은 무언가에 닿기 때문에
V(l)은 언제나 0이 되고, 자연스럽게 k_A 값도 0이 되는 문제가 생깁니다.

이런 닫힌 공간에서는 물리적인 가시성을 시뮬레이션 하지 않고 주변 폐색(AO)의 외형을 재현하려는 경험적인 접근 방법이
더 나은 결과를 내는 경우가 많습니다.

가시성과 차폐도(Visibility and Obscurance)

Miller의 접근성 셰이딩 개념에서 영감을 받아 Zhukov, Iones, Kronin는
가시성 함수 v(l)을 거리 매핑 함수 p(l)로 대체해, 주변 폐색 계산을 수정하는 obscurance라는 개념을 도입합니다.

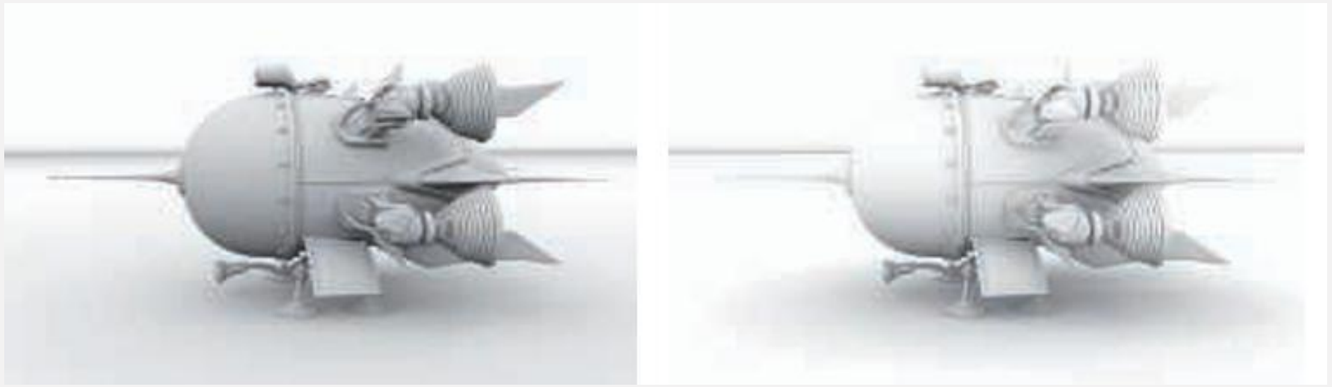
먼지가 쌓이거나, 변색된 표면은 주로 닦는 물체가 안 닿아서 생긴다는 내용
Accessibility를 표면p가 구형 probe에 얼마나 쉽게 닿을 수 있는가로 정의했다고 함

$$k_A = \frac{1}{\pi}\int_{\Omega} p\,(l)(n \cdot l)^{+} dl$$

* 수식 (11.11), 주변 폐색 – 거리매핑 함수

두 가지 유효한 값 (Shadow때도 그랬듯이, 0과 1)이 있는 v(l)과 달리, p(l)은 표면에 도달하기 전까지 광선이 이동한 거리를 기반으로 하는 연속함수입니다.
교차 거리가 0일때 0이며, 미리 지정한 $d_{\max}$ 보다 먼 거리에서 교차하거나 아예 교차하지 않는 경우는 1이 됩니다.
$d_{\max}$를 넘는 교차 거리는 검사할 필요가 없기 때문에, k_A의 계산 속도를 상당히 높힐 수 있게 됩니다.

그림은 주변 폐색과 주변 차폐도(ambient obscurance)의 차이를 보여줍니다.
주변 폐색은 먼 거리에서도 교차점이 감지되어 k_A 값에 영향을 미치기 때문에, 상대적으로 더 어둡게 나옵니다.



가시성과 차폐도(Visibility and Obscurance)

Obscurance의 거리 감쇠를 실제 광전달 현상과 연결해 물리적으로 설명하려는 시도도 있었지만,
차폐 효과는 단순히 occluder까지의 거리만으로 결정되지 않으므로 Obscurance 자체는 물리적으로 정확한 모델이 아닙니다.

그럼에도 불구하고 보는 사람의 기대에 부합하는 그럴듯한 결과를 내는 경우가 많습니다.
한가지 단점은 $d_{\max}$ 값을 수작업으로 조정해야 한다는 것입니다.

이처럼 직접적인 물리적 근거는 없지만 지각적으로 설득력 있는(perceptually convincing) 기법을 통한 절충안은 컴퓨터 그래픽스에서 흔한 일입니다.

대부분의 목표는 믿을만한 이미지를 만드는 것이기 때문에, 문제될 것은 없습니다.
이론에 기반을 둔 방법의 장점은 자동으로 동작할 수 있고, 현실 세계에 대한 추론을 통해 더 개선할 수 있다는 점입니다.

* 앞으로 다룰 많은 파트도 퉁치고 넘어갔던 부분을 어떻게 살릴지(개선할지)에 대해서 꾸준히 다룹니다.

상호 반사의 반영(Accounting for Interreflections)

AO로 생성된 결과는 시각적으로 설득력이 있지만, 완전한 전역 조명 모델(full global illumination)의 결과보다는 어둡게 나옵니다.

AO와 Full GI의 차이를 만드는 주요 원인은 상호 반사(Interreflections)입니다. 이전에 식 11.8에서 차폐된 방향에서 들어오는 radiance를 0이라고 가정했지만,
실제로는 그 방향들에서도 상호 반사에 의해 0이 아닌 radiance가 들어오게 됩니다.

이 Full GI와의 차이는 $\mathbf{k}_A$ 값을 키움으로서 보정할 수 있습니다.
가시성 함수 대신 직전에 다룬 거리 매핑 함수를 사용하면, 차폐된 방향에 대해서도 0보다 큰 값을 가지는 경우가 많아 문제를 완화하는데에 도움이 됩니다.





상호 반사의 반영 (Accounting for Interreflections)

상호 반사를 조금 더 정확하게 추적하려면 Full GI에서 다루는 재귀적인 문제를 풀어야 하므로 비용이 많이 들게 됩니다.
k_A 값을 계산하는 비용은 Full GI에 비해서 훨씬 저렴하지만, 과도하게 어두워지는 것을 막기 위해서는 누락된 빛을 어떤 형태로든 포함시키는 것이 좋습니다.

Stewart와 Langer는 AO가 가정했던 Lambertian surface와 diffuse illumination 환경에서는
한 점 p에서 보이는 주변 표면들의 radiance가 자신의 radiance와 비슷하다는 경향이 있다는 관찰에 기반하여 재귀 없이 해석적인 식을 얻어내었습니다.
책에서는 다루지 않았지만, 식이 유도되는 과정을 따라가봅시다.

기존 AO에서 차폐되지 않은 부분을 통해 받는 irradiance는 다음과 같습니다. $k_A \pi L_i$ * 수식 (11.9) 참고

차폐 방향에서의 incoming radiance는 현재 점의 outgoing radiance와 비슷합니다 (같다고 가정) $L_{blocked} \approx L_o(p)$

막힌 영역의 cosine-weighted 비율이 1-k_A 이기 때문에 차폐 방향에서 받은 irradiance는 다음과 같습니다. $\pi(1 - k_A)L_o$

따라서 전체 irradiance는 이렇게 정리 할 수 있고 $E = \pi k_A L_i + \pi(1 - k_A)L_o$

$L_o = \frac{\rho_{ss} E}{\pi}$ *Lambertian surface radiance

Lambertian surface를 가정했기 때문에, outgoing radiance항을 다시 irradiance로 표현할 수 있습니다.

$$E = \pi k_A L_i + \pi(1 - k_A)\frac{\rho_{ss} E}{\pi}$$

해당 식을 정리하면 다음과 같은 식을 찾을 수 있습니다.

$$E = \pi k_A L_i + \rho_{ss}(1 - k_A)E$$

$$E - \rho_{ss}(1 - k_A)E = \pi k_A L_i$$

$$E[1 - \rho_{ss}(1 - k_A)] = \pi k_A L_i$$

$$E = \frac{\pi k_A L_i}{1 - \rho_{ss}(1 - k_A)}$$ * 수식 (11.12)

위 식을 기존 AO 공식의 형태로 변환하면 k_A와 ρ_{ss}값을 통해 상호 반사를 복원하여 밝아진 k'_A 값을 구할 수 있게 됩니다.

$$k'_A = \frac{k_A}{1 - \rho_{ss}(1 - k_A)}$$ * 수식 (11.13)

$$E = \pi k'_A L_i$$ * 수식 (11.12)와 (11.13)을 합치면 기존의 AO와 동일한 형태의 식을 사용할 수 있다.

상호 반사의 반영(Accounting for Interreflections)

이렇게 유도된 결과는 상호 반사를 포함한 Full GI의 결과에 시각적으로 더 가깝게 만들어 주지만,
ρ_{ss} (subsurface albedo)값에 크게 의존적입니다.

* 물론 실제 Color Bleeding처럼, 주변 표면의 색상이 실제로 반영되진 않습니다.
* 같은 albedo값이라고 퉁쳤으니까요

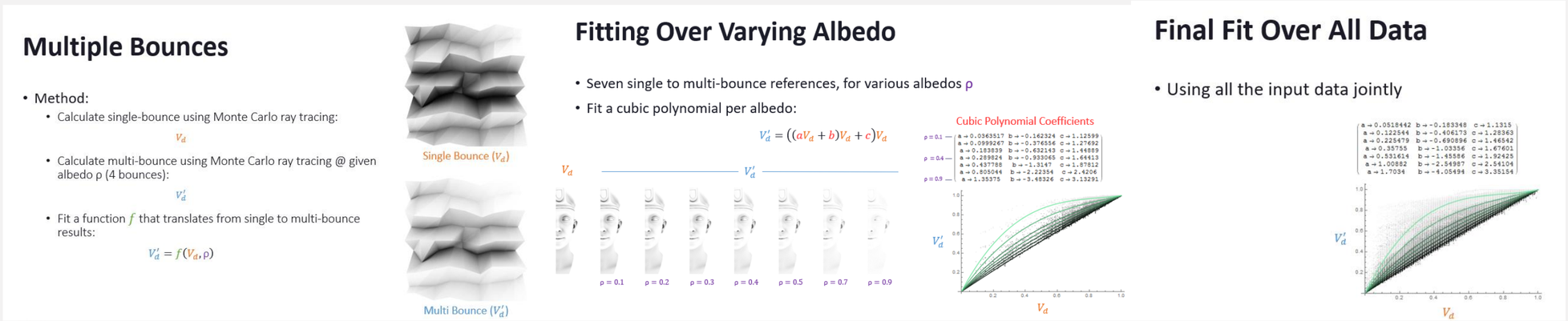
이 과정은 표면의 색상과 인접한 표면이 동일하다고 가정하여 작동하게 되며, 그렇기에 결과가 color bleeding과 비슷한 원리로 밝아지게 됩니다.

Hoffman과 Mitchell은 이 방법을 사용해 지형(terrain)에 하늘 전체에서 오는 확산 조명(diffuse lighting) 정보로 AO 정보를 미리 구워두었습니다.
이는 시간에 따라 sky color/radiance가 변경되더라도 GI를 다시 계산할 필요 없이, 변경된 값만 곱해주면 되게 되었습니다.

Jimenez는 주변 표면의 albedo가 크게 다르지 않고, diffuse illumination 환경에서 path tracing한 정보들을 모아 3차 다항식의 계수를 튜닝하였습니다.
k_A 와 ρ_{ss} 값만으로 k'_A 값을 예측할 수 있는 함수를 만들었다는 내용으로 이해하시면 될 것 같습니다.

Stewart-Langer의 방법은 해석적인 식을 통해 근사했지만, Jimemez는 실제 렌더링 환경의 path tracing정보를 기반으로 polynamial에 압축해두었기 때문에
물리적으로 올바름과 별개로 path tracing 결과와 평균적으로 더 유사하게 k'_A 값을 유추해내게 되었습니다.

*Jimenez의 방법은 블리자드 소속 인원 4명이 SIGGRAPH 2016에서 발표한 내용이네요 (출처)



리얼타임 렌더링 4판 · 11장 전역 조명 · 2차시

미리 계산된 주변 폐색(Precomputed Ambient Occlusion)

AO 계수의 계산은 시간이 많이 소요될 수 있어서, 렌더링에 앞서 오프라인으로 수행하는 경우가 많습니다.
엠비언트 오클루전을 비롯해 조명 관련 정보를 사전에 계산해두는 과정을 흔히 베이킹(baking)이라고 부릅니다.

AO를 사전 계산하는 가장 일반적인 방법은 몬테카를로 기법입니다.
1차시 RayTracing에서 짚고 넘어갔듯이, 반구 방향으로 광선을 쏘아 오브젝트의 차폐 여부를 검사하고, AO 계수를 수치적으로 계산합니다.

예를 들자면 법선 n 주위의 반구에 균일하게 분포된 N개의 무작위 방향 l을 고른 뒤, 그 방향으로 광선을 추적한다고 해봅시다.
RayCast 결과를 바탕으로 가시성 함수 v를 평가하면 AO는 다음과 같이 계산됩니다.

* 수식 (11.14), 몬테 카를로 방법 가시성 주변 폐색

$$k_A = \frac{1}{N}\sum_{i=1}^{N} v\,(l_i)(n \cdot l_i)^+$$

* 참고용 수식 (11.8), 주변 폐색

$$k_A = \frac{1}{\pi}\int_{\Omega} v\,(l)(n \cdot l)^+ dl$$

주변 차폐도(Ambient Obscurance)를 계산할 때는 광선의 거리를 최대치로 제한할 수 있으며, v의 값은 검출된 교차 거리를 기준으로 결정됩니다. (11.3.2 내용)

AO 계수의 계산에는 코사인 가중 인자가 포함됩니다 (각도에 따라 중요도가 다르다)
수식 11.14처럼 이를 직접 곱해서 넣을수도 있지만, 더 효율적으로 반영하는 방법은 중요도 샘플링(importance sampling)을 이용하는 것입니다.
반구 전체에 균일하게 광선을 쏘고 n·l을 곱하는 대신 광선 방향의 분포 자체를 normal vector 방향에 가까운 쪽으로 많이 쏘아지게 하는 것입니다.
이런 샘플링 기법을 Mally's method라고 합니다.

이러한 AO의 사전 계산은 CPU / GPU에서 수행할 수 있다. 두 경우 모두 복잡한 geometry에 대한 raycast를 가속하는 라이브러리를 사용할 수 있으며
가장 널리 쓰이는것은 Embree(CPU), OptiX(GPU)입니다.

과거에는 깊이 맵(depth map)이나 오클루전 쿼리 같은 GPU 파이프라인의 결과물을 AO계산에 사용하기도 했으나,
GPU에서 동작하는 RayCast 솔루션이 널리 보급되며 오늘날에는 그런 방식을 쓰는 경우가 줄었습니다.

상용 모델링/렌더링 소프트웨어 패키지는 대부분 AO를 사전 계산하는 기능을 제공하고 있습니다.

미리 계산된 주변 폐색(Precomputed Ambient Occlusion)

AO는 표면의 모든 위치 p마다 값이 다르지만, 무한히 많은 p에 대한 모든 정보를 저장할 수는 없습니다.
이를 해결하기 위해 일반적으로는 Texture, Volume, Mesh의 Vertex에 저장하고 런타임에는 이를 복원하는 과정을 거치게 됩니다.
저장 방식에 따른 특성과 문제점은 신호의 종류와 무관하게 대체로 비슷합니다.
이번 장에서 다루는 AO를 포함해서 다음장에서 다룰 방향성 차폐(DO), 사전계산된 조명등의 정보는 동일한 방법론을 적용할 수 있고, 3차시에 설명하게 될 예정입니다.

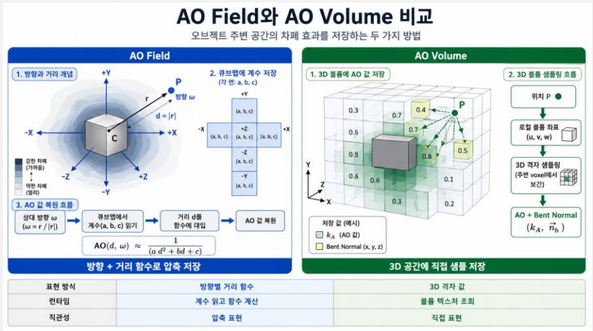
사전 계산된 데이터는 오브젝트들이 서로에게 미치는 AO효과를 모델링하는데에도 사용할 수 있습니다.

Kontkanen과 Laine는 오브젝트로부터의 거리에 따른 AO 값의 감쇠를 2차 다항식의 역수로 근사하고,
방향 ω별 계수를 큐브 맵에 저장하는 Ambient Occlusion Field를 제안하였고,

Malmer등은 AO계수와 bent normal(선택)을 3D Grid상에 저장하는
Ambient Occlusion Volume을 제안하였습니다.

동적인 AO처리가 무거우니 오브젝트단위로라도 구워둘 수 있을까?를 고민한 흔적이고
현대 AO는 대부분 이렇게 처리하지 않아서 넘어갑니다

*게임 엔진에서 이런 방식으로 처리하는거 본적도 없기도 하고, 비슷한 이름의 기능들은 현대 AO처리 방식에 더 가까웠음
*2000년 후~ 2010년대 상용 게임의 커스텀 엔진에서 동적인 AO처리를 위한 기능들이였다 보시면 될 듯



*물론 뒤에 다루겠지만, 동적인 오브젝트의 AO를 정말 싸게 처리해야하는 상황이 있습니다. 모바일 기기에서 동적인 오브젝트에 꼭 필요한 AO를 구현할 때 같은...

미리 계산된 주변 폐색(Precomputed Ambient Occlusion)

AO 값을 저장하는 방법으로 어떤것을 선택하더라도, 다루는 대상이 연속 신호라는 점을 유념해야 합니다.
공간상의 특정 지점에서 Ray를 쏘는것은 샘플링이고, 그 결과로부터 셰이딩에 사용할 값을 보간하는것은 복원입니다.

Kavan등은 최소 제곱 베이킹이라는 방법을 제안했습니다.
Mesh 전체에 균일하게 AO 계수를 샘플링하고, 보간된 값과 샘플링된 값의 차의 제곱의 합이 최대한 작게 만드는 값을 선정해 계수를 저장하게 합니다.
*분산이나 표준편차가 가진 아이디어랑 비슷하고, 이것을 최소화 하는 값을 만들어 저장하게 한다는 아이디어

예시로는 Mesh의 Vertex를 들었지만, Texture나 Volume에 해당 값을 저장할때도 동일한 방법론을 사용할 수 있습니다.

미리 계산된 주변 폐색(Precomputed Ambient Occlusion)

Destiny(2014)는 PS3(2006)에서 PS4(2013말) / XBOX360(2005)에서 XBOX ONE(2013말)으로 넘어가는 과도기에 출시되어
신규 플랫폼에 기대되는 높은 품질과 구세대 플랫폼의 성능-메모리 제약 사이의 균형잡힌 솔루션이 필요했습니다.

게임에는 동적으로 시간이 변하는 솔루션이 있어, 어떤 사전 계산 방식을 사용하더라도 이를 올바르게 반영할 수 있어야 했습니다.

개발진은 그럴듯한 외형과 낮은 런타임 처리 비용을 이유로 AO를 선택했으며
AO는 가시성 계산을 조명과 분리하여 시간대와 무관한 사전 데이터를 재사용할 수 있었습니다.

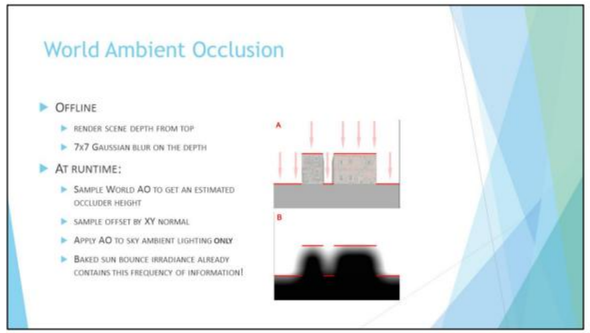
Ubisoft의 어세신크리드와 파크라이 시리즈도 간접 조명 솔루션을 강화하고자 사전 계산된 AO 형태를 사용했습니다.
월드를 위에서 내려다보는 시점에서 렌더링하여 깊이 맵을 여러 휴리스틱을 통해 월드 공간 AO맵을 만들어
각 오브젝트의 월드 공간 위치를 텍스쳐 공간으로 투영하는 방식으로 모든 오브젝트에 AO를 적용했습니다. (Ubisoft는 이를 World AO라고 부름)

*Digital Dragons 2014 유비소프트 발표 자료 (출처)





Looks of “World ambient occlusion” in Assassin’s Creed 3



Algorithm very approximate and requires artists to tweak some “magic values”, but extremely cheap to evaluate and quite robust.
…As long as your scene can be approximated by heightfield!

리얼타임 렌더링 4판 · 11장 전역 조명 · 2차시

주변 폐색의 동적 계산(Dynamic Computation of Ambient Occlusion)

정적인 장면의 경우 AO 계수와 bent normal을 사전 계산할 수 있습니다.
하지만 오브젝트가 움직이거나 형태가 바뀌는 장면에서는 이런 계수들을 실시간으로 계산하는 편이 더 나은 결과를 만들어냅니다.
이를 위한 방법은 크게 오브젝트 공간(object space)에서 동작하는 것과, 스크린 공간(screen space)에서 동작하는 것으로 나눌 수 있습니다.

11.3.5에서는 주로 object/world space에서 동작하는 AO를 다룰 예정입니다.

AO를 계산하는 오프라인 방식은 표면에서 장면을 향해 수십에서 수백개에 이르는 다량의 광선을 쏘고 교차 여부를 검사하는 과정을 포함합니다.
이는 비용이 큰 연산이기 때문에 실시간 기법들은 이 계산의 상당 부분을 근사하거나 피해가는 방법에 초점을 맞춥니다.

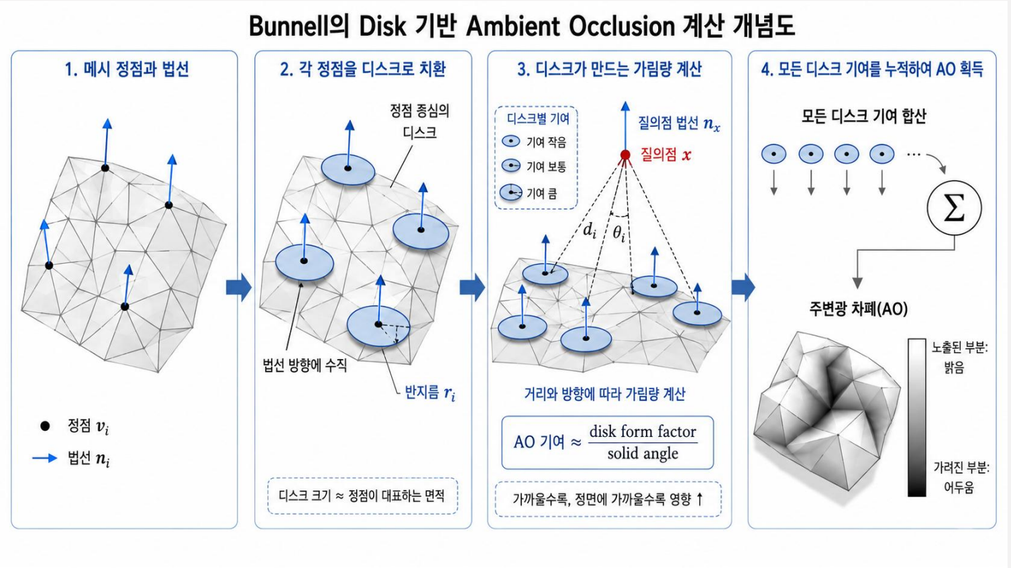
Bunnell은 표면을 Mesh Vertex에 배치된 원반(Disk)모양 요소들의 집합으로 모델링 하여 AO 계수와 bentnormal을 계산합니다.
원반을 택하면 한 원반이 다른 원반을 가리는 이유를 해석적으로 계산할 수 있어 ray를 쏠 필요가 없어지기 때문입니다.

하지만 단순히 모든 Disk의 폐색 계수를 합산하면 이미 가려진 부분을 새로 가리는 문제가 발생할 수 있습니다.
Bunnell은 이 문제를 2Pass로 처리해서, 두번째 Pass에서는 첫번째 Pass에서 계산한 AO계수에 따라 다른 Disk (Surface)에 주는 영향을 줄이도록 하여 해결했습니다.

각 Disk사이의 폐색을 계산하는것은 O(n²) 연산으로, 비용이 많이 나가게 됩니다.
이것은 Disk를 hierarchical tree로 구성하여 서로 먼 표면에 있을 때는 parent 하나로 처리해 실질적으로 O(nlogn)까지 처리 비용을 낮추게 됩니다.

해당 기술은 영화 캐리비안의 해적의 최종 렌더링에 사용되었습니다.

Hoberock은 11.3.2에서 다룬 Obsurance와 유사한 거리 감쇠 계수를 포함해 Bunnell의 알고리즘을 조금 더 비싸지만 고품질로 수정하였습니다.



주변 폐색의 동적 계산(Dynamic Computation of Ambient Occlusion)

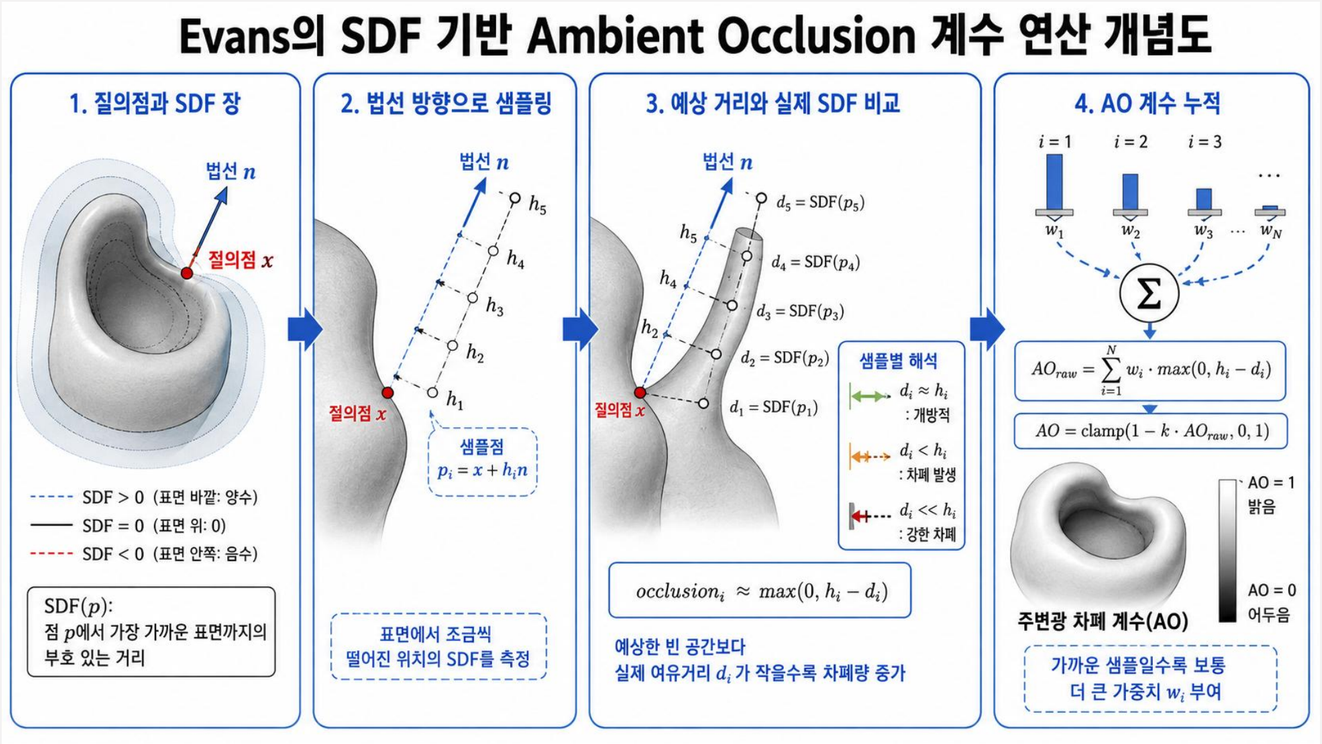
부호 있는 거리 필드 (SDF, Signed Distance Field)는
object 내부 깊숙할수록 음수, object의 표면(surface) 지점에서는 0, object 외부에서 멀어질수록 양수가 되는 함수입니다.

AO에서는 SDF를 읽었을때, 해당 지점에서 가장 가까운 geometry의 존재를 검사할 수 있게 되기 때문에 사용합니다.

Evans는 이 특징을 살려 기존의 hemisphere 방향을 검사하던 AO대신 normal방향으로 점점 먼 점에 대해 SDF를 sampling하는 방법을 제안하였습니다.

이는 SDF값이 3차원 텍스쳐 (Volume)에 저장되는 대신
해석적으로 표현하는 경우에도 같은 접근법을 쓸 수 있습니다.

이 방법은 물리적이지는 않지만,
시각적으로 만족스러운 결과를 만들 수 있습니다.



리얼타임 렌더링 4판 · 11장 전역 조명 · 2차시

주변 폐색의 동적 계산(Dynamic Computation of Ambient Occlusion)

SDF를 AO에 사용하는 방법은 Wirght에 의해 한층 더 발전했습니다.
휴리스틱으로 normal 방향의 SDF를 샘플링하여 AO 계수를 계산하는 대신 Cone tracing을 수행합니다.

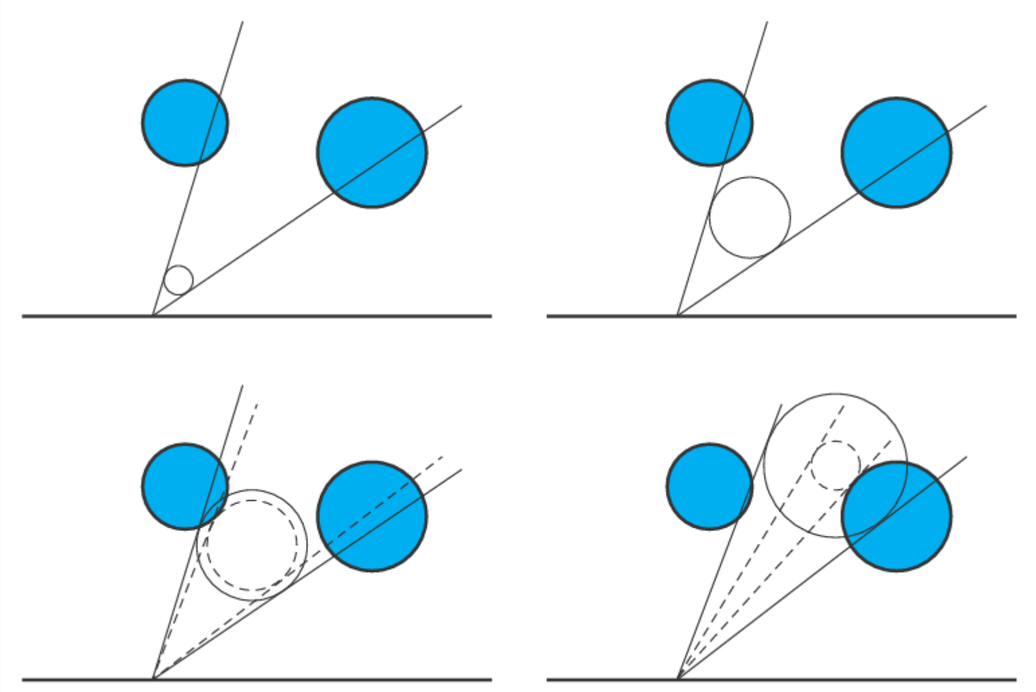
법선 기준 반구에 약 9개의 cone을 배치하고
각 cone 축을 따라 SDF sphere / cone steepping을 수행합니다.
하나의 cone내에서는 값을 합산하는게 아니라 minimum visibility를 유지하도록 합니다.

9개의 cone의 visibilit를 결합하여 AO계수와 bentnormal을 생성합니다.

SDF를 통해 빛이 실제로 통과할 수 있는 가능성을 근사하고 있었다 보심 될 듯

장면에 대한 전역 SDF뿐 아니라 개별 object나
논리적인 object group의 SDF도 함께 사용하여
정밀도를 높혔습니다.

*동적 AO에서 SDF를 사용한다는 것은 SDF를 매 프레임 생성한다는 뜻이 아닙니다.
일반적으로 geometry의 distance-field representation은 미리 준비하고,
런타임에는 현재 shading point와 object transform에 따라 SDF를 query하여 AO를 동적으로 계산합니다

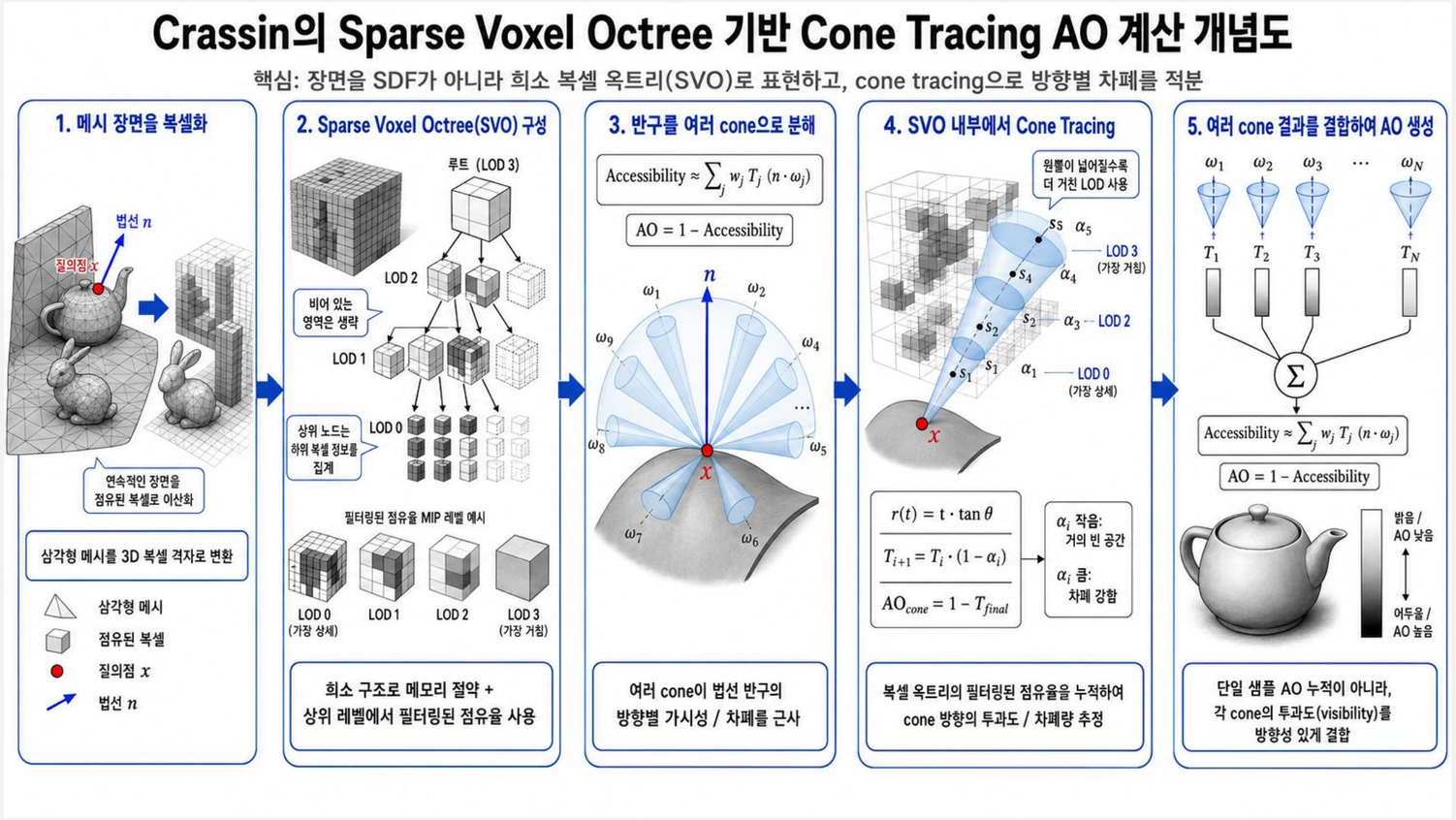


리얼타임 렌더링 4판 · 11장 전역 조명 · 2차시

주변 폐색의 동적 계산(Dynamic Computation of Ambient Occlusion)

Crassin은 장면을 먼저 voxel로 변환합니다.
거대한 3D 공간을 dense volume으로 저장하면 너무 크기 때문에, 빈 공간은 크게 묶고 geometry가 있는 곳만 세분화 하는 Sparse Voxel Octree(SVO)를 구성합니다.

Octree를 구성하였기 때문에, Cone Tracing에서 멀어진 cone은 상위 레벨의 octree를 참조할 수 있게 됩니다.



리얼타임 렌더링 4판 · 11장 전역 조명 · 2차시

오늘의 순서 - 3차시

00 주변 폐색 — 11.3 * 생각보다 많이 남았습니다

01 방향성 폐색 — 11.4

02 Heckbert 표기법 — 11.1 * 추진력을 얻기 위해 버렸지만 추진력은 얻지 못했다. 업보 청산의 시간

03 확산 전역 조명 — 11.5 * 근데 얘도 AO랑 분량이 비슷할거같아서 무서워요

주변 폐색의 동적 계산(Dynamic Computation of Ambient Occlusion)

Ren은 AO가 흐릿하다는 점을 이용해 Occulsion Geometry를 구 모음으로 근사했습니다.

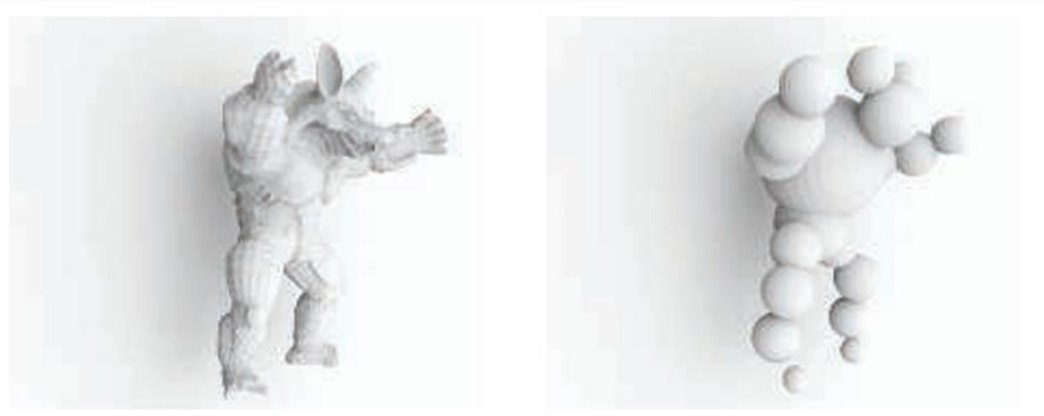
단일 구에 가려지는 표면점을 표현하는 가시성 함수를 구면 조화 함수를 사용해 표현할 수 있고
여러 개의 구에 의한 차폐의 결과는 개별 구의 가시성 함수를 곱한 결과가 됩니다.

하지만 구면조화 함수끼리의 곱을 계산하는것은 비용이 큰 연산이고
이를 해결하기 위해 개별 구면 조화 함수의 로그를 더한 뒤, 그 결과에 지수를 취하는 것입니다.
이렇게 하면 가시성 함수를 곱한것과 동일한 최종 결과를 얻으면서도, 훨씬 저렴하게 처리할 수 있습니다.

$$V_1(l)V_2(l)V_3(l)$$

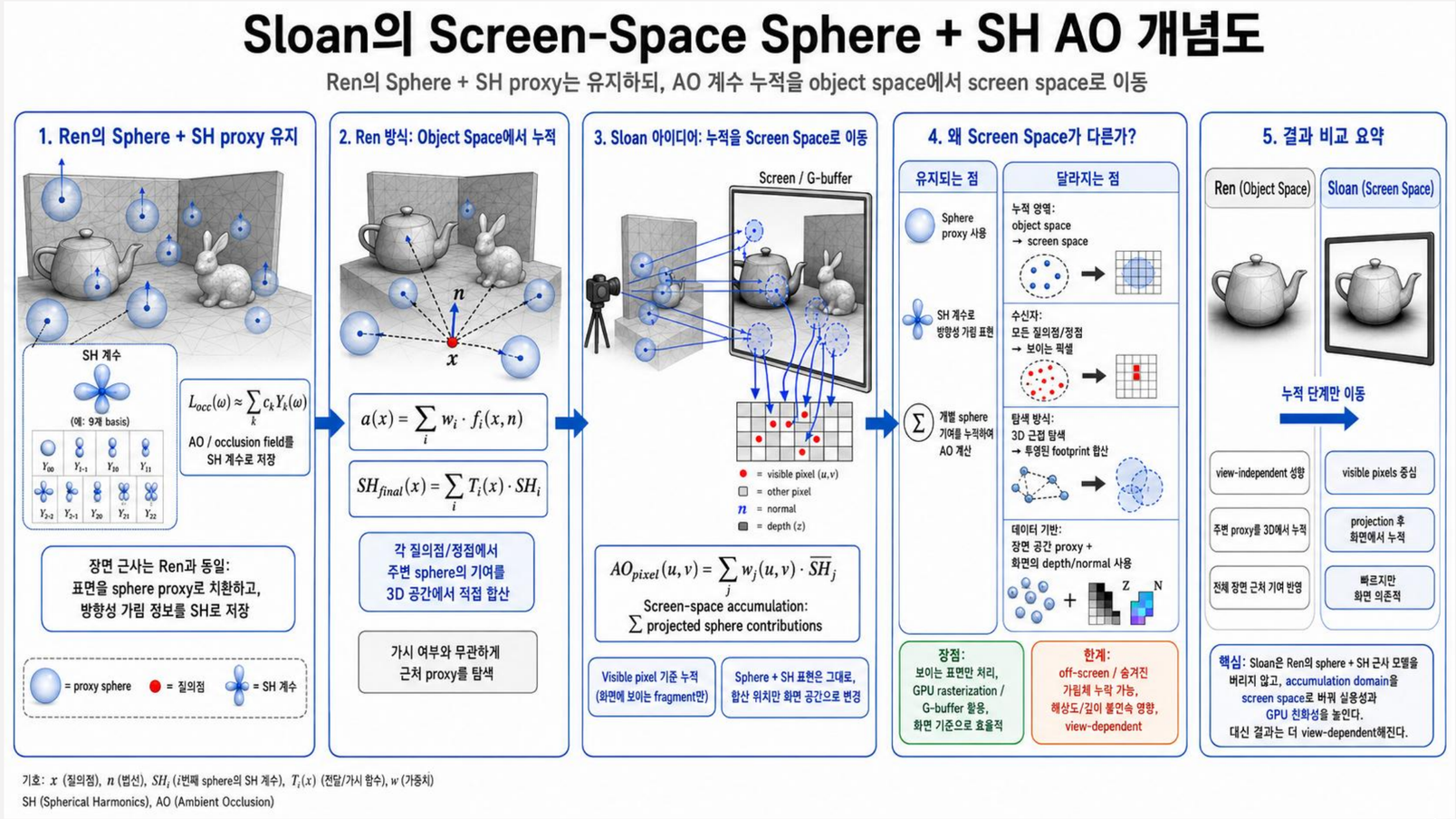
$$\log V_1(l) + \log V_2(l) + \log V_3(l)$$

$$\log \prod_{i=1}^{N} V_i(l) = \sum_{i=1}^{N} \log V_i(l)$$



주변 폐색의 동적 계산(Dynamic Computation of Ambient Occlusion)

Sloan은 Ren의 Sphere + SH방식의 visibility 누적 연산을 screen space에서 처리하는 방법을 제안하였고,
Hill은 SH 함수를 2차 계수까지만 저장하여 성능이 제한적인 하드웨어에서도 사용할 수 있지만, 흐릿한 AO 처리만 가능한 방법을 제안했습니다.



리얼타임 렌더링 4판 · 11장 전역 조명 · 3차시

스크린 공간 기법(Screen-Space Methods)

직전까지 다룬 object-space를 기반으로 만들어지는 AO들은 scene geometry를 계산해야 하기 때문에
구성된 씬의 복잡도에 따라 영향을 받게 됩니다. (안보이는 물체들도 연산 대상이 됩니다)

하지만 차폐에 대한 정보 중 일부는 깊이(Depth)나 법선(normal)처럼 이미 확보되어있는 스크린 공간 데이터 만으로도 유추해 낼 수 있습니다.

이러한 기법들은 장면이 얼마나 정밀한지와 무관하게, 오직 렌더링에 사용하는 해상도에만 좌우되는 일정한 비용을 갖습니다.
(이전 파트에서 Sloan의 방법이 가지는 메리트, 하지만 여기부터는 AO계수 누적을 위한 샘플링도 ScreenSpace에서 처리합니다.)

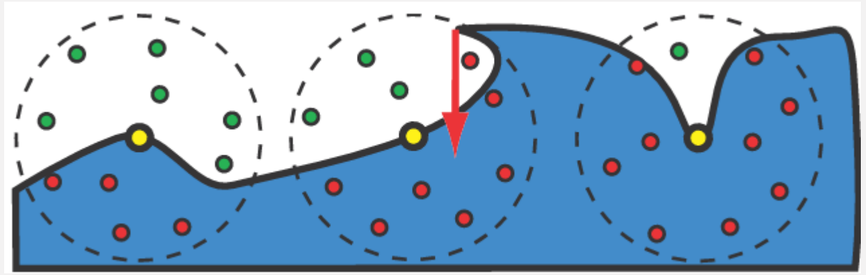
화면에 보이지 않는 geometry 정보는 알 수 없게 되는 문제가 있지만,
기존 렌더링 과정중에 남는 데이터거나, 렌더링 과정중에 조회한 데이터를 별도 버퍼에 저장해두기 때문에 GPU 친화적인 장점이 있습니다.

스크린 공간 기법(Screen-Space Methods)

Crytek은 Crysis (2007)에 사용된 동적 **스크린 공간 엠비언트 오클루전** (screen-space ambient occlusion, **SSAO**) 기법을 개발했습니다.

이들은 Z-Buffer만을 입력으로 사용하는 전체 화면 패스에서 AO를 계산합니다.

각 pixel의 폐색 계수 k_A는 픽셀 위치를 중심으로 한 구 내부에 분포시킨 점들의 집합을 z-buffer와 비교 검사하며 추정합니다.
k_A의 값은 현재 pixel의 z-buffer 값보다 앞에 있는 샘플의 개수에 대한 함수입니다.
이를 통과하는 샘플수가 적을수록 k_A값이 낮아지게 됩니다.



샘플들은 Obscurance factor와 마찬가지로 픽셀로부터 거리에 따라 감소하는 가중치를 가집니다.

스크린 공간 기법(Screen-Space Methods)

여기서 샘플에 $(n \cdot l)^+$ 인자로 가중치를 주지 않기 때문에, 결과로 나타나는 AO는 부정확하다는 점에 유의해야한다.
표면 위쪽 반구(surface hemisphere)대신 전체 sphere를 대상으로 하기 때문에, 평평한 면도 조금 더 어두워지고
Edge는 주변보다 밝아지는 현상이 생깁니다.

하지만 그럼에도 결과물은 시각적으로 만족스러운 경우가 많습니다.

*SSAO Only





*Albedo

*SSAO + Albedo





*SSAO + Albedo + Shadow + Specular

리얼타임 렌더링 4판 · 11장 전역 조명 · 3차시

스크린 공간 기법(Screen-Space Methods)

코사인 가중치도 없고, 반구도 아니고 구형의 샘플링으로 정확하지 않은 부분이 있는 SSAO를 보완하기 위해 여러가지 연구들이 있었지만
시간 관계상, 이후 바로 HBAO (2008)부터 소개합니다.

방금 다룬 SSAO기술을 몬테 카를로 적분으로 해석해 수식으로 표현하면 다음과 같은 형태가 됩니다.

$$v_A = \int_{x \in X} \rho\left(d(x)\right) o(x)\, dx$$

* 수식 (11.15), 몬테 카를로 적분으로 해석한 SSAO

ρ = obscurance파트에서 다룬것과 유사한 거리 매핑 함수
d = 거리 함수
$o(x)$ = 점유함수 (x가 depth buffer기준으로 가려져있으면 1, 아니면 0)

Bavoil등은 z-buffer정보가 연속적인 높이 장(height field)를 나타낸다고 가정하고,
특정 점에서 보이는 지평선(horizon)을 추정 할 수 있음을 관찰한 Max의 연구 결과에 집중했습니다.

코사인 항을 무시한다면, 어떤 점에서의 가시성은 탄젠트평면 위쪽의 차폐되지 않은 부분에 대한 적분으로 계산할 수 있고,
다르게는 1에서 차폐된 부분의 적분을 뺀 값으로 계산할 수 있습니다.

$$k_A = 1 - \frac{1}{2\pi} \int_{\varphi=-\pi}^{\pi} \int_{\alpha=t(\varphi)}^{h(\varphi)} W(\omega) \cos(\theta)\, d\theta\, d\varphi$$

* 수식 (11.16), HBAO

$W(\omega)$ = 거리기반 감쇠함수

Bavoil등은 z-buffer정보가 연속적인 높이 장(height field)를 나타낸다고 가정하고,
특정 점에서 보이는 지평선(horizon)을 추정 할 수 있음을 관찰한 Max의 연구 결과에 집중했습니다.

어떤 점에서의 가시성은 탄젠트평면 위쪽의 차폐되지 않은 부분에 대한 적분으로 계산할 수 있고,
다르게는 1에서 차폐된 부분의 적분을 뺀 값으로 계산할 수 있습니다.

스크린 공간 기법(Screen-Space Methods)

$$k_A = 1 - \frac{1}{2\pi}\int_{\varphi=-\pi}^{\pi}\int_{\alpha=t(\varphi)}^{h(\varphi)} W(\omega)\cos(\theta)\, d\theta\, d\varphi$$

* 수식 (11.16), HBAO

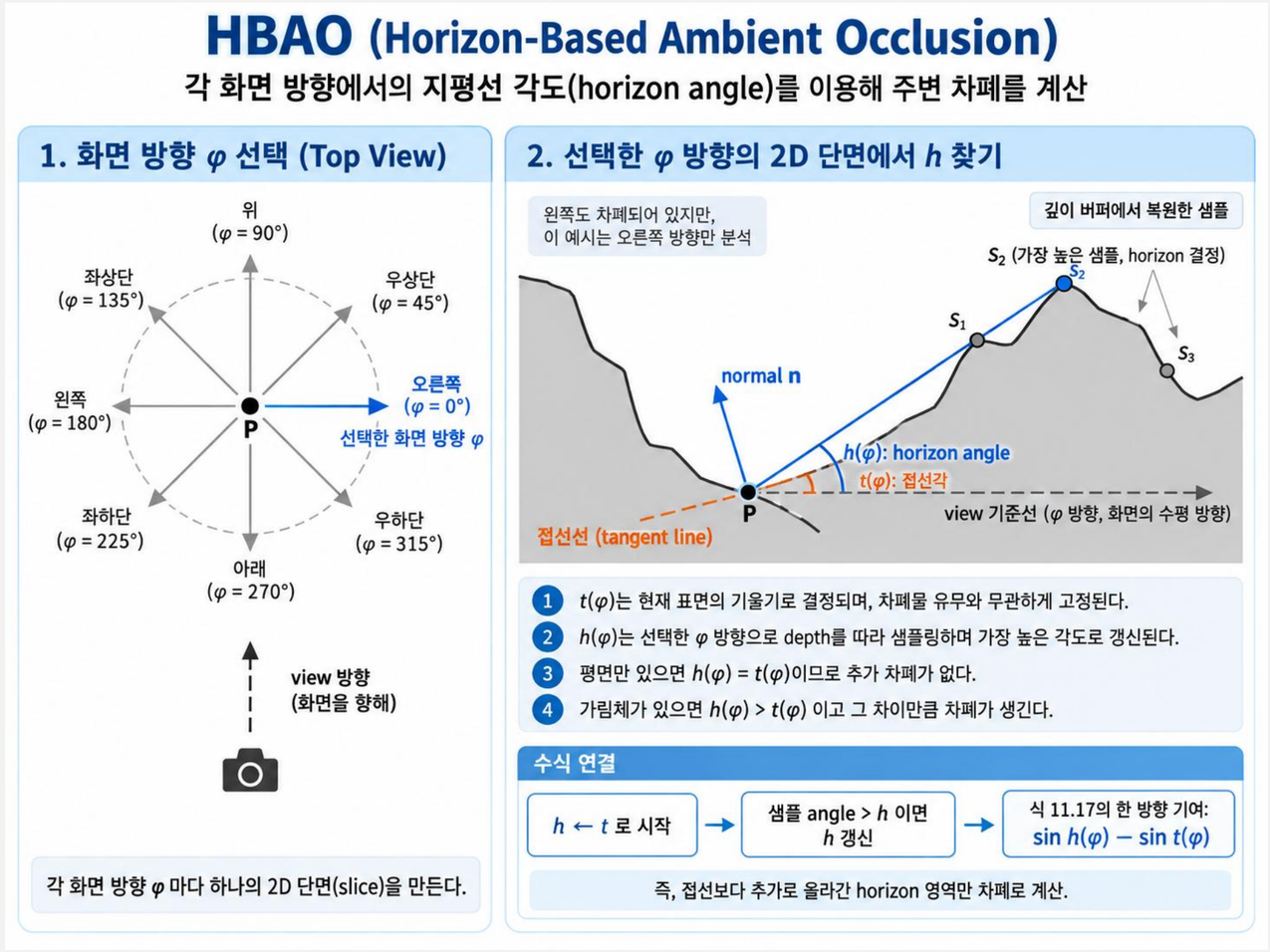
HBAO 아이디어는 알겠는데, 결국 반구 방향으로 다 Ray쏘라는 소리 아닌가? 싶겠지만 식을 잘 보면 다음과 같은 관찰이 가능합니다.

$$\int_a^b \cos\theta\, d\theta = \sin b - \sin a \quad \longrightarrow \quad \int_{t(\varphi)}^{h(\varphi)} \cos\theta\, d\theta = \sin\big(h(\varphi)\big) - \sin\big(t(\varphi)\big)$$

$$k_A = 1 - \frac{1}{2\pi}\int_{\varphi=-\pi}^{\pi} [\sin(h(\varphi)) - \sin(t(\varphi))]\, W(\varphi)\, d\varphi$$

* 수식 (11.17), 안쪽 적분을 푼 HBAO

Surface에서 tangentspace를 참고해 여러 ϕ방향으로 ray marching을 시행해 차폐된 영역의 AO값을 점진적으로 누적하는 방식으로 계산할 수 있습니다.



스크린 공간 기법(Screen-Space Methods)

HBAO의 구현체의 예시입니다. (의사코드)

```
float EstimateHBAO(Pixel P, Normal N)
{
   float occlusion = 0;

   for (Direction dir : directions)
   {
      // 현재 surface의 tangent를 기준으로 시작
      float horizon = TangentAngle(P, N, dir) + angleBias;

      for (int step = 1; step <= numSteps; ++step)
      {
         // 1. 화면상 다음 위치의 depth를 읽음
         vec2 sampleUV = P.uv + dir * step * stepSize;

         // 2. depth -> view-space 위치 복원
         vec3 S = ReconstructPosition(sampleUV);

         // 3. P에서 S를 보았을 때 elevation angle
         float angle = ElevationAngle(P.position, S, N);

         // 4. 기존 horizon보다 높은 occluder인가?
         if (angle > horizon)
         {
            float distance = length(S - P.position);

            // 경험적인 거리 감쇠
            float W = 1.0 - Square(distance / radius);

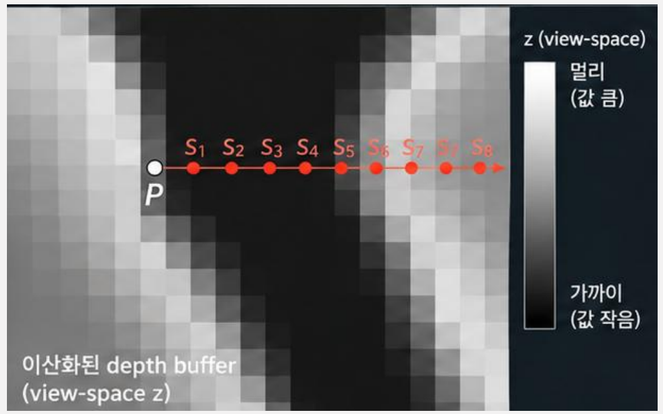
            // 새롭게 가려진 angular 영역만 추가
            occlusion += W * (sin(angle) - sin(horizon));

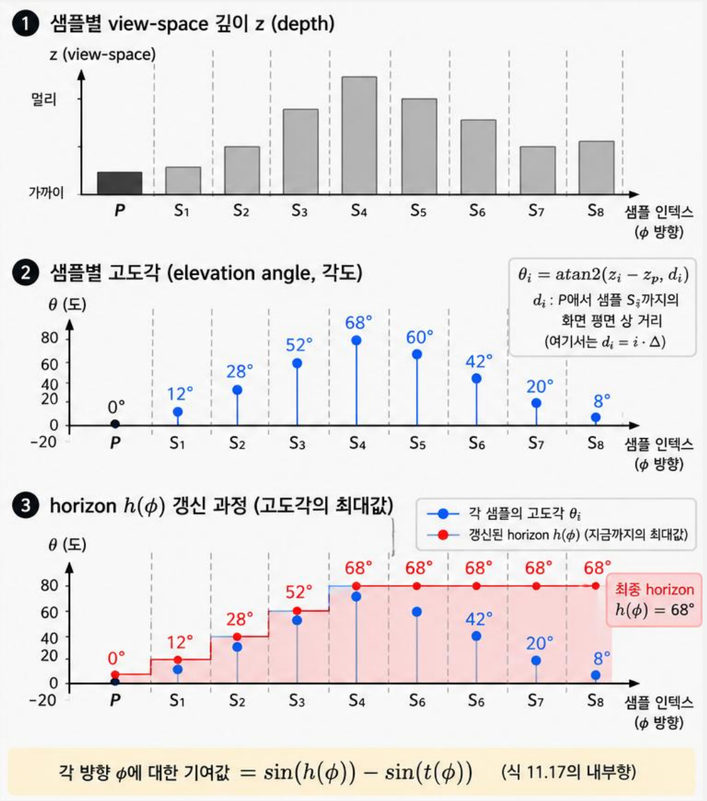
            horizon = angle;
         }
      }
   }

   return 1.0 - occlusion / numDirections;
}
```

$$k_A = 1 - \frac{1}{2\pi}\int_{\varphi=-\pi}^{\pi} [\sin(h(\varphi)) - \sin(t(\varphi)))]\, W(\varphi)\, d\varphi$$

* 수식 (11.17), 안쪽 적분을 푼 HBAO





리얼타임 렌더링 4판 · 11장 전역 조명 · 3차시

스크린 공간 기법(Screen-Space Methods)

SSAO도 그렇지만, HBAO도 코사인 가중치가 없습니다.
거리기반 감쇠함수도 경험적으로 설정하게 됩니다.

$$k_A = 1 - \frac{1}{2\pi}\int_{\varphi=-\pi}^{\pi} [\sin(h(\varphi)) - \sin(t(\varphi)))]\, W(\varphi)\, d\varphi$$

$W(\omega)$ = 거리기반 감쇠함수

* 수식 (11.17), 안쪽 적분을 푼 HBAO

이를 해결하기 위해 GTAO는 depth buffer의 heightfield를 실제 geometry라고 가정하여 이를 계산하는 아이디어를 사용합니다.

$$k_A = \frac{1}{\pi}\int_{\Omega} v\,(l)(n \cdot l)^+ dl$$

* 수식 (11.8), 주변 폐색

$$k_A = \frac{1}{\pi}\int_{0}^{\pi}\int_{h_1(\phi)}^{h_2(\phi)} \cos(\theta - \gamma)^+ \,|\sin\theta|\, d\theta\, d\phi$$

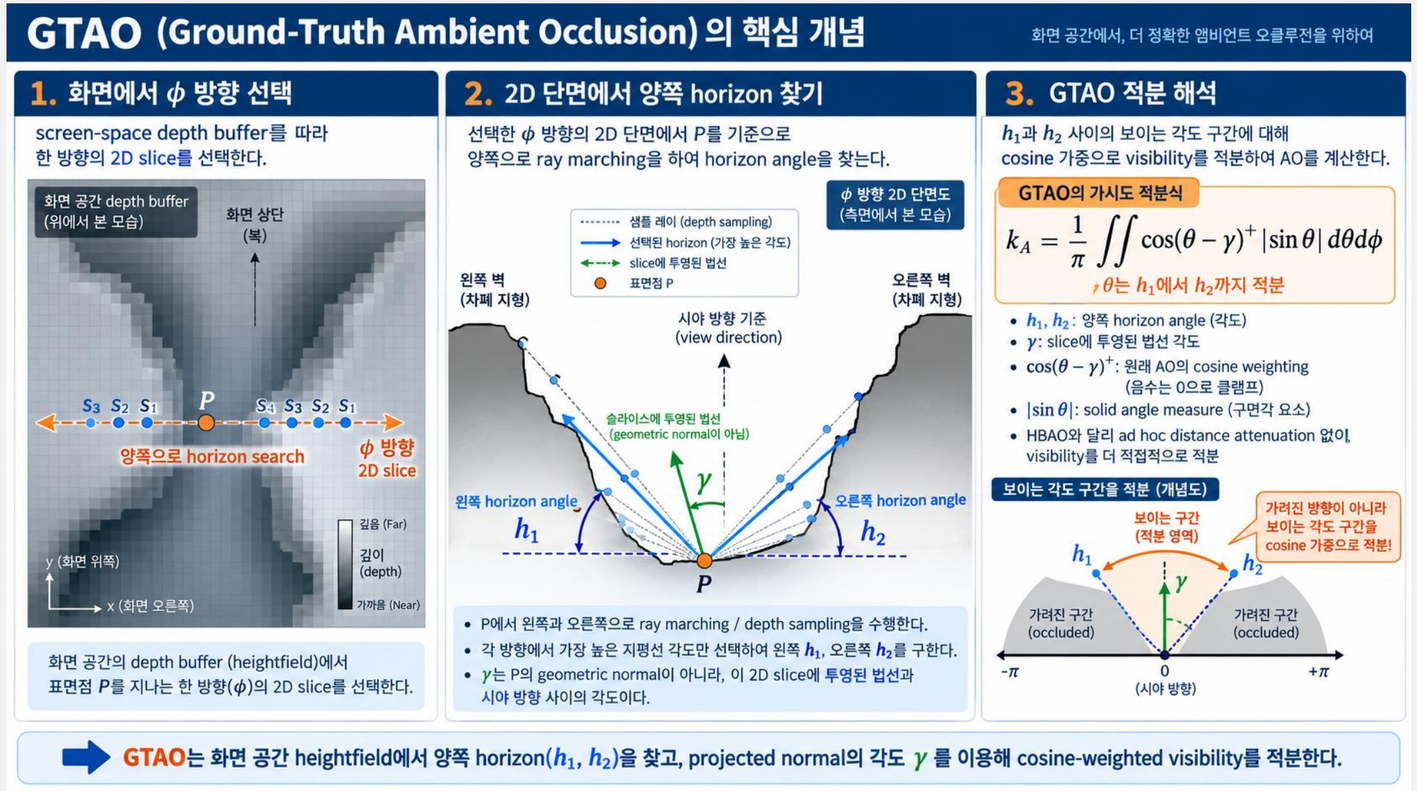
* 수식 (11.18), GTAO

γ = surface normal과 view direction 사이의 각

GTAO의 수식도 이중 적분이지만
HBAO와 같이 안쪽 적분은 해석적으로 풀 수 있어

HBAO와 GTAO가 상수 시간복잡도를 제외하면
크게 차이나지 않습니다.

(둘다 horizon 탐색 기반)



스크린 공간 기법(Screen-Space Methods)



1
기둥뒤에 공간 있어요

* Z Buffer기반으로는 기둥 뒤에 공간이 있는지 알 방법이 없다!
* 저런 공간에서 AO를 그리면 공간이 없다고 생각하고 차폐된 영역으로 계산하게 될 것

Z Buffer를 기반으로하는 Screen-Space Method들은 주어진 방향에 대해 가장 가까운 오브젝트 위치만 기록할 뿐,
차폐된 뒤에 무엇이 있는지를 나타내지 않습니다.
많은 기법이 보이는 오브젝트의 두께를 추론하기 위한 갖가지 휴리스틱을 사용하곤 합니다.
이런 근사는 많은 상황에서 충분하며, 사람의 눈은 부정확함에 관대한 편입니다 (책에는 그렇다는데, AO같은 변곡점이 눈에 띄지 않는 차폐라 더 그런편)

여러겹의 Depth Buffer를 사용하는등의 기법들도 연구되었지만, 렌더링 엔진과의 통합이 복잡하고 런타임 비용이 비싸 널리 쓰이지 못했습니다.

이를 해결하기 위해 최신 엔진이나 게임에서는 하드웨어의 발전으로 나타난 RayTrace기반의 AO기법들도 사용되곤 하지만,
이에 대한 구체적인 내용은 4차시에 다루도록 합시다.

스크린 공간 기법(Screen-Space Methods)

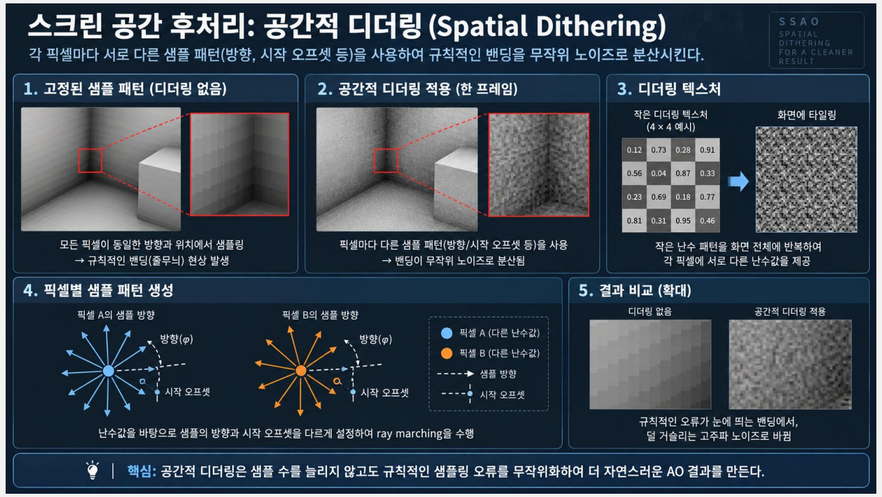
이런 스크린 공간 기법들은 대부분 주어진 점 주변 지오메트리의 단순화된 모델을 만들기 위해 z-버퍼를 반복적으로 샘플링하는 데 의존합니다.
실험에 따르면 높은 시각적 품질을 얻기 위해서는 수백개의 샘플이 필요하지만,
실시간 렌더링에서는 기기 사양에 따라 10~20개, 종종 그보다도 적은 수의 샘플이 작동할 수 있습니다.

이론과 실무 사이의 간극을 메우기 위해 스크린 공간 기법들은 일종의 공간적 디더링을 사용합니다.
화면의 픽셀마다 조금씩 다른 (회전시키거나 이동시킨) 무작위 샘플 집합을 사용하는것입니다.

적은 샘플로 이루어진 AO계단의 주 단계가 끝나면, 전체 화면 필터링 패스를 수행합니다.

표면의 불연속면을 가로질러 필터링하는것을 막고 선명한 경계를 보존하기 위해 결합 양방향 필터링(joint bilateral filtering)을 사용합니다.
이에 대해서는 12.1.1에서 다룰 예정입니다.

이 필터는 깊이나 normal정보를 활용해 동일한 표면에 속한 샘플만 사용하도록 필터링을 제한합니다.



스크린 공간 기법(Screen-Space Methods)

또한 AO계산은 시간에 걸쳐 슈퍼샘플링 되는 경우도 많습니다.

매 프레임마다 다른 샘플링 패턴을 적용하고 차폐 계수를 지수 평균하는 방식으로 이루어집니다.
이전 프레임들의 z-buffer, moving vector, 카메라 변환등의 정보를 이용해 현재 프레임의 pixel위치와 맞추고
신뢰할 수 없는 데이터의 경우 폐기합니다.

이 데이터를 만들때는 depth, normal, moving vector등에 기반한 휴리스틱이 많이 (정말 많이!) 사용됩니다.

AO에서는 이것이 치명적인 아티팩트를 만드는 경우가 드물지만, 신뢰 할 수 없는 데이터가 많아 샘플링할 데이터가 없는 경우
더 적극적으로 공간적 디더링을 사용하는 등의 방법으로 한 프레임의 샘플링이 인접 픽셀과 크게 다른 결과를 만드는 것을 방지합니다.



리얼타임 렌더링 4판 · 11장 전역 조명 · 3차시

주변 폐색을 이용한 셰이딩(Shading with Ambient Occlusion)

지금까지 주변 폐색 값을 일정하고 먼 거리의 조명 (diffuse illumination)이라는 맥락에서 유도하기는 했지만, 이를 더 복잡한 조명 상황에도 적용할 수 있습니다.

$$L_o(v) = \int_{l \in \Omega} f(l, v) L_i(l) v(l) (n \cdot l)^+ \, dl$$

출력 = 재질함수 × 입사광 × 가시성 × 입사각

* 수식 (11.19), 반사율 방정식

반사율 방정식을 다시 살펴봅시다. 위 형태는 11.3.1 (렌더링 방정식) 파트에서 도입했던 가시성 함수 v(l)을 포함합니다.
AO를 처음 정의했을때, 저희는 Lambertian surface를 가정했으므로, 재질 함수를 적분 밖으로 분리할 수 있습니다.

$$L_o = \int_{l \in \Omega} \frac{\rho_{ss}}{\pi} L_i(l) v(l) (n \cdot l)^+ \, dl = \frac{\rho_{ss}}{\pi} \int_{l \in \Omega} L_i(l) v(l) (n \cdot l)^+ \, dl$$

* 수식 (11.20), Lambertian 표면에서의 반사율 방정식

식 정리 과정은 생략하고, AO 정의까지 포함해 다음과 같이 식을 정의하면 다음과 같은 식이 나오게 됩니다.

$$L_o = k_A \rho_{ss} \int_{l \in \Omega} L_i(l) K(n, l) \, dl$$

* 수식 (11.22), Lambertian 표면에서의 주변 폐색을 포함한 반사율 방정식

$$K(n, l) = \frac{V(l) (n \cdot l)^+}{\int_{l \in \Omega} V(l) (n \cdot l)^+ dl}$$

* 수식 (11.23), 방향성 필터링 커널

식 11.22의 적분은 입사 래디언스 Li에 방향성 필터링 커널 K를 적용하는 것으로 생각할 수 있습니다.
필터 K는 위치와 방향에 따라 복잡하게 변하지만, 반구 범위에 있고 전체 적분은 1이 되도록 normalize되어있습니다.

주변 폐색을 이용한 셰이딩(Shading with Ambient Occlusion)

수식 11.22를 기반으로 셰이딩을 수행하기 위해서는 각 방향의 입사 radiance Li에 대해 방향성 필터링 커널 K를 적용하는 것으로 생각할 수 있지만

$$K(n,l) = \frac{V(l)(n \cdot l)^+}{\int_{l \in \Omega} V(l)(n \cdot l)^+ dl}$$ * 수식 (11.23), 방향성 필터링 커널

이를 수행하려면 입사 래디언스 Li와 필터함수 K라는 두 함수의 곱을 적분해야합니다.
Li와 K를 모두 구면 조화 함수등의 저비용으로 처리하는 방안도 있지만, 필터를 비슷한 성질을 가진 더 단순한 필터로 근사하는 것입니다.

가장 흔히 선택되는 것은 정규화된 코사인 커널 H입니다.

$$H(n,l) = \frac{(n \cdot l)^+}{\int_{l \in \Omega} (n \cdot l)^+ \, dl}$$ * 수식 (11.24), 정규화된 코사인 커널

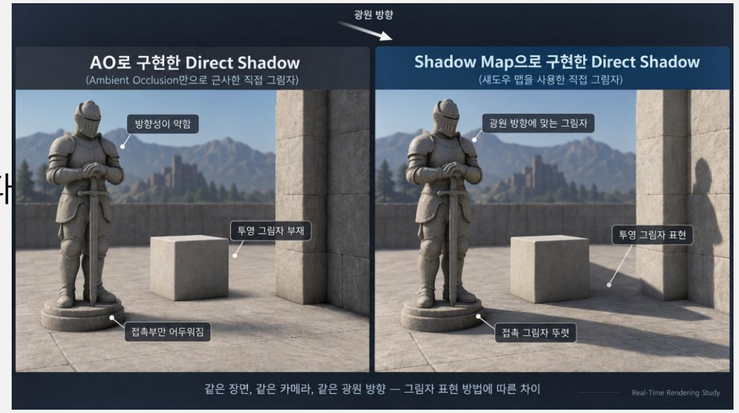
방향성 필터링 커널 K의 경우는 가시성 함수 V가 필요했지만, 코사인 커널 H는 가시성 함수가 필요하지 않습니다.

$$L_o = k_A \rho_{ss} \int_{l \in \Omega} L_i(l) \frac{(n \cdot l)^+}{\int_{l \in \Omega} (n \cdot l)^+ \, dl} \, dl = \frac{k_A}{\pi} \rho_{ss} E$$ * 수식 (11.25), 정규화된 코사인 커널

$$E(n) = k_A \pi L_A$$ * 수식 (11.9), 주변 폐색

주변폐색의 정의를 다루었을때와 마찬가지로 단순한 형태에서 AO를 이용하면 셰이딩이 Irradiance와 AO값을 곱하는 것 만으로 시행할 수 있게 됩니다.

주변 폐색을 이용한 셰이딩(Shading with Ambient Occlusion)



이 수식은 AO가 점광원이나 작은 면 광원의 가시성을 근사하기에는 부적절한지에 대한 통찰도 제공합니다.
그런 광원은 표면에서 볼때 작은 입체각만 차지하며, 이때 가시성 함수는 조명 적분값에 결정적인 영향을 미칩니다.

사실상 이진적으로 광원의 기여를 제어하는데 방향에 따른 광원의 기여를 날려버린 AO는 이를 잘 표현할 수 없습니다
그림자는 윤곽이 뚜렷하지 못하고 기대되는 방향성도 전혀 드러나지 않게 됩니다.

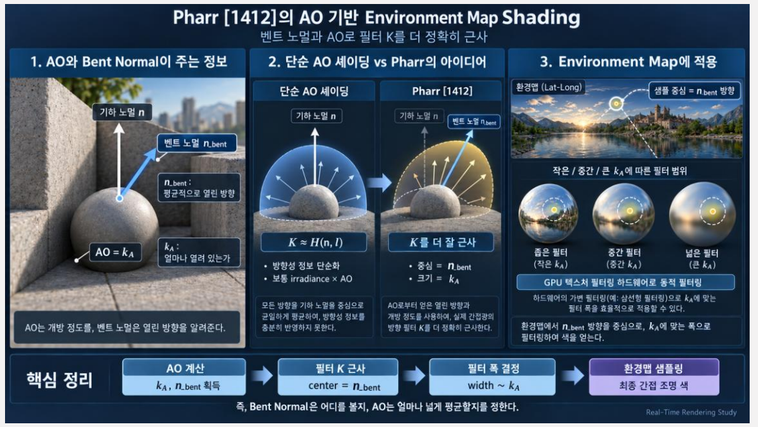
또한 지금까지는 램버시안 표면을 셰이딩한다고 가정해왔습니다.
더 복잡한 상수가 아닌 BRDF를 다룰때는 재질함수를 적분 밖으로 분리할 수 없고,
스페큘러 재질의 경우 필터 K(방향성 필터링 커널)는 가시성과 법선 뿐 아니라 시선 방향에도 의존합니다.

전형적인 microfacet BRDF로브를 정해둔 단일 형태로 근사하는것도 너무 거칠은 결과를 만들어 내기 때문에
이후 다룰 다른 방법을 사용하는것이 좋습니다.

AO 정의 초반에 다룬 Bent normal을 사용하는것은 필터 K를 좀 더 정밀하게 근사하는 한가지 방법으로 볼 수 있게 됩니다.
기하 법선과 bent normal이 일치하지 않는 경우 조금 더 정확한 결과를 제공하게 됩니다.

환경맵을 이용한 셰이딩의 경우 GPU의 텍스쳐 필터링 하드웨어를 통해 동적으로 수행하는 대안을 제시했습니다.

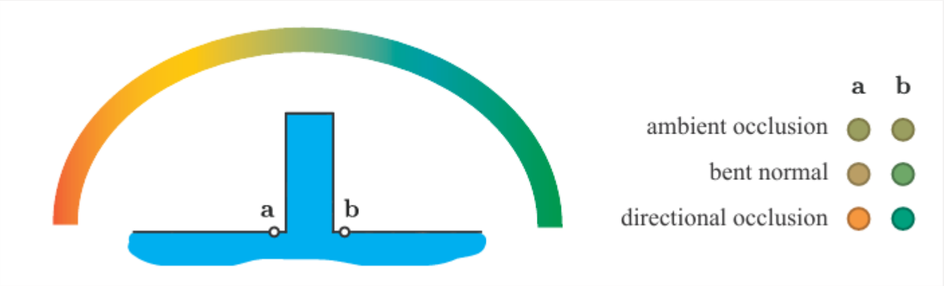
필터 K의 형태는 bent normal의 방향과 AO값에 따라 정해집니다.



리얼타임 렌더링 4판 · 11장 전역 조명 · 3차시

방향성 폐색 (Directional Occlusion)

11.3 파트에서 다룬 주변 폐색은 방향성 차폐 요소들을 계산하더라도, 셰이딩 시점에는 단일 scalar값으로 처리하여 방향성 정보를 손실했습니다.
Incoming light가 방향에 따른 색이나 세기가 다른 상황에는 AO값만으로는 이를 표현할 수 없게 됩니다.



이번 장에서 다루는 내용은 AO로는 처리하지 못했던, 어떤 방향이 보이는지에 대한 정보를 인코딩 하는 방법을 다룹니다.
해당 정보를 이용해 위치 광원의 그림자를 그릴 수 있지만, 이것이 방향성 폐색의 주 목적은 아닙니다.

(7장은 Shadow Map등을 통해, 명확한 visibility를 검사할 수 있었음)

미리 계산된 방향성 폐색(Precomputed Directional Occlusion)

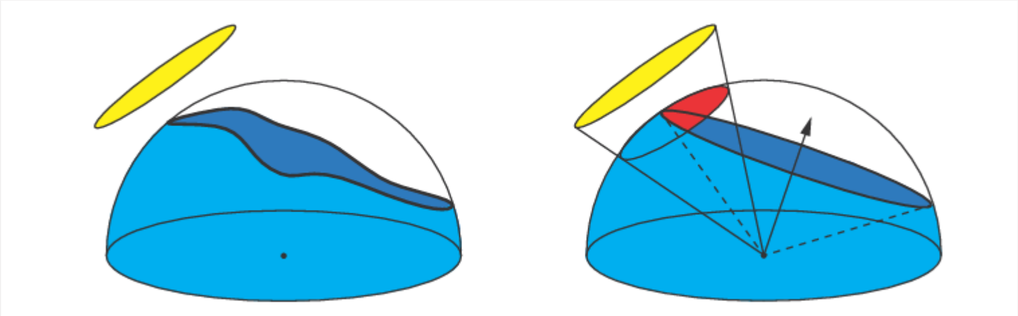
주변 폐색을 다루며, 반구 방향으로의 visibility를 계산하는 방법은 이미 다루었습니다.
하지만 런타임에는 v(l)함수를 평가하는 대신 이를 단일 AO계수로 압축했었는데, 이를 단일 AO계수로 압축하는 대신
미리 계산된 방향 가시성 정보들을 어떻게 저장할지에 초점을 맞춥니다.

Max는 heightfield 표면의 자기 차폐를 표현하기 위해 수평선 매핑(horizon mapping) 개념을 도입했습니다.
표면의 각 점마다 일정한 방위각 방향들에 대해 지평선의 고도각을 구합니다. (HBAO에서 사용하던 그 개념 맞습니다)

정해진 방위 방향별로 지평선 각도를 저장하는 대신, 차폐되지 않은 3차원 방향들의 집합 전체를 타원형 또는 원형 조리개로 모델링 할 수 도 있습니다.
이를 ambient aperture lighting이라고 합니다.

이는 수평선 매핑보다 저장 공간이 적게 들지만, 차폐되지 않은 방향의 집합이 타원이나 원과 닮지 않았다면 그림자가 잘못 나올 수 있습니다.

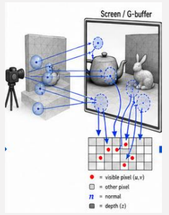
차폐 기법에는 여러가지 변형도 있습니다.
Wang등은 가시성을 표현하기 위해 구면 부호 거리 함수 (SSDF)등을 사용하였고, 구면 위에서 차폐 영역의 경계까지의 부호 있는 거리를 인코딩합니다.
10.3에서 다룬 구면/반구면 기저도 가시성 인코딩에 사용할 수 있으며, AO와 마찬가지로 해당 정보는 텍스쳐, 메시 정점, 볼륨등에 저장할 수 있습니다.



* Ambient aperture lighting

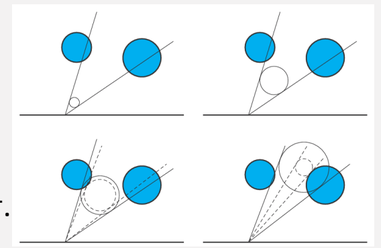
방향성 폐색의 동적 계산(Dynamic Computation of Directional Occlusion)

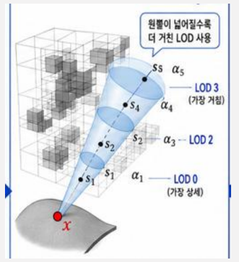




AO를 생성하는 기법 중 상당수는 방향성 가시성 정보를 생성할때도 쓸 수 있습니다.
Ren등의 구면 조화 함수 거듭제곱과 이를 화면 공간으로 옮긴 Sloan등의 기법은 가시성을 구면 조화 함수(SH) 벡터 형태로 생성합니다.
2개 이상의 SH 대역이 사용되는 경우, 별도 처리 없이도 방향 정보를 제공할 수 있으며 대역을 더 많이 사용할 수록 더 정밀하게 인코딩 할 수 있게 됩니다.

Crassin등과 Wright의 기법 같은 콘 트레이싱 방식은 트레이스 한번마다 차폐 값을 구하고,
AO를 추정할때조차 이를 여러 번 수행하므로 방향 정보를 확보할 수 있습니다.
특정 방향의 가시성 정보가 필요하면 원뿔을 더 적게 트레이스 하면 됩니다.





화면 공간 기법 중에서도 방향성 차폐 정보를 제공하도록 확장 할 수 있는 것이 다수 있습니다.

Klehm등은 z-buffe데이터로 화면 공간 bent cone을 계산합니다.
이는 사실상 원형 조리개로 픽셀 주변을 샘플링하면서 차폐되지 않은 방향들을 합산합니다.
합산된 벡터의 길이로 가시성 원뿔의 꼭지각을 추정할 수 있고, 벡터의 방향이 원뿔의 축이 됩니다.

Jimenez등은 지평선(horizon) 각도로부터 원뿔 축의 방향을 수정하고, 원뿔의 각도는 AO 계수로부터 유도합니다.

방향성 폐색을 이용한 셰이딩(Shading with Directional Occlusion)

방향성 폐색을 인코딩 하는 방법이 다양하므로, 셰이딩 방법을 하나로 정해서 제시할 수는 없습니다.
해법은 구체적으로 어떤 효과를 구현하려는지에 따라 달라지지만, 현대 게임엔진등에서는 아이디어를 차용할 뿐, 방향성 폐색을 직접 사용하는 경우는 드뭅니다.

* 언리얼의 경우 DFAO를 지원했지만, Lumen에서 GI계산 과정에 개념이 통합되었음

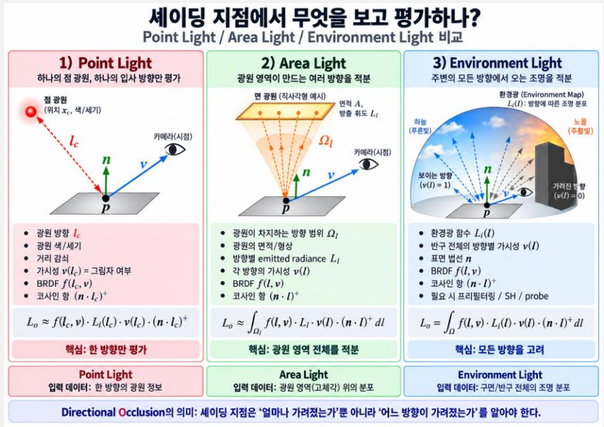
서적에서 다룬 구체적인 use case들은 생략하고, 이 파트에서 다룬 핵심 아이디어들만 짚고 넘어갑니다.

$$L_o(v) = \int_{l \in \Omega} f(l,v) L_i(l) v(l) (n \cdot l)^+ \, dl$$

출력 = 재질함수 × 입사광 × 가시성 × 입사각

* 수식 (11.26), 반사율 방정식

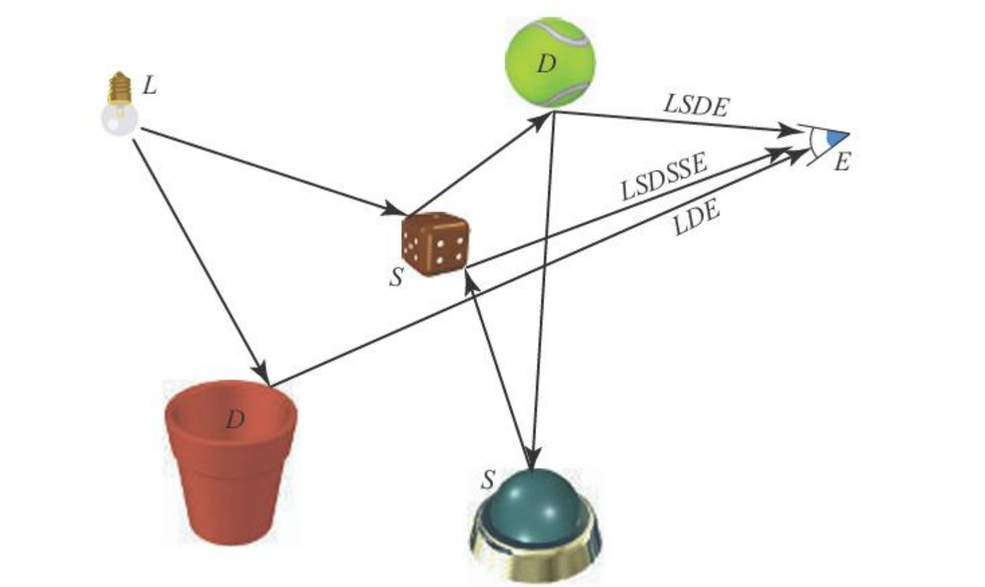
1. **Point light의 경우** 한 방향의 visibility만 보면 되기 때문에 가시성 함수의 평가가 쉽습니다. (단일 방향의 가시성만 고려하면 됨)
2. **Area light의 경우 visibility 평가가 적분**으로 평가됩니다. (Lambertian 표면의 경우 가시성과 입사각만 고려하면 됨)
   - 가시성과 입사각의 평가도 Cone이나 구면 조화 함수 기반의 overlap평가가 가능 (방향성 폐색의 동적계산 파트)
3. **환경(Environment)광의 경우 Lighting, Visibility, Cosine 계수를 모두 적분**해야 합니다 (입사광, 가시성이 입사각에 따라 모두 달라질 수 있음)
   - 적분을 더 싼 연산으로 바꾸거나, 복잡한 함수를 압축/손실된 표현으로 바꾸는 등의 노력이 필요 (11.5 파트부터 구체적으로 다룹니다)
4. 많은 정보를 Precompute 할수록 런타임이 빠르지만 자유도가 줄어듭니다.
   - Visibility와 cosine을 미리 결합하면 normal 정보까지 bake되어 런타임에 normal을 바꿀수 없다.
   - 결합될 수 있는 정보를 LookUp해두는건 런타임도 빠르고 자유도도 챙기지만 메모리가 필요하다
5. **Diffuse에서는 BRDF를 상수로 처리할 수 있었지만 Glossy한 재질등의 specular표현은 BRDF lobe와 함께 표현해야** 합니다.
   - BRDF lobe를 cone으로 근사하고 visibility도 cone으로 근사해서 overlap 하는 등의 계산을 시도할 수 있음



* AI 한테 시키면서 느끼는건데, GPT images 2.0은 normal, tangent 표현을 진짜 개 못해서 열불이 나요
* 하지만 감사합니다 직접 그릴 생각하면 까마득하다

Heckbert 표기법

11.1 장에서 생략하고 지나갔던 Heckbert 표기법을 보고
지금까지 다룬 내용과 앞으로 다룰 내용이 각각 어떤 빛의 경로를 표현하는지를 쉽게 표현해봅시다.



11.1 Heckbert’s notation

Heckbert 표기법(Heckbert's notation)

Heckbert 표기법은 렌더링 과정에서 빛이 광원 (L)에서 출발해서 눈 (E)까지 도달할때까지 어떤 종류의 표면 상호 작용을 거쳤는지를 짧은 문자열로 표현합니다.
핵심은 복잡한 전역 조명 알고리즘을 수식 전체로 설명하는 대신, 어떤 light transport path를 계산할 수 있는지를 정규 표현식 형태로 압축한다는 점입니다.

L : Light (빛이 출발하는 광원)
E : Eye (최종적으로 빛을 관찰하는 카메라 또는 눈)
D : Diffuse interaction (난반사 표면에서의 상호작용)
S : Specular interaction (정반사 표면에서의 상호작용)
필요에 따라서 더 세분화된 표면 종류를 추가할 수도 있음

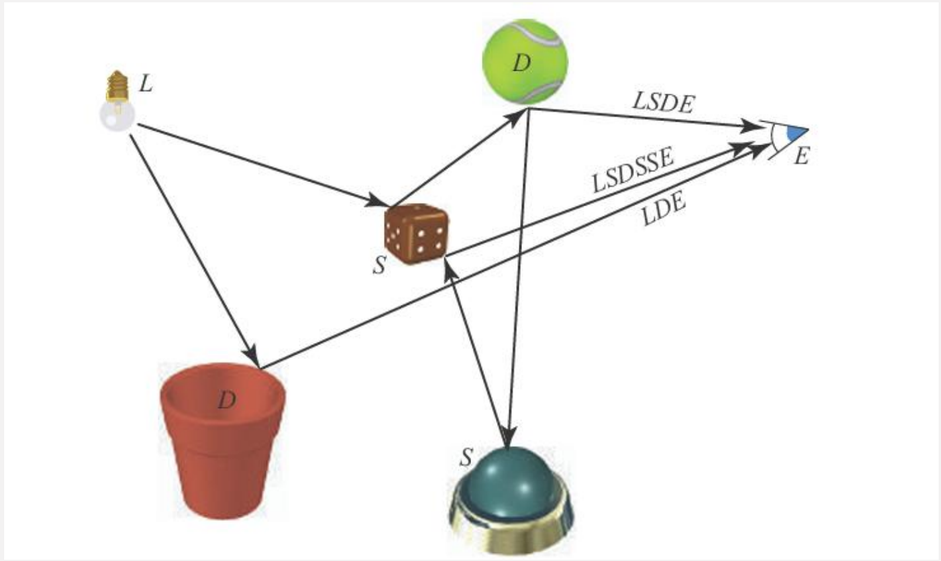
기본적으로 빛이 광원에서 눈까지 도달하는 경로를 다루기 때문에, L로 시작해서 E로 끝나는 표현 방식이 됩니다.

다음의 예시들을 보며 빛의 경로를 어떻게 표현하는지 확인해봅시다.

Heckbert 표기법(Heckbert's notation)

LE : 가장 간단한 경로, 직접 광원을 관찰합니다
LDE : 난반사 표면에서 반사된 빛을 관찰합니다
LSE : 정반사 표면에서 반사된 빛을 관찰합니다

LDSE : 난반사 표면에서 반사된 빛이 정반사 표면에 반사된 뒤 관찰
LSSSSSSSSE : 거울에비친거울에비친거울에비친....



Heckbert 표기법(Heckbert's notation)

표기법으로 뇌절하라고 쓴 게 아니라 정규식 표현법으로 간결하게 표현하기 위함입니다.
또한 다양한 빛의 경로를 하나의 표현식으로 작성할 수 있게 되어
알고리즘이 어떤 빛의 경로를 다루고자 하는지 이해하고
해당 알고리즘의 의의나 특이 사항을 보다 쉽게 이해할 수 있게 됩니다.

Operator	Description	Example	Explanation
$*$	zero or more	$S*$	zero or more specular bounces
$+$	one or more	$D+$	one or more diffuse bounces
$?$	zero or one	$S?$	zero or one specular bounces
$\|$	either/or	$D\|SS$	either a diffuse or two specular bounces
$()$	group	$(D\|S)*$	zero or more of diffuse or specular

Table 11.1. Regular expression notation.

Full Rendering Equation : L(D|S)*E
10장 지역 조명 (별도 매핑 참조 없음) : L(D|S)E
10장 Environment mapping : L(D|S)가 저장되고, 런타임에는 SE
10장 Irradiance Environment Mapping : L(D|S)가 저장되고, 런타임에는 DE
Radiosity : LD*E
Path Tracing : L(D|S)*E
Ambient Occlusion을 사용한 diffuse shading : LDE
상호반사를 고려한 AO : 표현하려는 것 $LD^{+}E$, 런타임에는 DE
Directional Occlusion을 사용한 diffuse shading : LDE

지금까지 본 여러 기법들은 모두 전체 렌더링 방정식 L(D|S)*E의 일부 경로를 계산하거나 근사합니다.
여기서 중요한 구분은, 중간 상호작용이 diffuse인지 specular인지에 따라 필요한 정보의 성격이 크게 달라진다는 점입니다.

확산 전역 조명 (Diffuse Global Illumination)

빛의 차폐만을 다룬 11.3, 11.4 파트를 지나 이제 빛의 반사를 실시간으로 시뮬레이션 하는 여러 방법을 다룹니다.

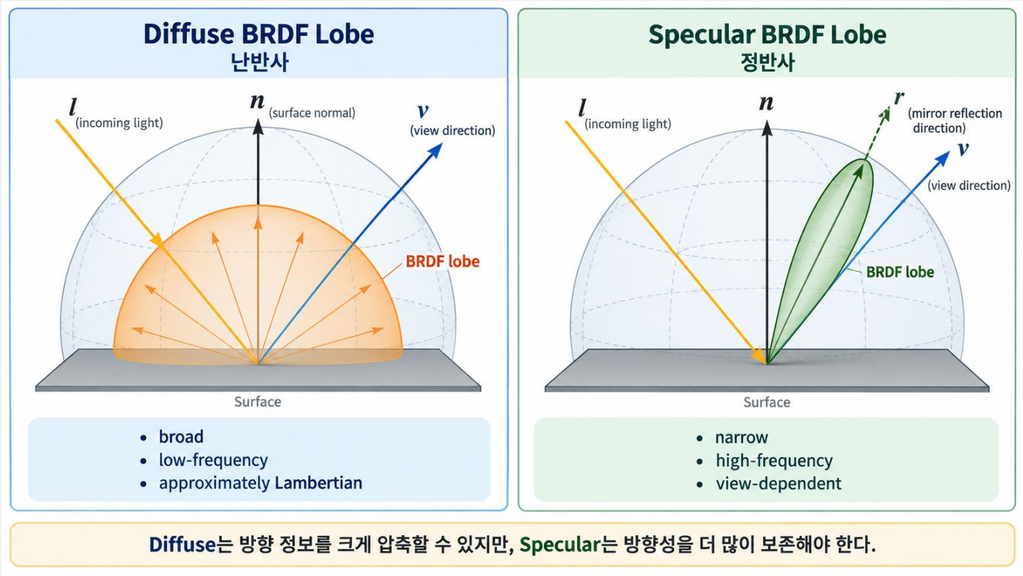
이 방법들은 빛이 눈에 들어오기 직전에 Diffuse 표면 (난반사 표면)에서 반사되는지
아니면 Specular 표면 (정반사 표면)에서 반사되는지를 가정하는 두 부류로 나눌 수 있습니다.

이에 대응하는 빛 경로는 각각 L(D|S)*DE와 L(D|S)*SE로 쓸 수 있으며
많은 기법이 눈으로 들어오기 가장 직전에 반사된 표면의 종류에 따라 일정한 제약을 둡니다.

첫 번째 부류 (시야에 들어오기 전 최종 반사 표면이 Diffuse)는 주로 셰이딩 지점 위 반구 전체의 입사 조명이 부드럽게 변한다고 가정하거나, 그 변화를 무시하고,
두 번째 부류 (시야에 들어오기 전 최종 반사 표면이 Specular)는 입사 방향에 따른 변화율이 크고, 조명이 비교적 작은 입체각 안에서만 참조된다는 사실에 의존합니다.

두 부류의 제약이 이처럼 크게 다르기 때문에, 이들을 따로 다루는 편이 유리합니다.

11.5 파트에서는 확산 전역 조명을,
11.6 파트에서는 스페큘러 표면을 다룬뒤
11.7 파트에서 전망이 밝은 통합적 접근을 살펴보게 됩니다.



표면 사전 조명(Surface Prelighting)

* 4차시에 다룰 내용이지만, 하이엔드 PC나 최신 콘솔 타겟 기기의 경우
많은 아이디어를 차용하고 온몸을 비틀어 어느정도? 성숙해진 기술들이 있습니다.

Radiocsity와 path tracing은 둘 다 오프라인 용도로 설계된 기법입니다.
이들을 실시간 환경에서 사용하려는 시도가 있어왔지만, 그 결과를 실제 제품에 쓰기에는 충분히 성숙하지 않습니다.

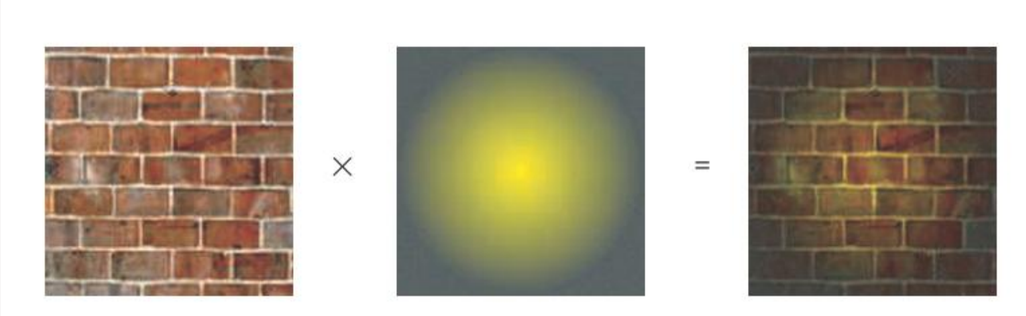
일반적인 방식 중 하나는 이 기법들을 조명 관련 정보를 미리 계산(precompute)하는데 사용하는 것입니다.
비용이 큰 오프라인 처리를 사전에 수행해두고, 그 결과를 저장했다가 화면에 출력할 때 사용하여 고품질 조명을 얻는 식입니다.
AO에서도 다루었듯이, 정적인 장면에 대해 이런 방식으로 미리 계산해 두는 것을 베이킹 (Baking)이라고 합니다.

AO때도 그랬듯이 구우면 런타임에 장면 구성을 변경할 수 없습니다.
모든 장면 지오메트리, 광원, 재질이 유지되어야 합니다.

이런 환경에서 동적인 객체를 구현한다면 지오메트리를 정적(static)인 객체와 동적(dynamic)인 객체로 나눌 수 있습니다.
정적 객체는 사전 계산 과정에 포함되어 조명과 완전히 상호작용하고 (정확히는 상호작용한 데이터를 구워두고)
동적 객체는 수신자 역할만 하게 됩니다. (빛을 차단하지도, 간접 조명 효과를 만들어내지도 않는다)

표면 사전 조명(Surface Prelighting)

미리 계산할 수 있는 조명 정보중, 가장 단순한 형태는 irradiance입니다.
평평한 램버시안 표면이라면 표면의 색상과 irradiance 값 만으로도 조명에 대한 재질의 반응을 완전히 표현할 수 있습니다.
또한 어떤 광원의 기여는 다른 광원과 독립적이기 때문에, 미리 계산된 irradiance 위에 동적 광원을 추가로 얹을 수도 있습니다.



1996년의 Quake와 Quake II는 irradiance 를 미리 계산해 활용한 최초의 상용 인터랙티브 애플리케이션이였습니다.
Quake는 주로 성능 향상을 위해 정적 광원의 직접광 기여분을 미리 계산했고,
Quake II는 여기에 간접광 성분까지 포함시켜 GI 알고리즘으로 더 사실적인 조명을 만들어낸 최초의 게임이 되었습니다.

이 게임은 radiosity 기반 알고리즘을 사용했는데, 램버시안 환경에서 irradiance를 계산하는데 잘 맞았고, 특유의 흐릿하고 저주파적인 그림자와도 어울렸습니다.

표면 사전 조명(Surface Prelighting)

미리 계산된 irradiance값은 보통 별도의 텍스쳐 집합에 저장된 diffuse 또는 albedo맵과 곱해집니다.
이론상으로는 방사도(irradiance * diffuse color)를 **미리 계산해 하나의 텍스쳐 집합에 저장할 수도 있지만, 실무적인 이유로 이 방식은 배제**됩니다.
색상 맵은 보통 주파수가 상당히 높고 각종 타일링을 사용하며 모델 곳곳에서 일부를 재사용합니다.
반명 irradiance값은 주파수가 훨씬 낮고, 쉽게 재사용할 수 없습니다.

오늘날 이 방식은 제약이 극심한 하드웨어 플랫폼을 제외하고는 사용되지 않습니다.
Irradiance는 정의상 특정 법선 방향에 대해 계산되기 때문에, normal mapping을 사용해 고주파 디테일을 표현할 수 없기 때문입니다.



* irradiance는 단위 면적당 표면에서 **받는** 에너지 양

오늘의 순서 - 4차시

00 확산 전역 조명 — 11.5

*아... 결국 5차시를 가는구나...

01 반사 전역 조명 — 11.6

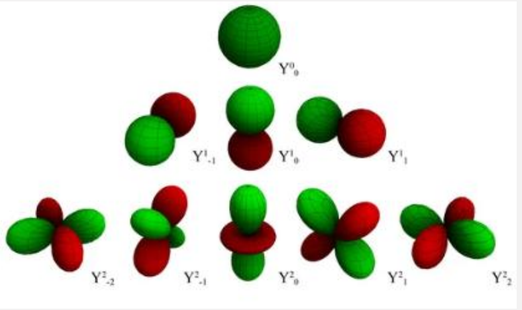
*아마도 오늘 못합니다 6차시 가니...?

방향성 표면 사전 조명(Directional Surface Prelighting)

Lambertian 표면에서 사전 조명과 normal mapping을 함께 쓰려면, irradiance가 법선에 따라 어떻게 변하는지를 표현할 방법이 필요합니다.
또는 동적인 지오메트리에 간접광을 제공하기 위해서는, 가능한 모든 표면 방향에 대한 값이 필요합니다.

하지만 이미 우리는 10.3 (지역조명 – 구면과 반구면 함수)에서 법선 방향에 달라지는 조명을 구하는 여러 방식을 보았습니다.

가장 일반적인 방법은 구면조화 함수 (SH)를 사용해 구면의 irradiance를 저장하는 것입니다.
구면 조화 계수 개수를 9개 (3차 SH) 사용하면 품질은 훌륭하지만 저장 공간과 대역폭이 비싸지고,
4개 (2차 SH)만 쓰면 디테일이 사라지고 조명의 대비가 약해지며, normal map도 덜 두드러지게 됩니다.



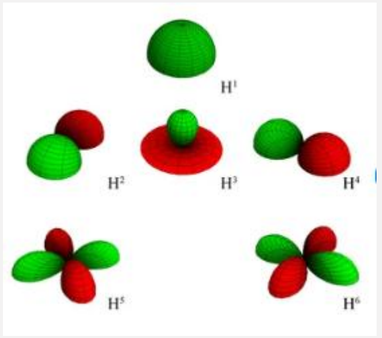
Chen은 Halo 3에서 이 방법의 변형을 사용했는데, 더 낮은 비용으로 3차 SH 수준의 품질을 얻기 위해
구면에서 가장 지배적인 광원을 뽑아 색과 방향을 따로 저장하고, 잔차를 2차 SH로 인코딩했습니다.

이렇게 하면 스칼라 계수가 27개 ($9_{(3차\ SH)} * 3_{(RGB)}$)에서 18개 ($4_{(2차\ SH)} * 3_{(RGB)} + 3_{(RGB)} + 3_{(Direction)}$)로 줄어들며 품질 손실은 거의 없습니다.

방향성 표면 사전 조명(Directional Surface Prelighting)

Habel등의 H-Basis는 또 다른 대안입니다. 이는 반구 신호만 인코딩 하기 때문에, 구면 조화 함수와 같은 정밀도를 더 적은 계수를 사용해 얻을 수 있습니다.
계수 6개 만으로 3차 SH에 필적하는 품질을 낼 수 있으나, 반구에 대해서만 정의되기 때문에 이를 올바르게 표현하기 위해서는
표면상의 지역 좌표계 (tangent space)가 필요합니다.

H-basis성분을 텍스쳐에 저장한다면, 그 해상도는 기저를 이루는 탄젠트 공간의 변화를 따라갈 수 있을 만큼 충분히 높아야 합니다.
탄젠트 공간이 크게 다른 여러 삼각형이 같은 텍셀을 덮으면 복원된 신호가 정확하지 않게 됩니다.



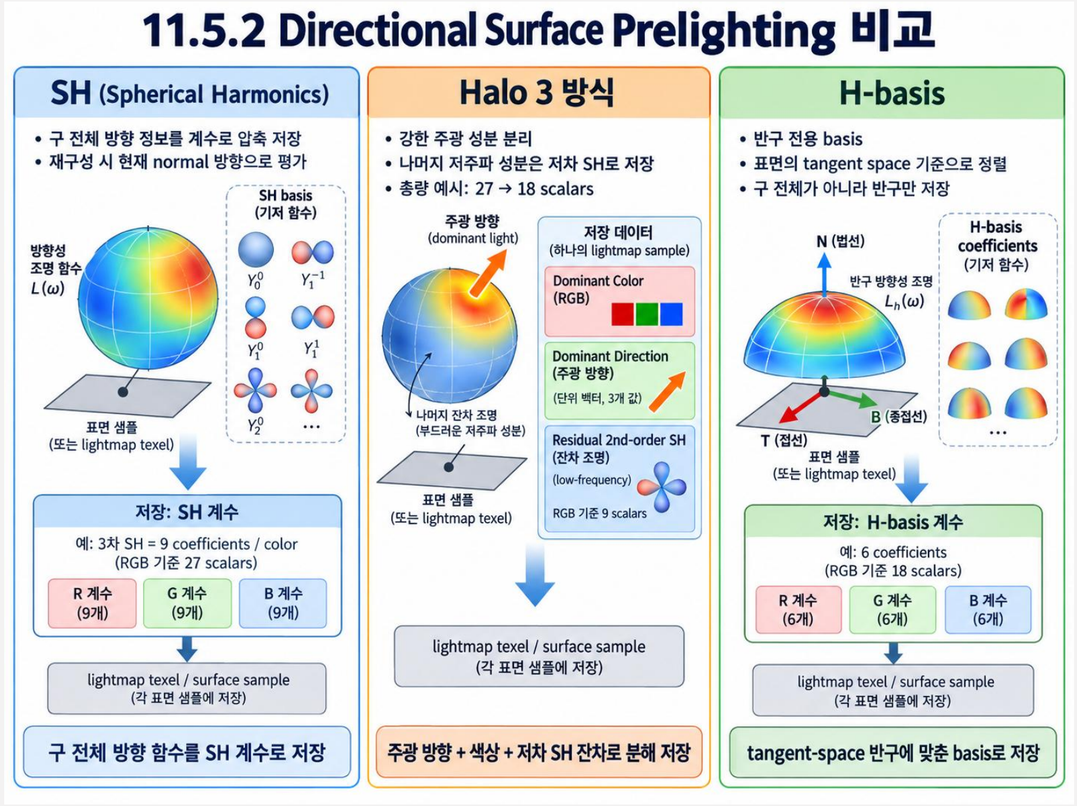


리얼타임 렌더링 4판 · 11장 전역 조명 · 4차시

방향성 표면 사전 조명(Directional Surface Prelighting)

구면 조화 함수 (SH)와 H-basis 모두 링잉 현상이 나타날 수 있다는 문제가 있습니다.
프리필터링으로 이 효과를 완화할 수 있지만, 그 과정에서 조명도 함께 부드러워지는데 이것이 늘 바람직하지는 않습니다.

게다가 비용이 덜한 변형들조차 저장공간과 연산 양쪽에서 부담이 상당하게 됩니다.
저사양 플랫폼이나 VR 렌더링 처럼 제약이 심한 경우, 이 비용을 감당하기 어려울 수 있습니다.

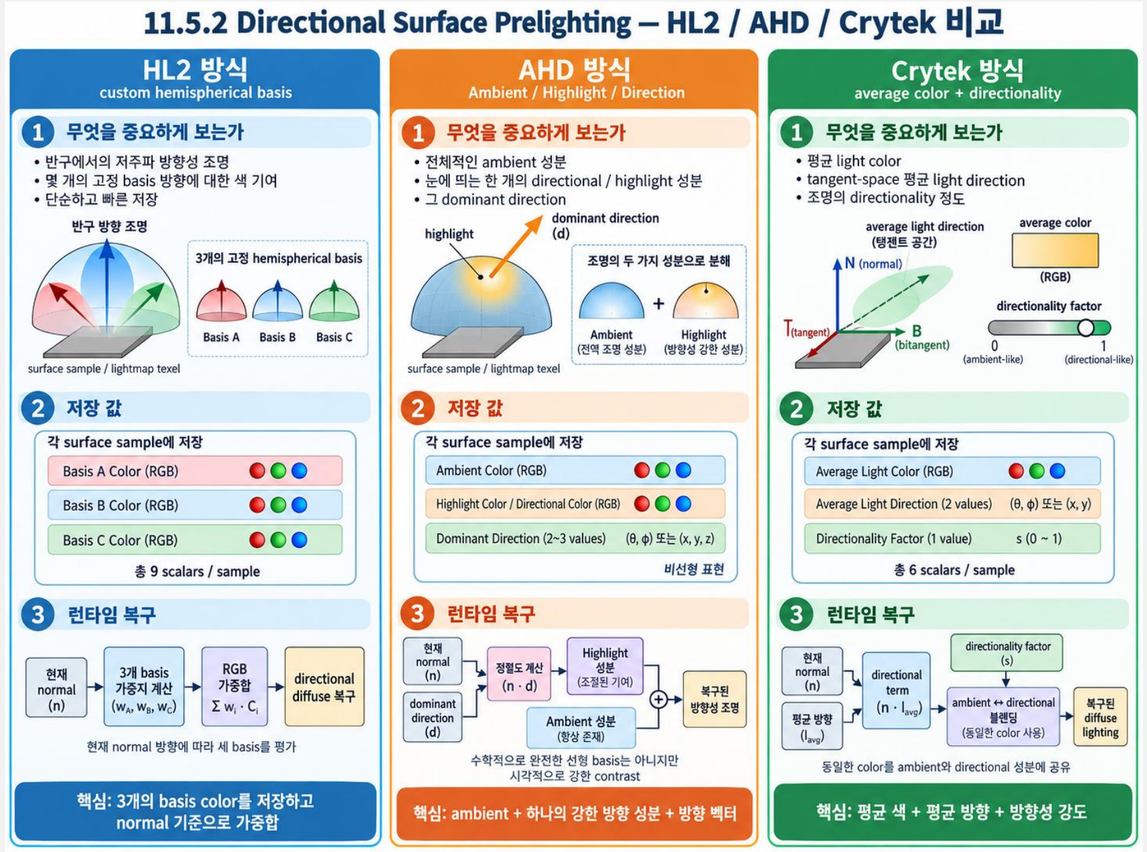


방향성 표면 사전 조명(Directional Surface Prelighting)

구면/반구면 기저의 비용 문제 때문에 더 단순한 대안들도 쓰이곤 합니다.
Half-Life 2는 세개의 색상값, 즉 샘플당 총 9개의 계수를 저장하는 맞춤형 반구 기저 (10.3.3)를 사용합니다.

또한 Ambient/HighLight/Direction(AHD)기저 역시 단순함에도 인기 있는 선택지입니다.

Crytek은 Far cry에서 이 방식의 변형을 사용하였고, 유니티의 LightMap도 이와 비슷한 형태의 구현을 포함하고 있습니다.



리얼타임 렌더링 4판 · 11장 전역 조명 · 4차시

방향성 표면 사전 조명(Directional Surface Prelighting)

Half-Life 2, AHD, Crytek의 표현방법은 비선형입니다.
따라서 엄밀히 말하면 텍셀, 정점정보등을 통해 이 사이의 개별 성분을 선형보간 하는 것은 수학적으로 옳지 않습니다.
예를 들어 그림자의 경계처럼 지배적 광원의 영향이 급격히 바뀌는 곳에서는 시각적인 아티팩트가 나타날 수 있습니다.

하지만 이러한 부정확성에도 불구하고 결과물은 보기에 좋습니다.

Ambient영역과 방향성 조명 영역 사이의 대비가 크기 때문에 노멀 맵의 효과가 강조되는데, 이것은 종종 바람직한 결과가 됩니다.

덧붙여 지배적인 조명의 방향을 표현하는 성분은 BRDF의 specular 반응을 계산할 때도 활용할 수 있어,
광택이 낮은 재질에 대해 환경 맵을 대신할 저비용 대안이 될 수 있습니다.

방향성 표면 사전 조명(Directional Surface Prelighting)

저비용 처리 방법 대신 고비용 처리 방법도 알아봅시다.
디 오더 : 1886에서는 표면의 탄젠트 표면에서 입사 방향으로의 radiance를 표현한 가우시안 로브 집합 (10.3장에서 다루었던)으로 표현합니다.
장면에서 사용되는 조명의 복잡도에 따라 5~9개의 로브를 사용해 텍스쳐 맵에 저장하고
Diffuse 값을 계산 할때 각각의 구면 가우시안을 표면의 normal(법선) 방향의 코사인 로브와 곱하여 표현합니다.

이 표현은 Diffuse 뿐 아니라 스페큘러 BRDF로브와도 결합하여 흐린 광택 효과를 낼 수 있을만큼 정밀합니다.



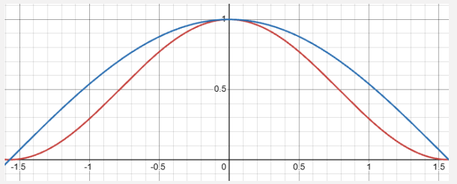
방향성 표면 사전 조명(Directional Surface Prelighting)

표면 위쪽 반구 뿐 아니라 임의의 방향에 대한 조명 정보가 필요하다면 (동적인 물체에도 간접광을 제공하려면)
반구가 아니라 전체 구 방향에서 오는 값을 인코딩 할 수 있어야 합니다.
SG lobe를 반대쪽 반구 방향에도 배치하여 이를 해결할 수 있겠지만, 2배 더 많은 비용이 필요해지는 문제가 생깁니다.

이 비용을 해결하기 위해 단일 구면 조화 함수에 인코딩하는건 링잉 현상이 눈에 띄고, 이럴때 사용되는 것이 10.3장에서 다루었던 Ambient Cube입니다.

이 6개의 방향 정보에 clamped $\cos^2$ lobes를 배치합니다.

런타임에서는 반구방향의 값만 평가하면 되기 때문에, 3개의 로브만 평가하게 되어 비용을 절감할 수 있게 됩니다.
이 Ambient Cube의 복원 품질은 2차 구면 조화 함수 (SH)와 비슷한 수준입니다.

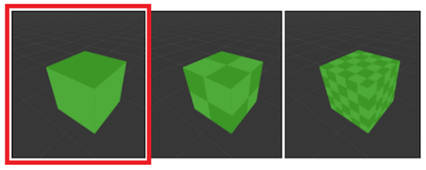


*왜 $\cos^2$인지는 파악 못했습니다
*그냥 방향에 대한 가중치를 cos보다 더 강하게 주기 위함인듯
*큐브 형태니까 뭔가 감은 오는데 제가 수알못이라

Ambient cube

- 각 큐브 면에 텍셀 하나가 대응되는 cube map과 비슷하게 볼 수 있다.
  + Sloan은 ambient cube와 spherical harmonics 사이의 변환식을 유도하였음
- 어떤 방향 d에 대해 평가할 때 d의 x, y, z 부호에 맞는 세 면만 읽으면 된다.
  + 기하 구조 상 최대 세 개의 면까지만 고려가 가능하기 때문
- 샘플이 극단적으로 적은 환경이나 마찬가지이기 때문에, 품질은 낮은 편이다.
  + 큐브의 여섯 면 대신 여덟 개의 꼭짓점을 보간하여 개선하는 우회법이 있음

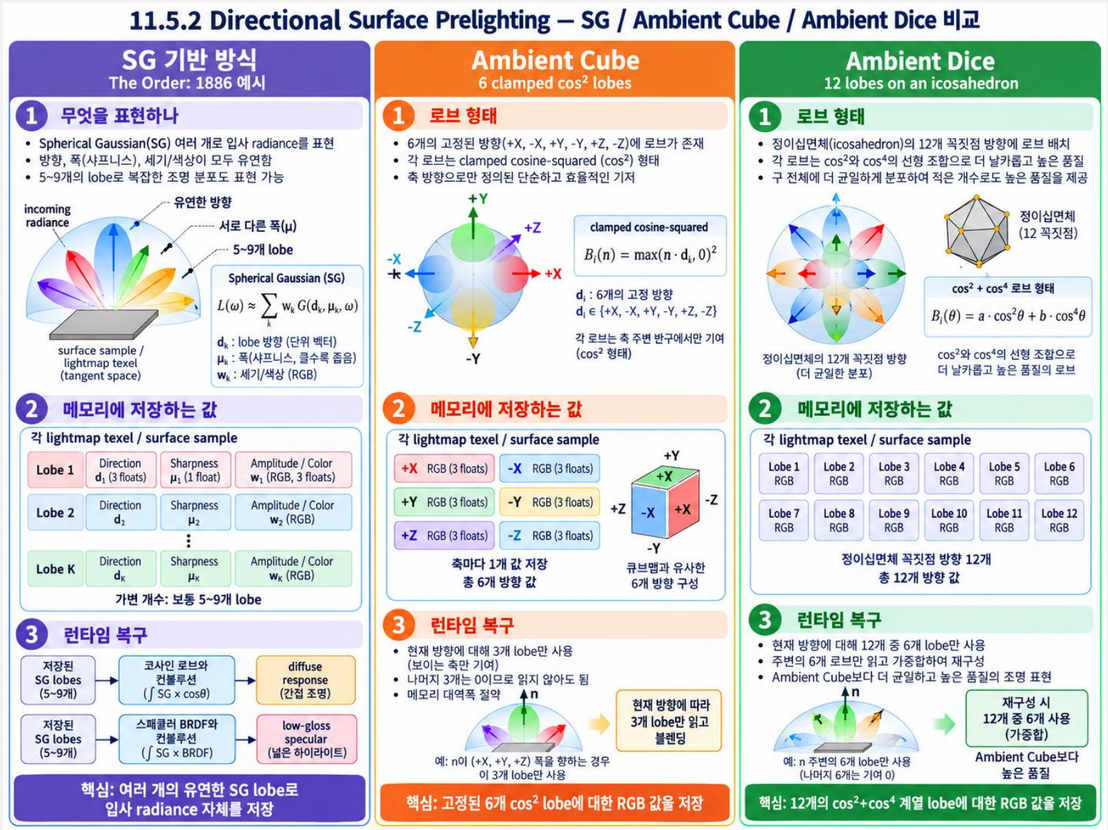
Lightmapping Settings(라이트매핑 설정)에서 **Lightmap Resolution**(라이트맵 해상도) 매개변수는 라이트맵에서 단위당 사용되는 텍셀 수를 제어합니다. 다음은 다른 **Lightmap Resolution**(라이트맵 해상도) 설정이 사용된 정육면체의 예입니다. 해상도가 높아지면 할 일이 많아짐을 확인할 수 있습니다.

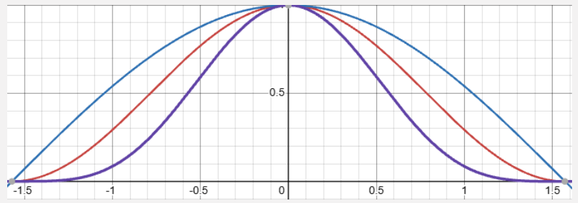


방향성 표면 사전 조명(Directional Surface Prelighting)

Ambient Cube보다 더 높은 품질이 필요하다면 Ambient Dice를 사용할 수 있습니다.
이 방식은 정이십면체의 꼭짓점 방향으로 정렬된 12개의 로브를 저장하며, 각 로브는 $\cos^2$와 $\cos^4$ 의 선형 결합입니다.
Ambient Cube가 그랬듯이, 반구 방향으로만 평가하면 되기 때문에 6개의 로브만 읽고 가중합으로 색상을 평가할 수 있게 됩니다.

이런 Ambient Cube의 복원 품질은 3차 구면 조화 함수에 견줄만 하다고 합니다.





*얘도 모르겠다! 면적이 좁아진만큼 더 방향에 대한 가중치가 강하게 들어가나

미리 계산된 전이 함수(Precomputed Transfer)

미리 계산된 조명은 대단히 훌륭해 보이지만, 본질적으로 정적씬을 구워둔 것입니다.
지오메트리나 조명에 조금이라도 변화가 생기면, 구워둔 전체 값이 무효가 될 수 있습니다.

씬의 지오메트리가 변하지 않고 조명만 변한다고 가정하면,
빛과 모델이 어떻게 상호작용하는지를 미리 계산해둘 수 있습니다.

이런 입사 조명을 받아 장면 전반의 입사광 분포로 변환하는 함수를 전이함수 (transfer function)이라고 합니다.
이를 미리 계산하는 방법을 사전 계산 전이(precomputed transfer), precomputed radiance transfer(PRT) 방식이라고 부릅니다.

조명을 오프라인에서 완전히 베이킹 하는것과 달리, 이 파트부터 설명할 기법은 무시할 수 없는 런타임 비용을 수반합니다.

장면을 화면에 표시할 때 특정 조명 구성에 대한 radiance값을 계산해야 하기 때문입니다.
이를 위해 실제 직접 광의 양을 시스템이 주입(inject)한 뒤, 전이 함수를 통해 그것을 장면 전체로 전파합니다.

일부 기법은 이 직접광이 환경 맵에서 온다고 가정하며, 다른 방식들은 조명 구성을 임의로 두고 유연하게 변경할 수 있게 합니다.

미리 계산된 전이 함수(Precomputed Transfer)

PRT 개념을 그래픽스에 도입한건 Sloan등입니다. 이들은 이를 구면 조화 함수의 관점에서 설명했지만, 이 방법이 반드시 SH를 사용해야 하는 것은 아닙니다.

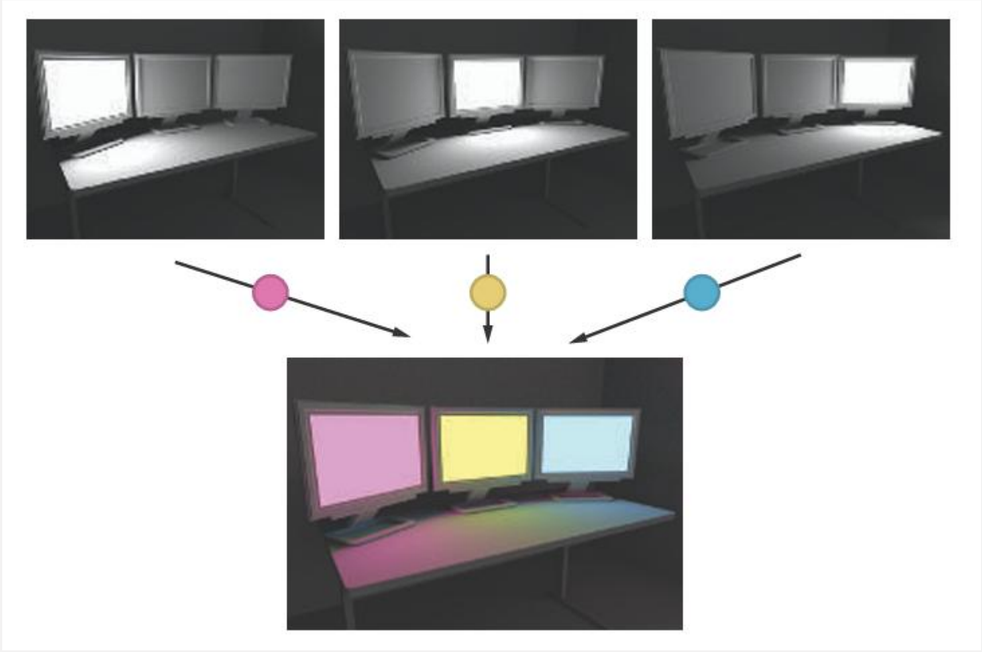
직접광을 (되도록이면 적은 개수의) 광원으로 기술 할 수 있다면, 각 광원이 장면을 어떻게 비추는지를 미리 계산해둘 수 있습니다.
아래 예시 이미지를 보도록 합시다.

각 화면의 모니터는 단일 색상만 표현할 수 있고, 밝기는 0에서 1까지로 제어할 수 있다면 각 모니터가 방에 미치는 영향을 독립적으로 미리 계산 할 수 있습니다.
빛의 전달과 합산은 선형 연산이기 때문에 세 모니터를 모두 켰을때 장면이 받는 조명은 각 모니터에서 오는 빛 (직, 간접광 모두)의 합과 같습니다.

각 모니터의 조명은 다른 해에 영향을 주지 않기 때문에,
한 화면의 밝기를 절반으로 낮추더라도 그 화면 자신의 기여분만 달라집니다.

미리 계산된 조명 해에 해당 화면의 밝기를 곱하고, 그 결과들을 더하기만 하면 됩니다.

모니터를 켜고 끄거나, 더 밝거나, 더 어둡게 하거나, 색을 바꾸더라도
최종 조명 결과를 얻는데 필요한 것은 곱셈과 덧셈 뿐입니다.



리얼타임 렌더링 4판 · 11장 전역 조명 · 4차시

미리 계산된 전이 함수(Precomputed Transfer)

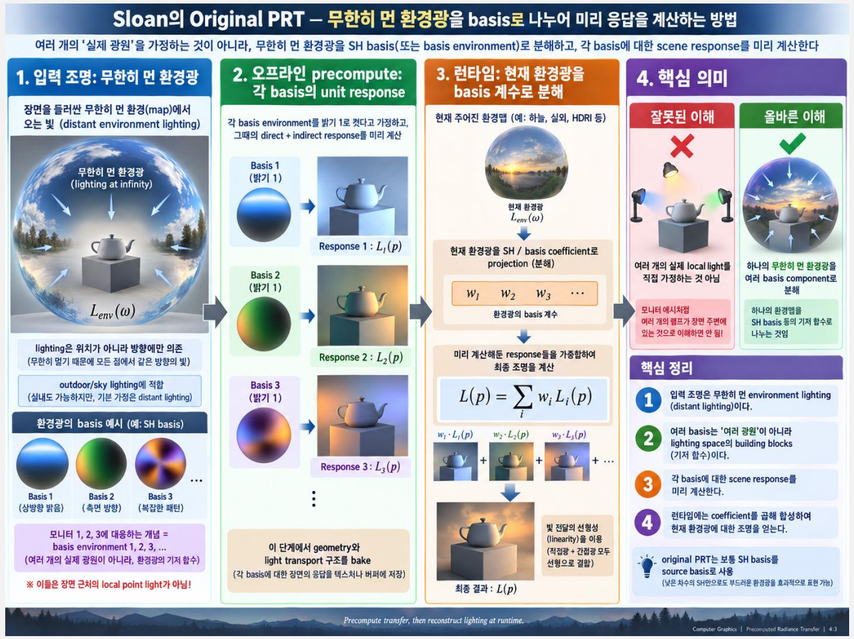
이런 미리 계산된 전이 함수를 사용해 최종 radiance를 계산하는것을 아래와 같이 수식으로 표현할 수 있습니다.

$$L(p) = \sum_i L_i(p) w_i$$

$L(p)$ = 점 p에서 최종 radiance $L_i(p)$ = 화면 i로부터의 미리 계산된 단위 기여분 w_i = 화면 i의 현재 밝기

이 식은 수학적 의미에서 벡터 공간을 정의하고, Li가 이 공간의 기저 벡터가 됩니다.
가능한 모든 조명은 각 광원 기여분의 선형 결합으로 만들어 낼 수 있습니다.

Sloan등의 원 PRT논문도 동일한 논리를 사용하지만, 구면 조화 함수로 표현된 무한히 먼 조명 환경이라는 맥락에서 다룹니다.
장면이 모니터 화면에 어떻게 반응하는지를 저장하는 대신, 환경광을 여러 개의 구면 조화 기저 함수로 분리하여 계산하는 방법을 다루었습니다.



리얼타임 렌더링 4판 · 11장 전역 조명 · 4차시

미리 계산된 전이 함수(Precomputed Transfer)

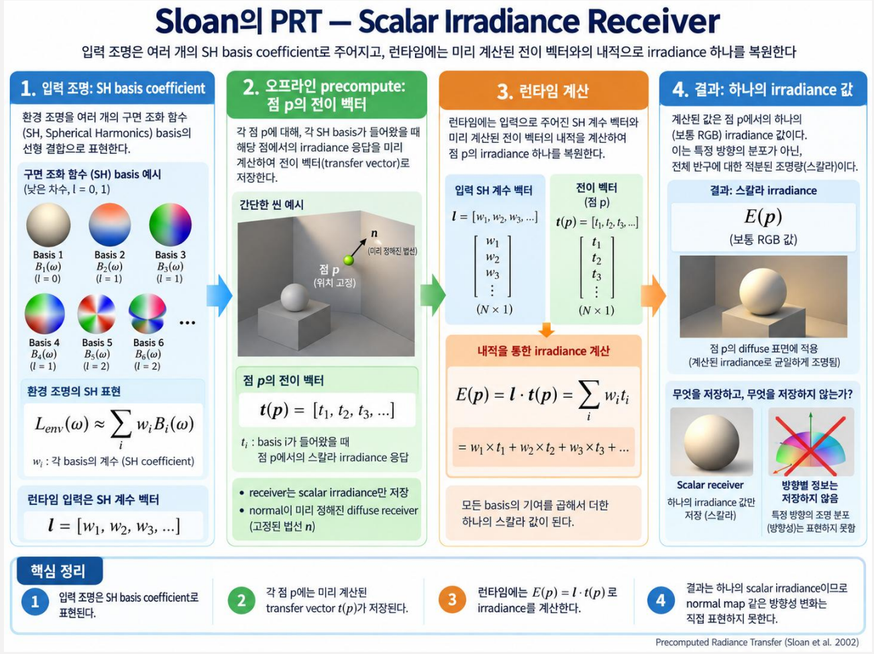
장면에 빛을 주입하는데에 쓰이는 기저의 선택은, 최종 조명을 표현하는데에 쓰이는 표현 방식과 무관합니다.

예를 들자면 장면에 광원이 어떻게 조명되는지는 구면 조화 함수를 사용하면서, 임의의 점에 도달하는 radiance의 양을 저장하는 데에는 다른 기저를 고를 수 있습니다.
저장에 ambient cube를 사용했다면 위쪽에서 오는 radiance와 옆면에서 오는 radiance를 계산하게 됩니다.
이 각 방향에 대한 전달이, 총 전달을 나타내는 하나의 스칼라 값이 아니라 따로 저장되게 됩니다.

* 당장 이해되지 않아도 괜찮습니다. 뒤에 구체적인 예시를 통해 자연스럽게 둘을 구분 할 수 있습니다.

Sloan의 PRT논문은 두가지 경우를 분석하였습니다.
첫 번째는 수신 기저 (receiver basis)가 표면의 irradiance 값인 경우입니다.
이때 수신 표면은 법선이 미리 정해진 완전한 확산 표면이여야 하며, 따라서 디테일을 위한 노멀 맵을 사용할 수 없습니다.

전달 함수는 입력 조명의 SH 투영과, 장면 전반에 걸쳐 공간적으로 변하는 미리 계산된 전달 벡터 사이의 내적 형태를 띄게 됩니다.



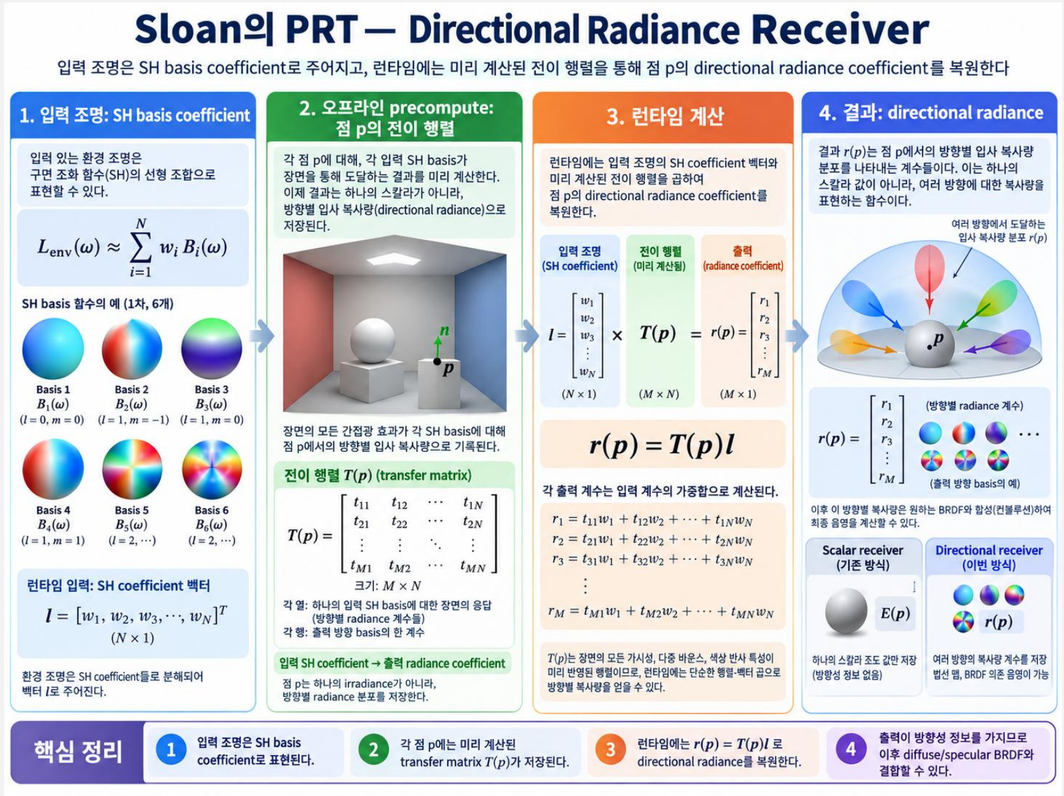
리얼타임 렌더링 4판 · 11장 전역 조명 · 4차시

미리 계산된 전이 함수(Precomputed Transfer)

Scalar Irradiance Receiver는 11.5.1장에서 다룬 것 처럼 normal의 변경에도 취약하고, 비 Lambertian 재질을 사용할 수도 없습니다.
논문에서는 이를 해결하기 위해 Receiver basis도 구면조화 벡터를 표현하게 하는 두번째 방법을 제시합니다.

이 연산은 구(또는 반구) 전체에 걸친 완전한 radiance분포를 제공하므로, 어떤 BRDF와도 제대로 연산할 수 있습니다.

이때 전이 함수는 SH벡터를 다른 SH벡터로 매핑하며, 행렬 곱셈의 형태를 가지게 됩니다.



미리 계산된 전이 함수(Precomputed Transfer)

이 곱셈 연산은 계산량과 메모리 양쪽 모두에서 비용이 큽니다. 소스와 수신 기저 양쪽에서 3차 SH를 사용한다면,
단색 표현을 위해 장면의 모든 점마다 9x9 행렬을 저장해야 합니다. RGB처리를 위해서는 한 점당 243개의 scalar값을 저장하게 됩니다.

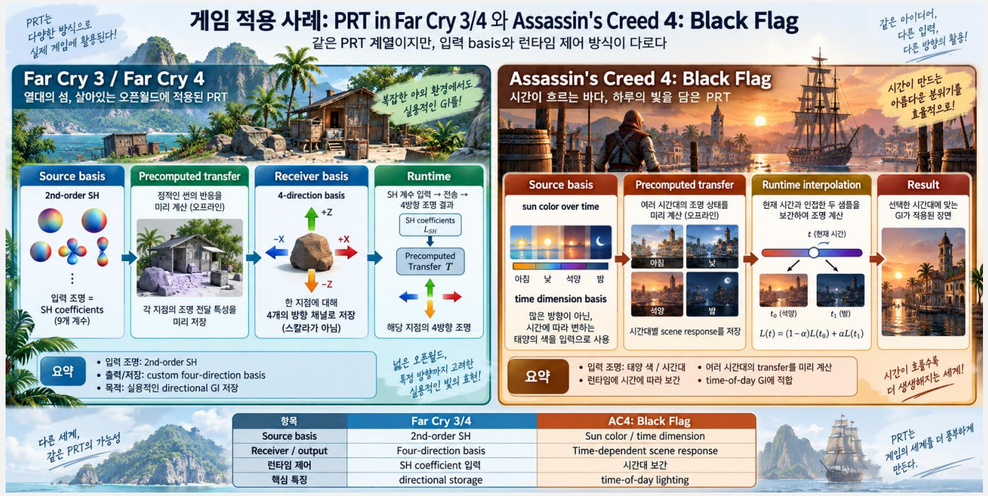
이 문제를 해결 하기 위해 전이 벡터나 행렬을 직접 저장하는 대신, 그 전체 집합을 주성분 분석 기법(PCA)으로 분석하였습니다.

Sloan의 경우는 25x25 transfer matrix (아마도 5차 SH를 사용했나봐 너무 무섭다), 625차원의 데이터를 256차원으로 줄였습니다.
이 정도도 일반적인 실시간 응용에서 사용하기에는 여전히 비싸지만, 이후의 많은 빛 전달 알고리즘이 데이터 압축 수단으로 PCA를 받아들였습니다.

이런 종류의 차원 축소는 본질적으로 손실이 발생합니다. 대개는 근사에 그치므로 어느정도 품질 저하가 발생하고
이를 해결하기 위해 전이 행렬 집합을 여러 군집으로 나누고, 각각에 대해 PCA를 수행하며
군집 경계에서 불연속이 생기지 않도록 하는 최적화 단계도 포함하였다고 합니다.

* PCA에 대한 세부 내용은 다루지 않습니다. 갈길이 멀어욧

PRT는 시간대와 날씨가 동적으로 변하는 야외 지역의 게임플레이 중심인 작품에서 특히 인기가 높았습니다.
Far Cry 3와 Far Cry 4는 Source Basis로 2차 SH를, 수신 기저로는 맞춤형 4방향 기저를 사용하는 PRT를 사용합니다.
Assassin's Creed 4는 Source Basis로 기저 함수 하나 (태양 색상)만 사용하되, 서로 다른 시간대에 대한 전이를 미리 계산하였습니다.



리얼타임 렌더링 4판 · 11장 전역 조명 · 4차시

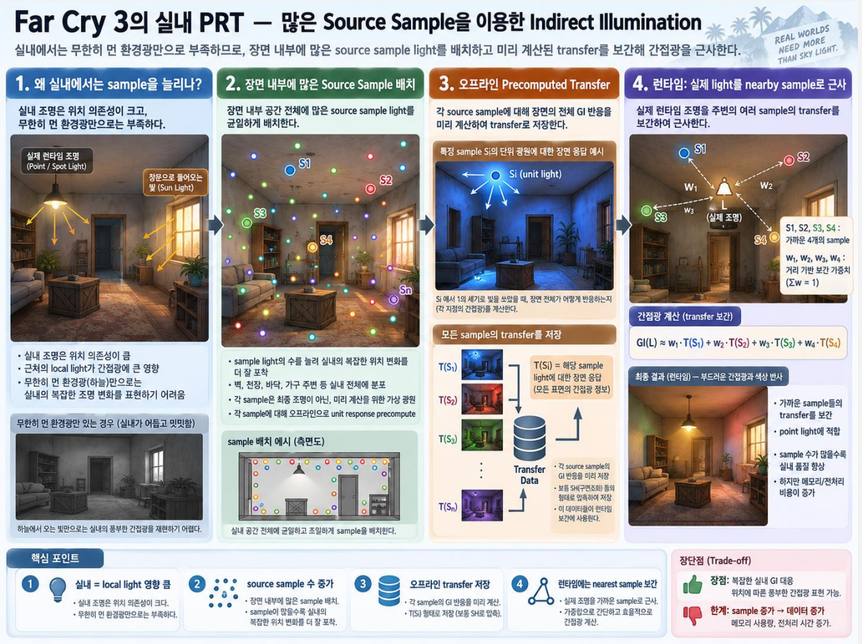
미리 계산된 전이 함수(Precomputed Transfer)

원래의 PRT 기법은 무한히 먼 주변 조명을 가정했습니다. 이는 야외 장면은 꽤 잘 모델링 하지만, 실내 환경에는 지나치게 제약적입니다.
하지만 앞서 모니터의 예시를 들었듯이 이 개념 자체는 조명의 원래 출처가 무엇인지에 완전히 무관합니다.

Kristensen등은 장면 전체에 흩어진 광원 집합에 대해 PRT를 계산하는 방법을 설명했습니다. 이는 Source basis 함수를 아주 많이 두는 것에 해당합니다.
이어서 광원들을 군집으로 묶고, 수신 지오메트리를 여러 구역으로 분할하여 각 구역이 서로 다른 광원 부분 집합의 영향을 받도록 합니다.
이 과정으로 전이 데이터를 크게 압축할 수 있습니다.

런타임에는 임의로 배치된 광원에 의한 조명을 미리 계산된 집합 중 가장 가까운 광원들의 데이터를 보간하여 근사합니다.

Gilabert와 Stefanov는 이 방법을 Far Cry 3의 간접 조명 생성에 활용했습니다.
이는 기본적으로 점 광원만 처리할 수 있으며, 다른 유형의 광원을 지원하도록 확장할 수는 있지만 각 광원의 자유도에 따라 비용이 지수적으로 늘어납니다.



리얼타임 렌더링 4판 · 11장 전역 조명 · 4차시

미리 계산된 전이 함수(Precomputed Transfer)

지금까지 논의한 PRT 기법들은 광원을 모델링 하는데 쓰이는 몇 개의 요소로부터의 전달을 미리 계산했습니다.
또 다른 기법들은 표면 사이의 전달을 미리 계산합니다. 이런 시스템에서는 조명의 실제 출처가 무관해지게 됩니다.

이들 기법의 입력을 어떤 표면 집합에서 나가는 Radiance (또는 Diffuse Surface를 가정하면 irradiance 같은 관련 물리량) 이므로, 어떤 광원이든 사용할 수 있습니다.

이 직접 조명 계산에는 7장에서 다룬 그림자나, 10장의 irradiance 환경맵, 또는 11장에서 다룬 Occlusion기법을 쓸 수 있습니다.
또한 어떤 표면이든, 나가는 radiance를 원하는 값으로 설정하기만 하면 손쉽게 발광체로 만들어 면광원으로 바꿀 수 있습니다.

이런 surface-to-surface transfer 계열의 가장 유명한 시스템이 Geomerics의 Enlighten입니다.

Enlighten에서는 빛 전달을 계산하기 위해 장면은 램버시안이라고 가정됩니다.
Heckbert의 표기법으로 쓰면 처리되는 경로 집합은 LD*(D|S)E 인데 눈에 들어오기 직전의 마지막 표면은 확산 전용일 필요가 없기 때문입니다.
(light의 transfer는 diffuse로 싸게 계산하지만, 최종 shading 단계에서는 surface의 BRDF를 정확히 반영한다)

https://docs.unity3d.com/2018.2/Documentation/uploads/ExpertGuides/Using_Enlighten_with_Unity.pdf

미리 계산된 전이 함수(Precomputed Transfer)

이런 Enlighten류 시스템은 scene surface들을 Source elements와 Receiver elements로 추상화 합니다.

Source elements는 표면 위에 존재하며, diffuse color, position, normal등 surface property의 일부를 가지고,
Precompute 과정에서는 source element의 light가 얼마나 receiver에게 전달되는지를 계산합니다.

이 정보의 정확한 형태는 소스 요소가 무엇인지, 수신자 측에서 조명을 모으는 데 어떤 기저를 쓰는지에 달려있습니다.
가장 단순한 형태는 소스 요소가 점이고, 수신 위치에서 irradiance를 생성하는 것이 관심사가 됩니다.
이 경우 전달 계수는 단지 소스와 수신자 사이의 상호 가시성 일 뿐입니다.
런타임에는 모든 소스 요소의 나가는 radiance가 시스템에 제공됩니다.
이 정보로부터 미리 계산된 가시성과 source, receiver의 위치, 방향과 같은 알려진 정보를 이용해 반사율 방정식을 수치적으로 적분할 수 있게 됩니다.

이런 방식으로 빛의 한번의 반사를 수행할 수 있으며, 간접 조명의 대부분은 이 첫 반사에서 나타나기 때문에 그럴듯한 조명을 얻을 수 있게 됩니다.

또한 한번 propagation하고 나면, receiver가 받은 indirect light를 통해 이를 다시 source로 두고 다음 프레임에 두번째 반사를 구현할 수 있게 됩니다.

* 당장 이해가 되지 않더라도 넘어갑시다, 이후 개념까지 설명한 뒤에 Radiosity 알고리즘과 비교해보면 조금 더 쉽게 파악할 수 있습니다

미리 계산된 전이 함수(Precomputed Transfer)

하지만 source를 점으로 두게 되면, 연결 수가 대단히 많아지게 됩니다.
성능을 개선하기 위해 법선과 색상이 비슷한 영역을 나타내는 점들의 군집을 소스 집합으로 사용할 수도 있습니다.

이 경우 전이 계수는 Radiosity 알고리즘에서 다루었던 form factor와 같아지게 됩니다.
이는 고전적인 radiosity알고리즘과 유사해보이지만, 이는 점진적 래디오시티 (progressive radiosity)의 아이디어에 기반하여
한 번에 빛의 반사 한 번만 계산하며, 연립 일차 방정식을 직접 풀지 않습니다.

이 방식에서는 하나의 패치가 반복적인 과정을 통해 다른 패치들로 부터 얼마만큼의 에너지를 받는지 결정할 수 있습니다.
저장된 radiance를 수신위치로 전달하는 과정을 gathering이라고 부릅니다.



리얼타임 렌더링 4판 · 11장 전역 조명 · 4차시

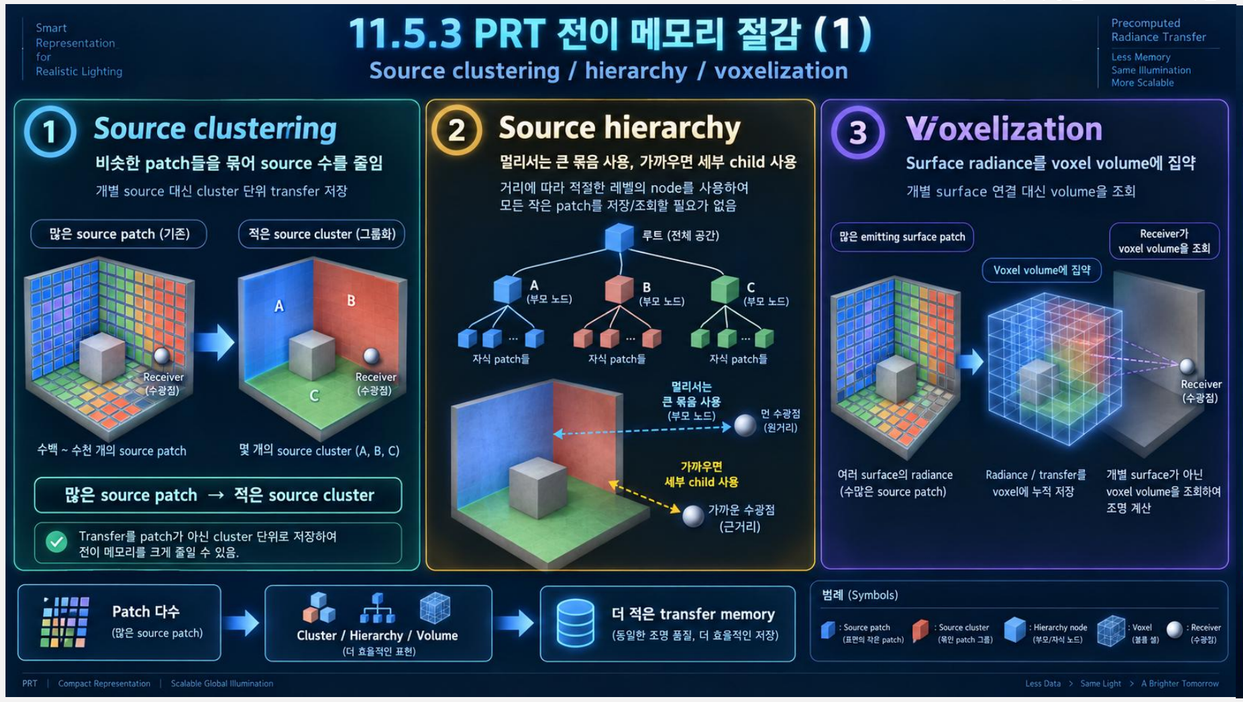
미리 계산된 전이 함수(Precomputed Transfer)

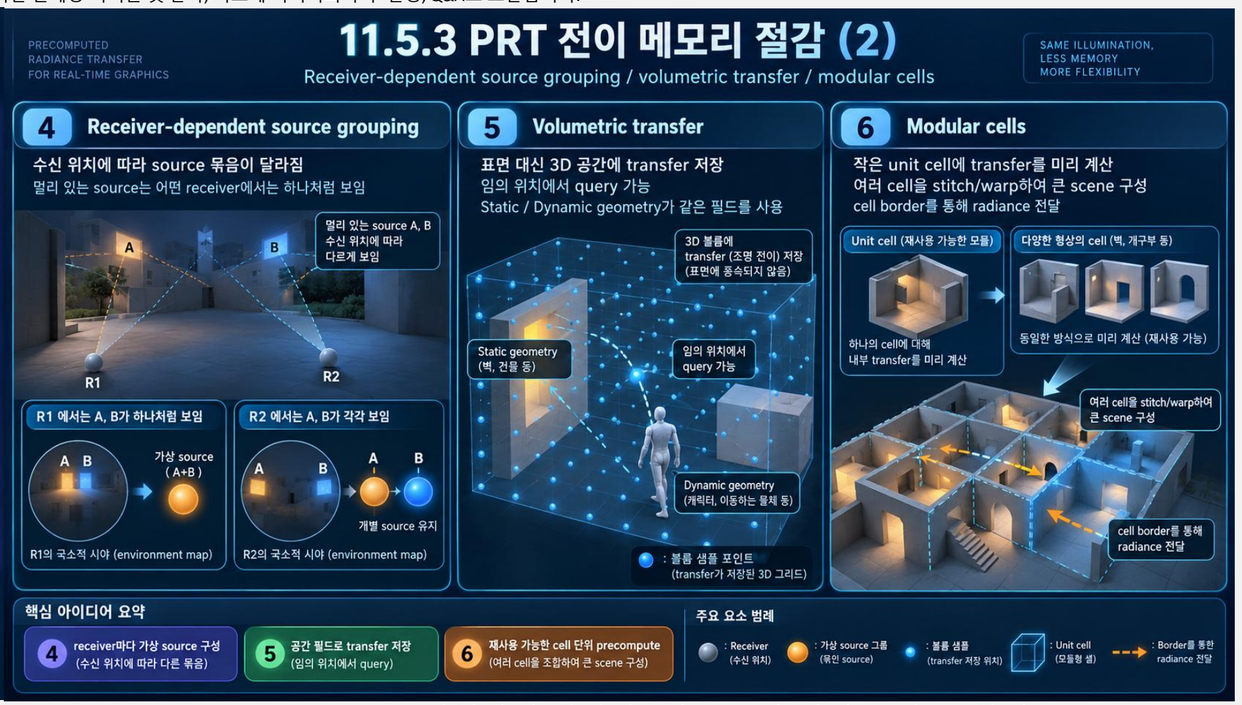
Source를 어떻게 분할하고, 어떻게 빛을 전파할지에 대해서도 다루었지만, Receiver가 어떻게 radiance를 수집할지도 이야기해야 합니다.
source에서 receiver에 전달하는 정보가 scalar하나라면, 다시 normal map을 사용할 수 없는 문제가 생기게 됩니다.

이를 해결하기 위해 receiver가 방향성 기저를 가지게 한다면 11.5.2 (방향성 표면 사전 조명)에서 다룬 것 처럼
노멀 매핑과 함께 사용하거나, 광택이 낮은 스페큘러 반응도 제공할 수 있게 됩니다.

11.5.2 파트의 경우에도 그렇듯이, 이런 방향성 기저 정보를 계산하기 위해서는 많은 양의 메모리와 런타임 연산 자원이 필요해져 이를 어떻게 줄이는지도 다룹니다.

* 세부 내용을 모두 다루기에는 시간 관계상 어려울 것 같아, 빠르게 이미지와 구두 설명, Q&A로 보완합니다.





리얼타임 렌더링 4판 · 11장 전역 조명 · 4차시

오늘의 순서 - 5차시

00 확산 전역 조명 — 11.5

* 7차시는 있어선 안된다....

01 반사 전역 조명 — 11.6

* 입구라도 맛 보는게 목표입니다. 물론 하면서 어떻게 될진 잘 몰루

저장 방법(Storage Methods)

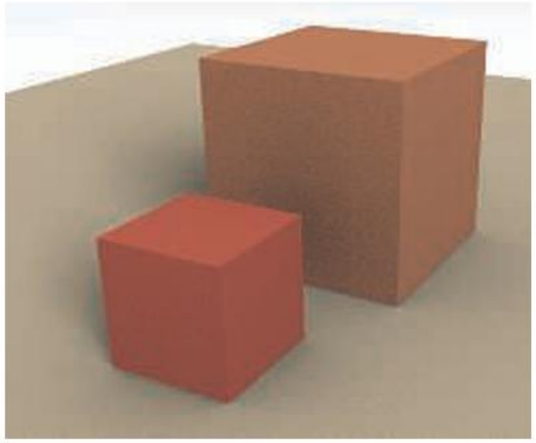
조명을 완전히 계산하는 방법이든, 전달 정보만 사전 계산하고 조명의 변화가 허용되든 그 결과는 어떤 형태로든 저장되어야 하기에 GPU 친화적이여야 합니다.

Light map은 미리 계산된 조명을 저장하는 가장 흔한 방법중 하나입니다. light map은 사전 계산된 정보를 담은 텍스쳐로,
저장된 데이터의 구체적인 종류를 가리키기 위해 irradiance map같은 용어를 쓰기도 하지만, 이런 것들을 통칭할 때 light map이라고 부릅니다.
런타임에는 GPU에 내장된 텍스쳐 메커니즘이 활용 됩니다. (irradiance를 저장할수도, SH 계수를 저장할수도, AHD, Ambient Cube, transfer data를 저장할수도 있음)

값은 보통 이중선형(bilinear)필터링을 거치는데, 일부 표현 방식에서는 이것이 엄밀히 옳지 않을 수도 있습니다.
예시로 AHD 표현을 사용하고자 할 때, 텍스쳐에 저장하여 표현하면 필터링된 방향 (Direction) 성분이 보간 후에 더이상 unit vector가 아니게 되어 재정규화가 필요하며
Ambient와 Highlight도 샘플링 지점에서 직접 계산하는 것과는 다른 색상이 나오게 되지만, 표현이 비선형이더라도 결과는 대체로 받아들일 만하게 보입니다.

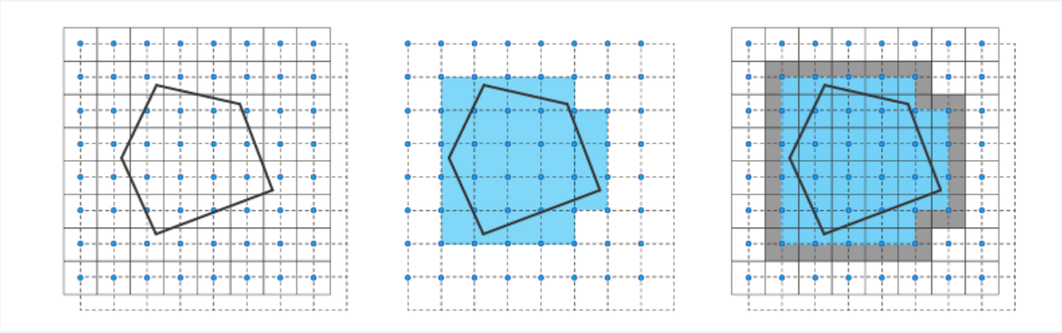
대부분의 경우 라이트 맵은 밉매핑을 사용하지 않습니다. 라이트 맵의 해상도는 일반적인 알베도 맵이나 노말 맵에 비해 작기 때문에 보통은 필요하지 않습니다.
고품질의 프로그램에서도 라이트맵 텍셀 하나가 대략 20x20센티미터 이상의 면적을 덮으며, 그보다 큰 경우도 많습니다. *라이트맵은 비교적 저 해상도여서 밉맵의 필요성이 적고,
*인접한 chart가 겹치지 않도록 여유 영역을 확보하기도 어렵습니다.

* 씬기준 면적입니다. 런타임 화면 기준 아님에 유의





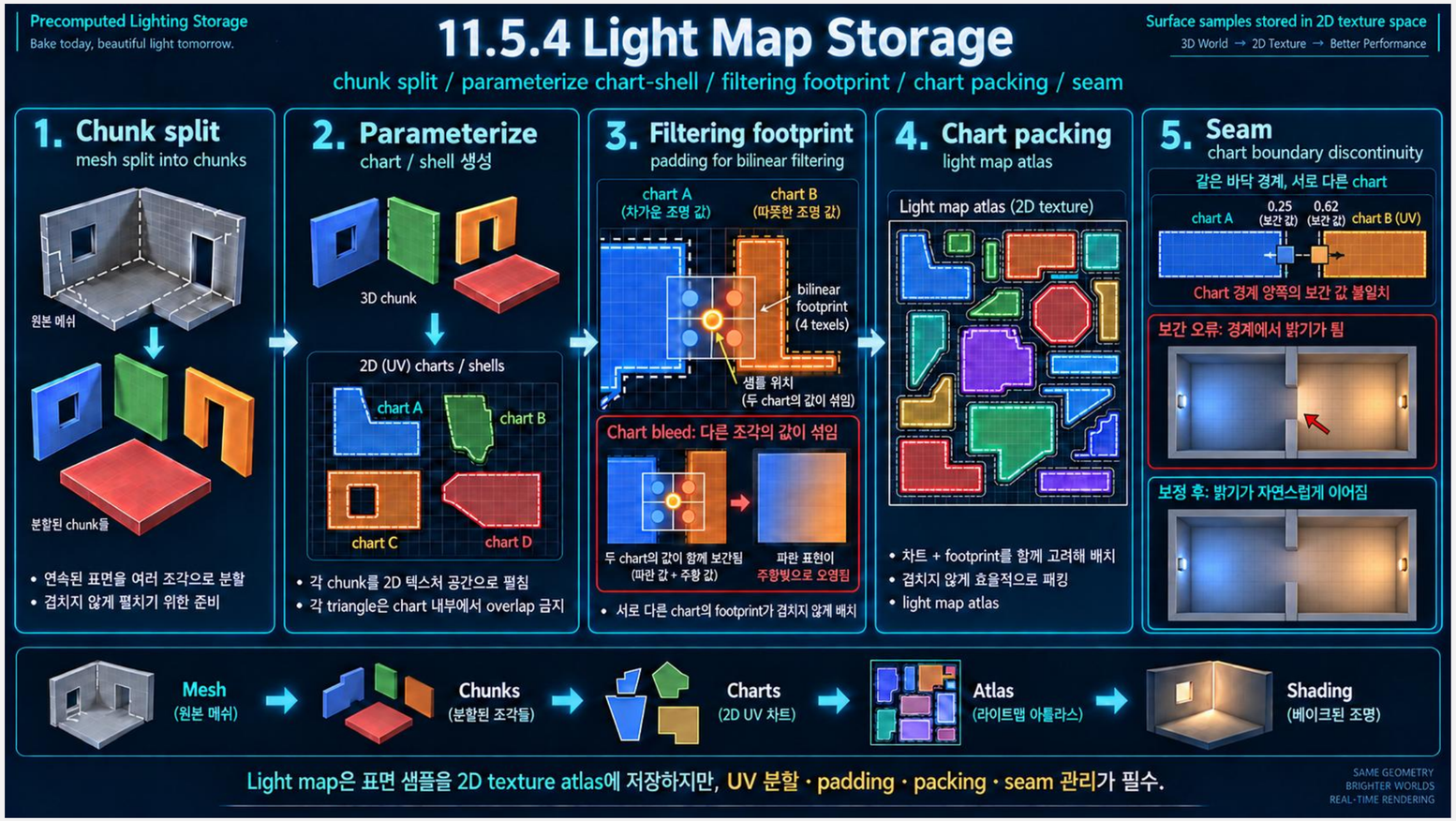
* 미리 구워진 라이트 맵의 예시
* 왼쪽 아래가 바닥면이고 오른쪽에서 2번째, 가장 윗줄이 뒤에 있는 Cube의 정면 방향 Light Map입니다.
* Source Clustering은 법선·확산색이 비슷한 표면을 묶어, 소스와 수신자 사이의 전달 계산을 줄입니다.
* 수신자와의 거리·가시성에 따라 전달이 달라집니다. 가까이 보이거나 차폐가 다르면 세부 구분이 필요합니다.

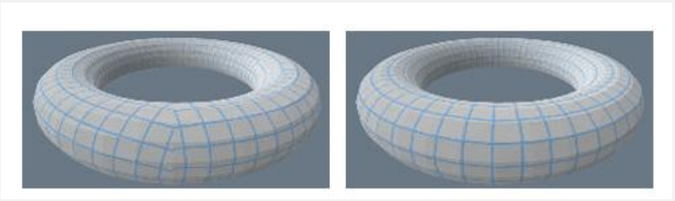


리얼타임 렌더링 4판 · 11장 전역 조명 · 5차시

저장 방법(Storage Methods)

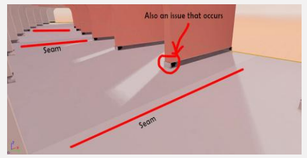
텍스쳐에 조명 정보를 저장하기 위해서는 객체가 고유한 uv 값을 샘플링 해야합니다.
일반 albedo texture는 UV를 겹쳐서 사용할 수 있지만, light map은 직/간접광의 영향으로 surface의 위치마다 고유한 texel영역을 가져야 합니다.
모든 surface의 위치가 고유한 texel영역을 가질 수는 없기 때문에, 최대한 이 Texel영역의 차이에서 발생하는 이슈를 막기 위한 여러가지 고민과 방법론이 존재합니다.

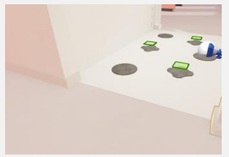




* 이런 형태의 mesh를 여러 조각으로 나누는거라던가
* 이미 존재하는 표면을 어떻게 분할할지는 수많은 휴리스틱이나 사람의 개입이 존재합니다.
* 잘 쪼개지 못하면, 못생긴 결과물이 나올 수 있다고만 정리하고 넘어갑니다.

*https://forums.unrealengine.com/t/light-map-seams/246456
light map seams의 예시 이미지

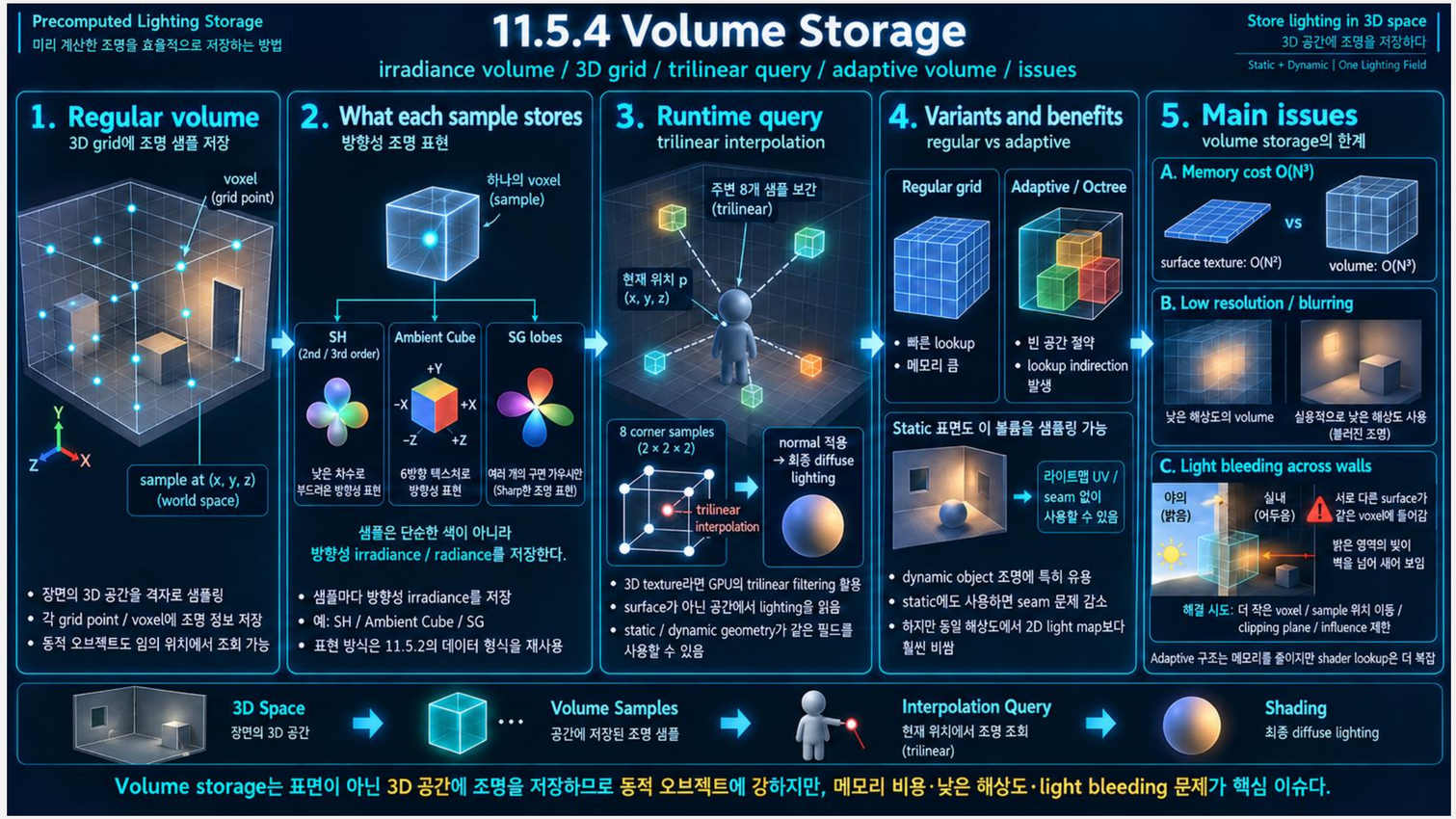




리얼타임 렌더링 4판 · 11장 전역 조명 · 5차시

저장 방법(Storage Methods)

Mesh Vertex에 이를 저장하는 방법도 있지만, 너무 퀄리티가 낮아 생략하고, surface가 아닌 3D volume에 저장하는 방법을 소개합니다.
3D Volume에 매핑할 경우, Dynamic한 물체들에도 효과적으로 미리 구워둔 조명 값을 전달 할 수 있다는 장점이 생깁니다.
마찬가지로 모든 volume을 고유한 gpu 메모리 위에 올릴 수는 없기 때문에, 이를 고려해서 어떻게 저장하고 샘플링하는지도 나열해보았습니다.



* Unreal에서는 Volumetric Light Map이
* Unity에는 Light Probe Proxy Volume (레거시)이 있습니다.

* 동적인 물체에 조명을 전달할 수 있지만, 그것이 그 물체가 동적 GI를 바꾼다는 것은 아닙니다.
* 해당 오브젝트로 인한 색번짐등의 반사광이 지금 계산된 상태는 아님.
* 물론 미리 구운 조명이라는 전제하의 이야기입니다.
* 이후에 해당 저장공간을 런타임에 바꾸는 내용도 다룰것임.

* https://discussions.unity.com/t/how-to-avoid-volumetric-light-bleeding-through-objects/791228
낮은 해상도의 Volume을 사용하거나, 다른 surfac가 같은 voxel에 들어가며 생긴 light bleeding의 예시



리얼타임 렌더링 4판 · 11장 전역 조명 · 5차시

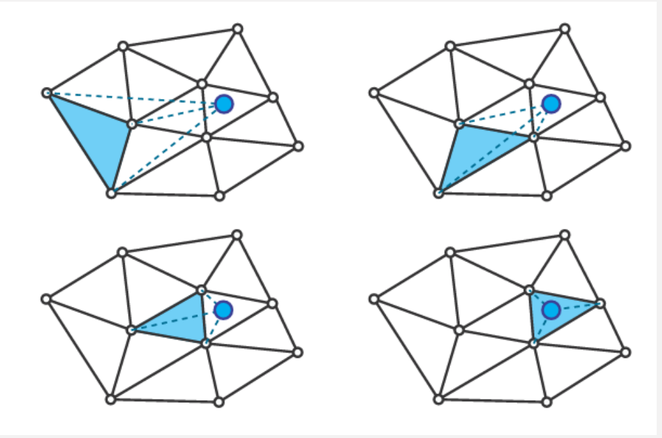
저장 방법(Storage Methods)



순수하게 3D 공간을 Volume Storage에 저장하는 것은 $O(n^3)$의 공간 복잡도를 가지게 됩니다.
런타임에 잘 닿지 않을 위치나, 조명이 천천히 변하는 곳에서 높은 밀도로 저장 할 필요는 없습니다.

공간에 저장하는 또 다른 방법은, 불규칙한 샘플 점들에 조명을 저장하고 이 점들을 들로네 사면체로 연결하는 것입니다.
조명을 조회하기 전에 샘플링 위치가 속한 사면체를 찾는 과정을 거칩니다.

사면체 탐색 과정은 GPU친화적이지 않기 때문에 GPU 가속을 받기 어렵지만, 3D Volume에서 8개의 지점에서
삼선형 (Trilinear) 보간 하는것보다 적은 값(4개 지점)으로 보간 할 수 있게 됩니다.



조명을 미리 계산해 저장할 위치는 수동으로 배치할수도 있고, 자동으로 배치할 수도 있습니다.

이런 위치들은 조명 신호를 샘플링한다는 뜻에서 라이트 프로브 (Light Probe)라고 불립니다.
프로브는 공간상의 조명 샘플이며, 앞서 본 격자형 volume에도 배치할 수 있습니다.

이런 Light Probe를 자동으로 배치하는 방법론이나, Mesh나 씬의 형태에 따른 여러 휴리스틱들도 논의되었습니다.

저장 방법(Storage Methods)

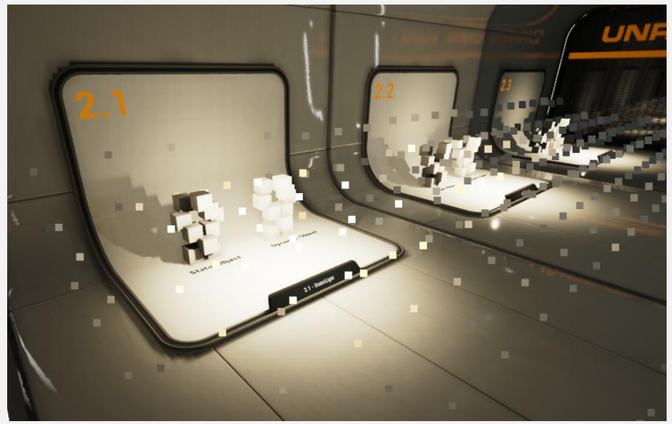
저장 위치는 크게 표면과 공간으로 나눌 수 있으며, 엔진과 게임은 각 방식의 장점을 조합합니다.

표면의 Light Map은 상대적으로 높은 해상도를 제공하지만, Dynamic Object에 적용하기는 어렵습니다.
공간에 저장하면 Dynamic Object도 조명을 조회할 수 있습니다. 정규 격자는 GPU 조회·보간에 유리하지만 메모리 비용이 큽니다.
같은 공간 저장에서도 불규칙한 프로브와 사면체를 쓰면 배치 밀도를 조절하기 좋지만, 사면체를 탐색하는 비용이 생깁니다.
프로브는 조명을 저장하는 샘플 위치를 뜻하며, 격자형 volume과 별개의 저장 분류는 아닙니다.

Static 객체와 Dynamic 객체의 특성에 맞춰 서로 다른 storage를 사용하는 경우,
서로 다른 저장 방식의 차이에서 발생하는 해상도나 필터링 방식으로 인해 조명 색과 밝기가 다르게 보일 수 있습니다.

이런 괴리를 줄이기 위해 서로 다른 Storage에 저장된 정보를 다시 평균화 하는 regularization을 거치기도 합니다.

Bake할 때는 샘플 위치도 유효해야 합니다. 벽·물체 내부나 자기 교차 영역에서 계산한 값은 잘못된 조명을 퍼뜨릴 수 있습니다.
샘플 수를 늘리는 것과 함께, 유효하지 않은 위치의 샘플을 제외하거나 위치를 조정해야 합니다.



* static mesh에는 Light Map을
* dynamic object에는 공간의 프로브 조명을 사용하는 등…
* 아예 엔진에서 적절한 지점을 제공하는 시스템을 제공하기도

동적 확산 전역 조명(Dynamic Diffuse GI)

사전 계산 결과를 갱신해야 하는 이유

지금까지는 조명이나 전달 정보를 미리 계산하고, 이를 표면이나 공간에 저장하여 런타임에 참조하는 방법을 다루었습니다.
하지만 저장된 조명은 계산 당시의 장면을 반영하므로, 광원이나 지오메트리가 바뀌면 다시 갱신할 필요가 생깁니다.

광원을 움직이면 반사광의 출발점이 달라지고, 문을 열거나 벽을 없애면 빛이 전달되는 경로도 달라지게 됩니다.
움직이는 물체가 기존 GI를 받을 수 있다는 것과, 그 물체가 주변에 만드는 반사광과 차폐를 계산하는 것은 구분해야 합니다.

Bake에는 긴 시간이 필요하며, 런타임에 변하거나 생성되는 장면의 모든 상태를 미리 계산하기도 어렵습니다.
이를 해결하기 위해 사전 계산 없이 처리하거나, 필요한 준비를 매 프레임 수행할 수 있을 만큼 빠르게 계산하는 기법들이 제안되었습니다.
* PRT는 일부 광원 변화를 허용하지만, 미리 계산한 전달 관계가 유효한 범위에서 사용할 수 있습니다.



리얼타임 렌더링 4판 · 11장 전역 조명 · 5차시

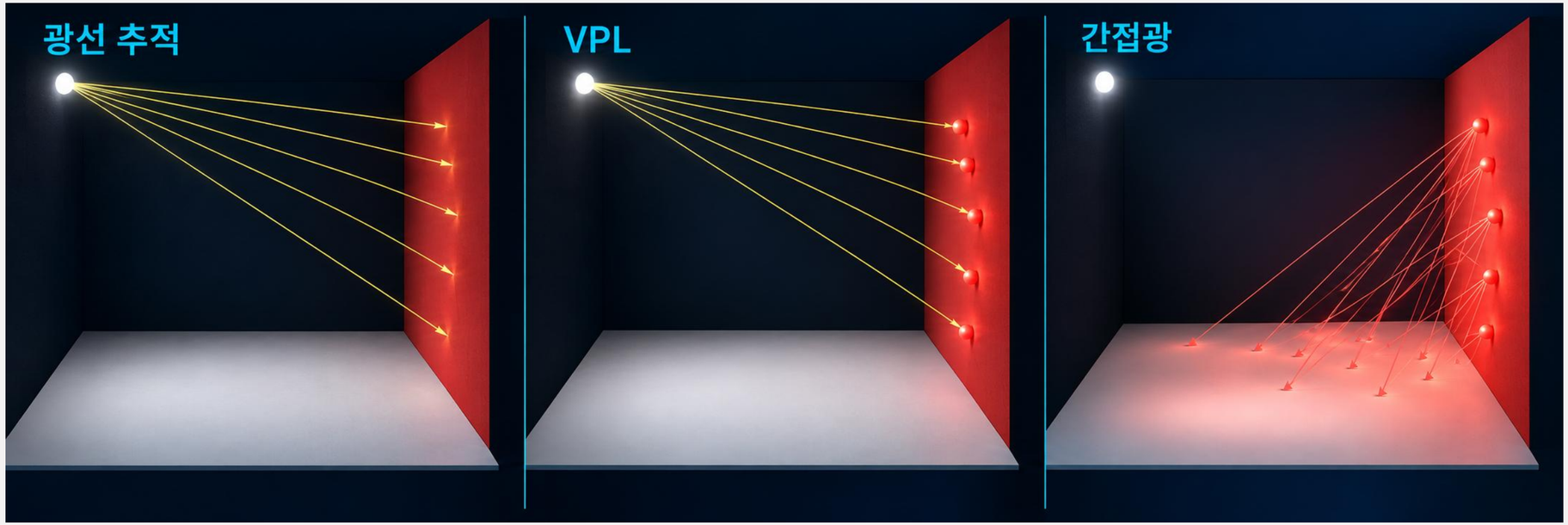
동적 확산 전역 조명(Dynamic Diffuse GI)

Instant Radiosity와 Virtual Point Lights

빛을 받은 표면은 다시 다른 표면을 비추는 간접광의 출발점이 될 수 있습니다.
Instant Radiosity는 이런 반사광을 여러 개의 작은 가상 광원으로 근사하여, 변화하는 장면의 간접 조명을 계산합니다.

먼저 실제 광원에서 광선을 쏘고, 표면에 도달한 위치마다 해당 표면의 반사광을 대표하는 광원을 배치합니다.
이런 광원을 Virtual Point Light (VPL)라고 부르며, 위치와 표면 방향, 반사광의 색과 세기를 가지게 됩니다.

다른 표면에서는 VPL의 조명 기여분을 합산하여 간접광을 구합니다. 흰 빛을 받은 빨간 벽은 주변에 붉은 반사광을 전달하게 됩니다.
여기서는 직접광을 받은 표면에서 나온 한 번의 간접 반사를 중심으로 살펴보겠습니다.
* 이름은 Radiosity지만, 앞서 다룬 패치 사이의 연립방정식을 푸는 고전적인 Radiosity와는 다른 접근입니다.



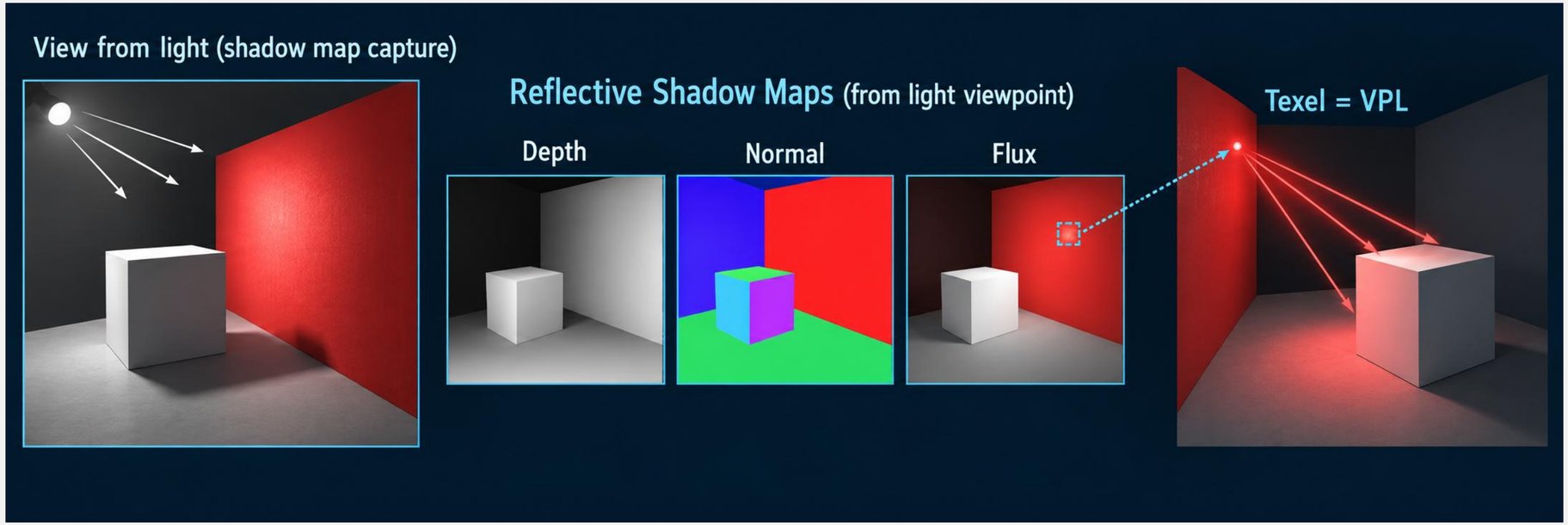
동적 확산 전역 조명(Dynamic Diffuse GI)

Reflective Shadow Maps (RSM)

VPL을 만들기 위해서는 직접광을 받은 표면이 어디에 있고, 어떤 반사광을 내보내는지 알아야 합니다.
Reflective Shadow Map (RSM)은 광원 시점에서 장면을 렌더링하여, 이 정보를 텍스쳐에 저장하는 방법입니다.

일반 Shadow Map이 깊이를 저장한다면, RSM은 표면 위치를 얻기 위한 깊이와 normal, 반사광량(flux) 등의 정보를 함께 저장합니다.
반사광의 색과 세기에는 표면이 받은 직접광과 재질색의 영향이 반영됩니다.

최종 shading에서는 RSM의 texel을 VPL처럼 취급하여, 해당 표면에서 나온 한 번의 간접 반사를 계산합니다.
하지만 RSM의 모든 texel을 사용하면 비용이 커지므로, 관련 있는 일부 샘플만 선택하여 조명에 사용하게 됩니다.
* 표면 위치는 depth로 복원하거나 별도로 저장할 수 있습니다. 아래 그림은 각 정보의 역할을 단순화한 개념도입니다.



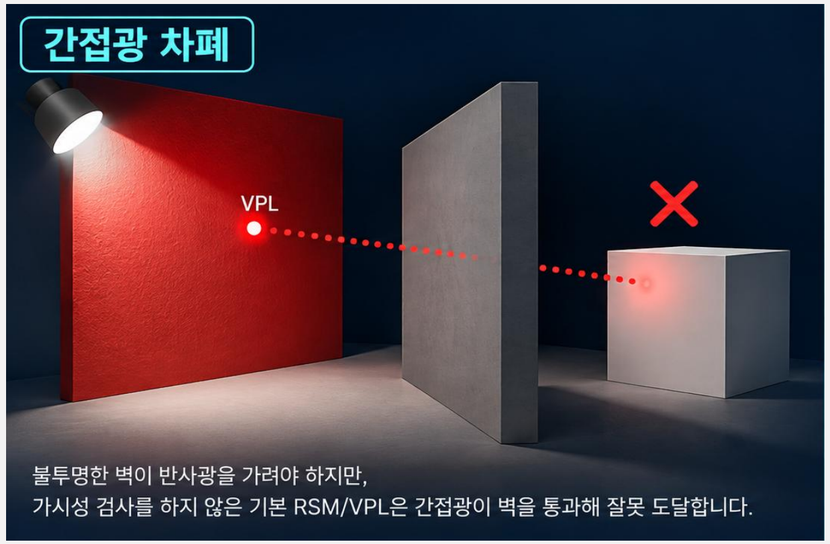
동적 확산 전역 조명(Dynamic Diffuse GI)

간접광 차폐

기본 RSM은 직접광을 받은 표면에서 반사광을 생성하지만, VPL과 수신 표면 사이의 차폐는 별도로 검사하지 않습니다.
광원 시점의 Shadow Map은 각 방향에서 가장 가까운 표면의 깊이를 저장하여, 광원에서 표면까지 빛이 도달하는지를 판단합니다.
하지만 반사광은 VPL 위치에서 다시 출발하므로, 원래 광원의 깊이 정보만으로 그 이후 경로의 차폐까지 판단할 수는 없습니다.

움직이는 물체를 Shadow Map에 다시 그리면, 그 물체가 원래 광원의 직접광을 가리는 상황은 반영할 수 있습니다.
그러나 그 물체가 VPL과 수신 표면 사이를 가리는 상황은 별도의 가시성 검사가 필요합니다. 이를 생략하면 빛이 벽을 통과할 수 있습니다.

이를 정확히 처리하려면 VPL마다 해당 위치에서 본 Shadow Map을 만들거나, VPL과 표면 사이에 광선을 추적해야 합니다.
VPL이 많아지면 검사해야 할 시점과 경로도 늘어나며, 물체가 움직일 때 해당 차폐 정보도 갱신해야 하므로 비용이 커집니다.



광원 → 빨간 벽
빨간 벽의 VPL → 흰 물체

위 두 경로는 출발점과 방향이 다르므로,
원래 광원의 Shadow Map 하나로 함께 검사할 수 없습니다.

책에서는 VPL의 Shadow Map을 여러 프레임에 나누어
생성하거나, 단순화된 지오메트리와 낮은 해상도의
Shadow Map으로 차폐를 근사하는 방법을 소개합니다.
정확도와 갱신 비용 사이의 절충이 필요합니다.

동적 확산 전역 조명(Dynamic Diffuse GI)

시간적 필터링 — Uncharted 4의 RSM

RSM의 많은 texel을 모두 VPL로 계산하면 비용이 커지므로, 실제 shading에서는 일부 샘플만 선택하여 간접광을 구합니다.
광원이 움직여 RSM이 갱신되거나 선택한 샘플이 달라지면, 적은 샘플로 구한 결과는 노이즈나 밝기의 깜빡임으로 드러날 수 있습니다.

Uncharted 4에서는 한 프레임에 픽셀당 16개의 VPL을 샘플링하며, 다음 프레임에서는 다른 16개의 VPL을 사용합니다.
즉 장면 전체에 VPL을 16개만 만드는 것이 아니라, 각 픽셀이 간접광을 계산할 때 참조하는 수를 16개로 제한하는 것입니다.
현재 결과를 과거 프레임의 결과와 함께 시간적으로 필터링하면, 더 많은 VPL의 기여를 반영하면서 안정적인 간접광을 얻을 수 있습니다.

간접광 없음



간접광 있음



그림 11.32 · 우측 확대: 위 필터링 없음 / 아래 필터링 있음
UNCHARTED 4 ©/™ 2016 SIE · Naughty Dog

한 픽셀에서 사용하는 샘플

프레임 1 16개 VPL · 집합 A
프레임 2 다른 16개 VPL · 집합 B
프레임 3 다른 16개 VPL · 집합 C

현재 샘플로 구한 간접광
+ 과거 프레임의 결과
↓
시간적 필터링
↓
16개보다 많은 VPL의 기여를 반영

빛 전파 볼륨(Light Propagation Volumes)

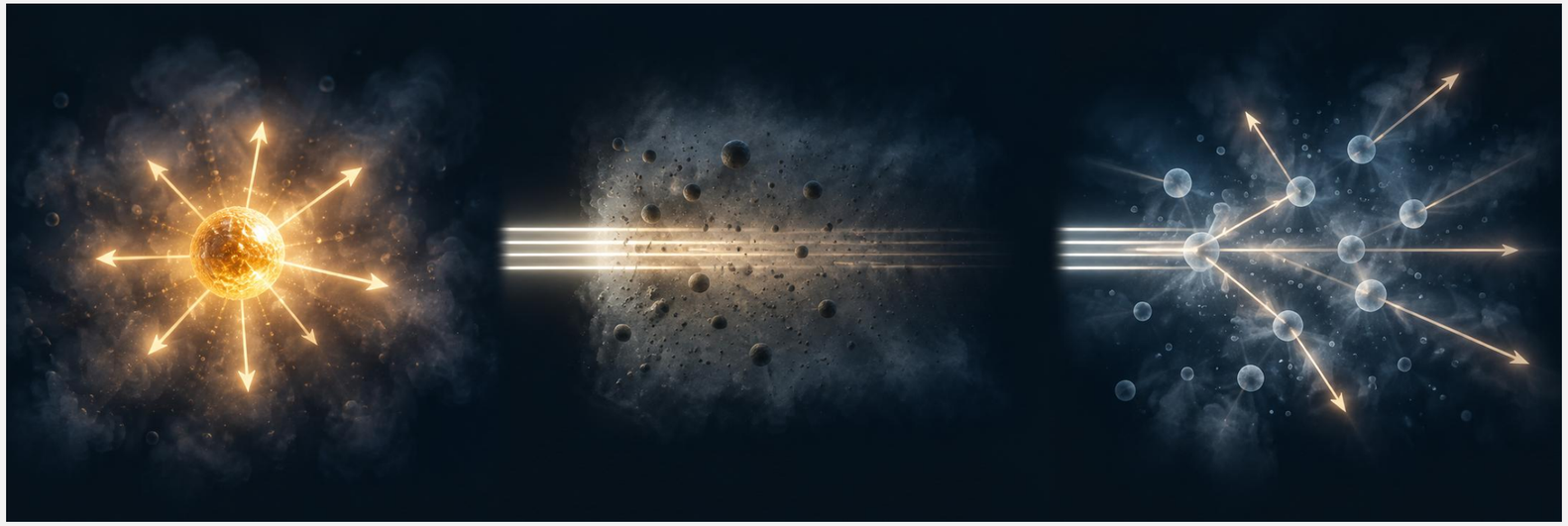
Radiative Transfer의 개념

Radiative Transfer는 빛이 매질을 통과하면서 어떻게 이동하고 변화하는지를 설명하는 이론입니다.
빛이 발생하는 emission, 에너지가 줄어드는 absorption, 진행 방향이 달라지는 scattering 등을 함께 고려합니다.

예를 들어 안개 속에서는 빛이 통과하기도 하지만, 일부는 흡수되거나 다른 방향으로 산란됩니다.
따라서 공간의 각 위치에서 빛의 세기뿐 아니라, 어느 방향으로 이동하는지도 알아야 합니다.

이를 정밀하게 계산하는 것은 비싸지만, 작은 영역 사이에서 빛을 전달하는 방식은 실시간 렌더링에도 활용할 수 있습니다.
LPV는 이러한 접근을 간접광 계산에 적용한 방법이며, 기본 GI에서는 표면에서 나온 반사광을 공간에 주입합니다.

Emission · 빛의 발생 **Absorption · 에너지 감소** **Scattering · 방향 변화**



리얼타임 렌더링 4판 · 11장 전역 조명 · 5차시

빛 전파 볼륨(Light Propagation Volumes)

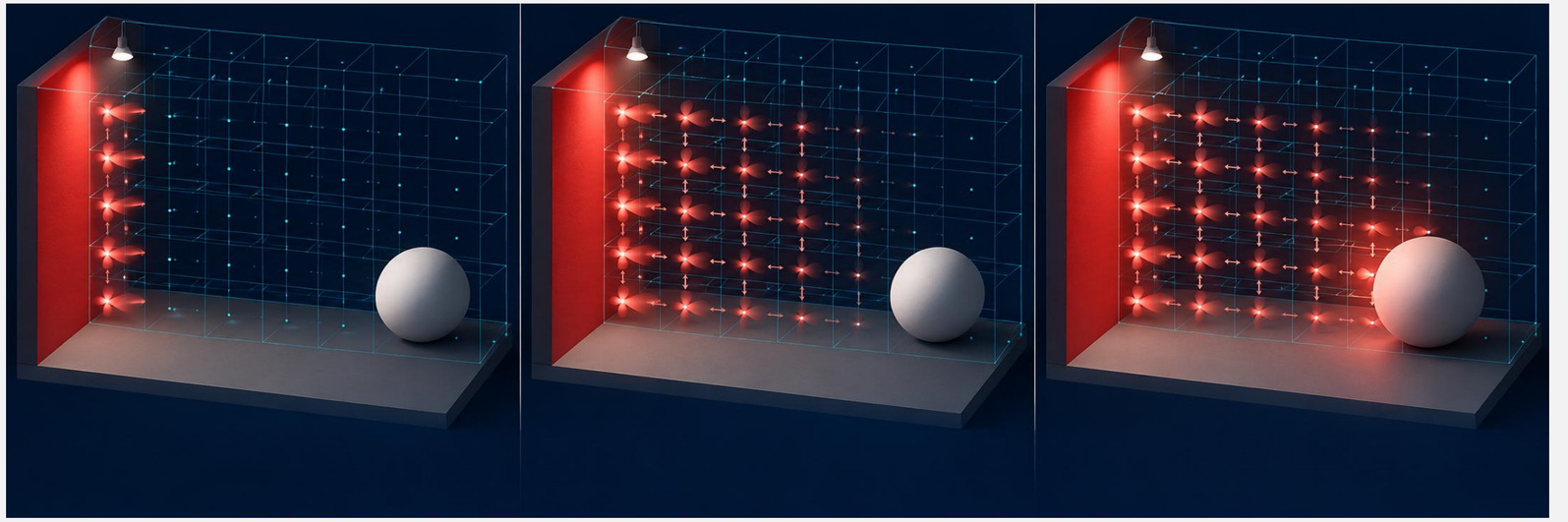
3D cell에 저장하고 전파하는 간접광

LPV는 장면의 3D 공간을 규칙적인 cell로 분할하고, 각 cell에 그 공간을 통과하는 빛의 방향 분포를 저장합니다.
단순 RGB 값 대신 SH로 방향 정보를 압축하며, 책의 기본 방식에서는 second-order SH를 사용합니다.

먼저 RSM에서 직접광을 받은 표면의 위치와 반사광 정보를 얻고, 해당 표면이 포함된 cell에 빛을 주입합니다.
빨간 벽의 반사광은 빨간색을 띠며, 표면의 normal을 기준으로 바깥쪽을 향하는 분포를 가집니다.

이후 이웃 cell 사이에서 빛을 전달하여 volume을 갱신하고, 물체를 shading할 때 해당 위치의 조명을 조회합니다.
이전의 baked volume이 계산된 조명을 저장하고 조회했다면, LPV의 volume은 전파 계산에도 사용됩니다.

표면의 반사광 주입 **이웃 cell 사이의 전파** **물체 위치에서 조회**



리얼타임 렌더링 4판 · 11장 전역 조명 · 5차시

빛 전파 볼륨(Light Propagation Volumes)

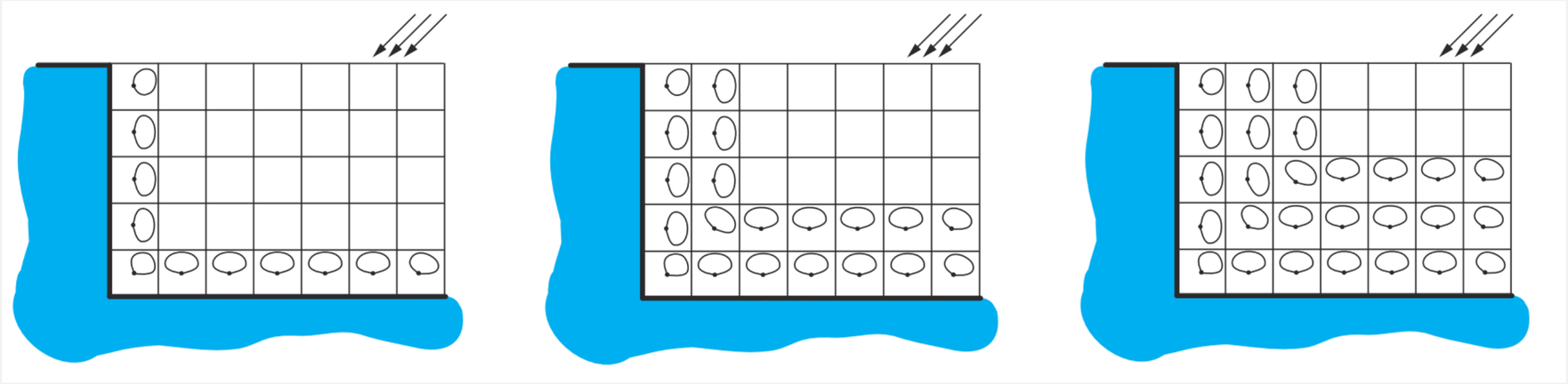
반복 전파와 프레임 갱신

한 iteration에서는 모든 cell이 이전 단계의 이웃 값을 읽고, 들어오는 방향에 따른 빛의 기여를 계산합니다.
기본 방식은 상하·좌우·앞뒤의 6개 이웃을 참조하며, 한 번에 빛이 전달되는 거리는 한 cell 정도입니다.
같은 이전 단계의 값을 읽으므로 cell 계산을 병렬로 처리할 수 있습니다. 각 단계의 기여는 별도의 결과 volume에 누적합니다.
더 먼 위치까지 전달하려면 여러 번 반복해야 하며, 이 반복 횟수가 표면에서 반사된 횟수나 경과 프레임 수를 뜻하지는 않습니다.

한 프레임의 갱신: 초기화 → RSM 갱신 → 반사광 주입 → K회 전파·누적 → shading

전파 비용은 대략 cell 수 × 반복 횟수에 비례합니다. 한 축에 N개의 cell이 있다면 $O(KN^3)$로 볼 수 있습니다.
예를 들어 32^3 cell에서 8회 반복하면, 6개 이웃 기준으로 약 157만 개의 cell–이웃 조합을 처리합니다.

반사광 주입 직후 **이웃 cell로 전파** **다음 이웃으로 전파**



* 그림 11.33의 타원은 방향 분포를 나타냅니다. 157만은 조합 수의 예시이며, 각 조합에는 SH 연산 등이 추가됩니다.

빛 전파 볼륨(Light Propagation Volumes)

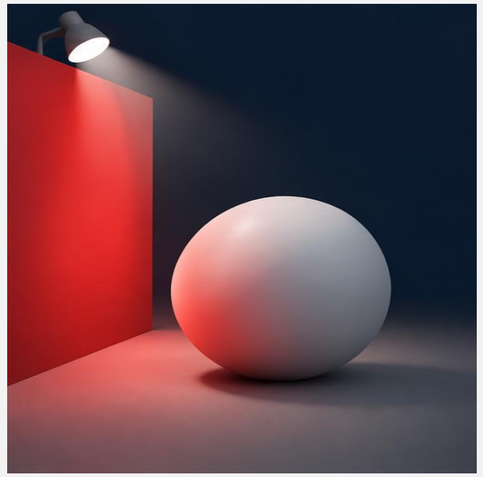
볼륨 방식의 장점과 한계

LPV는 공간에 조명을 저장하므로 움직이는 물체도 자신의 위치에서 간접광을 조회할 수 있습니다.
RSM과 volume을 다시 계산하여 장면 변화에 대응하며, 전파 결과를 여러 shading 위치에서 재사용합니다.
규칙적인 격자와 이웃 참조는 병렬 처리에 유리하며, 부드러운 간접광과 색 번짐을 낮은 해상도로 표현하는 데 적합합니다.

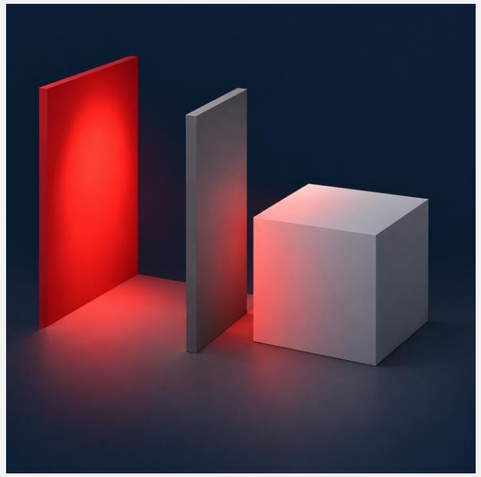
반면 이전 볼륨 저장에서 다룬 해상도와 light bleeding 문제도 이어집니다. 얇은 벽을 cell 단위로 구분하지 못하면 빛이 새어 나올 수 있습니다.
낮은 차수의 SH로 방향 분포를 거듭 근사하면 빛이 넓게 퍼지고, 격자 방향의 전파와 겹쳐 줄무늬 같은 공간적 artifact도 발생할 수 있습니다.
방향 분포를 BRDF에 활용할 수 있지만, 낮은 차수의 SH로는 날카로운 glossy 반사를 표현하기 어렵습니다

* 해상도를 높이면 메모리와 전파 비용이 함께 커집니다. 공간적 필터링은 일부 artifact를 완화하지만 디테일한 표현을 부드럽게 만들어 버립니다.

부드러운 간접광과 색 번짐

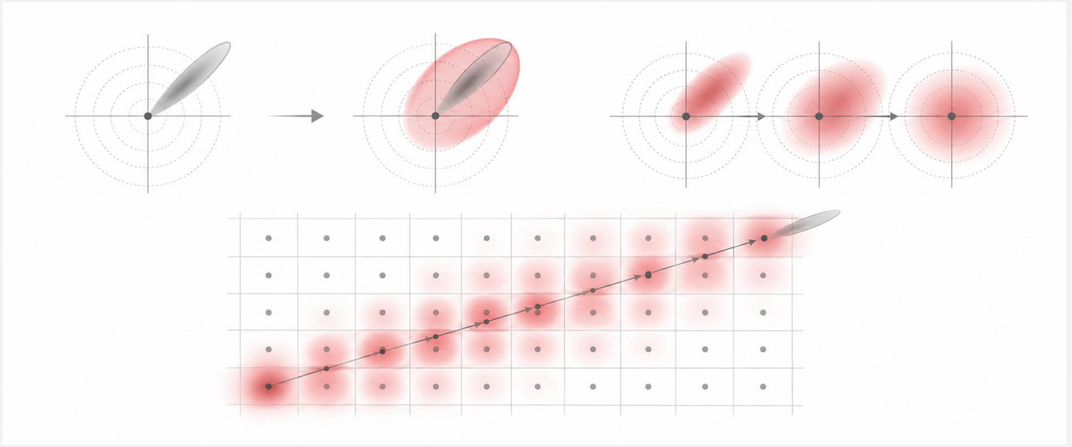


얇은 벽의 light bleeding



저차 SH 근사와 반복 전파

원래 방향 분포　　저차 SH로 근사　　이웃 cell에서 재전파



회색: 원래 방향 분포　빨강: SH 근사와 격자 전파로 퍼진 분포 (개념도)

리얼타임 렌더링 4판 · 11장 전역 조명 · 5차시

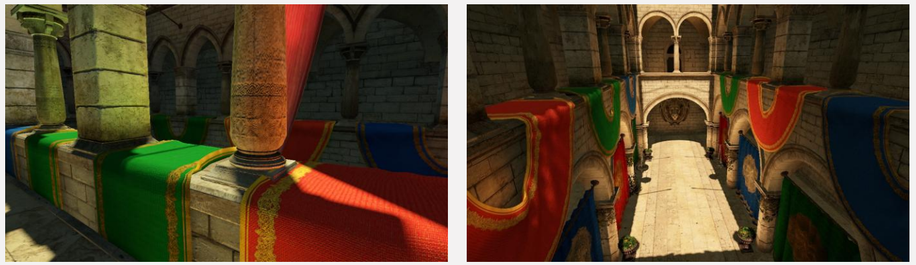
빛 전파 볼륨(Light Propagation Volumes)

Cascaded LPV와 간접광 차폐 개선

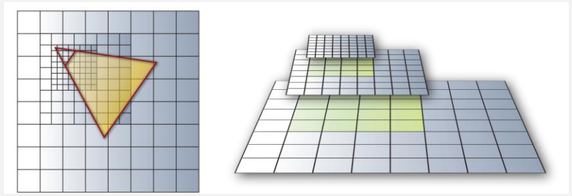
Cascaded LPV는 카메라 주변을 작은 cell로, 먼 영역을 큰 cell로 처리하는 volume들을 중첩하고, 카메라 이동에 맞춰 갱신합니다.
각 cascade에 조명을 주입하고 독립적으로 전파한 뒤, shading 위치를 포함하는 가장 세밀한 volume을 조회합니다.
큰 cell에서는 같은 반복 횟수로 더 멀리 전파할 수 있으며, 넓은 범위 전체를 작은 cell로 채우는 것보다 메모리를 절약합니다.
* 한 변이 L, 2L, 4L인 volume을 각각 N^3 cell로 구성하면 총 $3N^3$입니다. 4L 전체를 가장 작은 cell로 채우면 $64N^3$가 필요합니다.

간접광 차폐를 위해서는 빛을 가로막는 지오메트리 정보도 공간에 저장하고, 전파 과정에서 전달되는 빛을 줄입니다.
개선된 방법은 RSM의 깊이와 카메라의 depth buffer를 사용하지만, 두 시점에서 보이지 않는 표면은 빠질 수 있습니다.
미리 voxelization한 지오메트리로 이를 보완할 수 있습니다. 다만 cell 단위의 근사이며, 움직이는 지오메트리를 반영하려면 갱신도 필요합니다.

Crytek Sponza · 3단계 LPV 적용



카메라 주변은 세밀하게, 먼 영역은 큰 cell로 처리



Geometry Volume으로 간접광 차폐



차폐 적용 전



GV로 차폐 근사

같은 장면에서 지오메트리 정보로 간접광을 차폐하면,
물체 뒤와 주변으로 전달되는 빛이 줄어드는 모습을 볼 수 있습니다.

Kaplanyan & Dachsbacher (2010) · 논문 그림 1, 4, 7
https://cg.ivd.kit.edu/publications/p2010/CLPVFRII_Kaplanyan_2010/CLPVFRII_Kaplanyan_2010.pdf

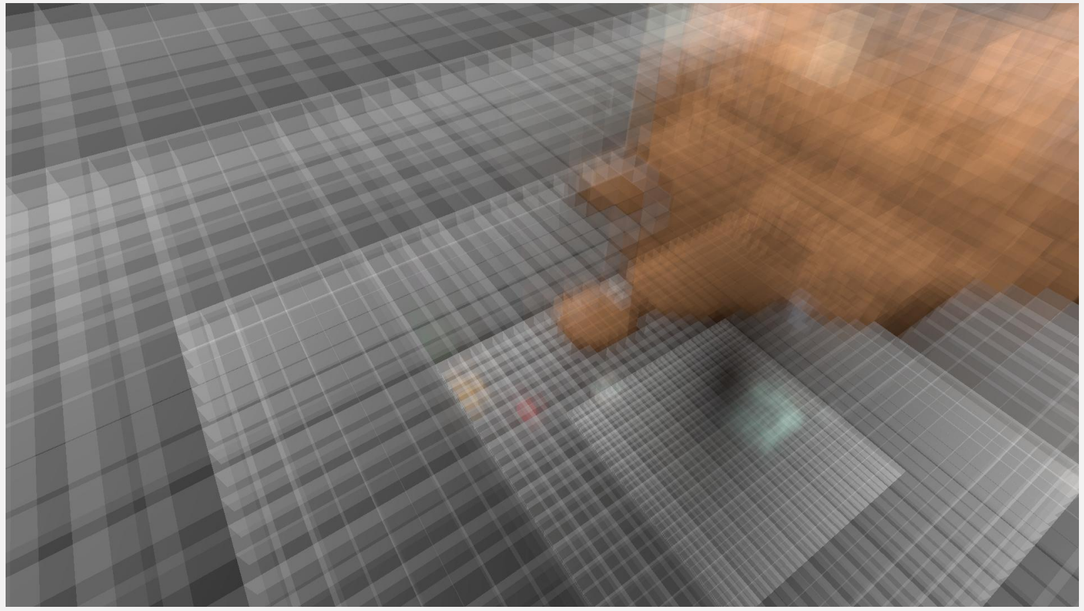
복셀 기반 방법(Voxel-Based Methods)

Voxel에 지오메트리와 조명 저장하기

Voxel-Based Methods는 장면을 작은 3D 영역인 voxel로 표현하고, 표면에서 반사된 빛도 함께 저장하여 사용합니다.
각 voxel에는 지오메트리가 차지하는 비율과 재질색, 반사광 등의 정보를 저장합니다.

먼저 RSM 등을 이용하여 직접광을 받은 표면의 반사광을 세밀한 voxel에 주입합니다.
반사광은 방향에 따라 다르므로, 책에서 소개하는 방식에서는 6개의 주요 방향으로 나누어 저장합니다.

세밀한 voxel의 정보를 필터링하여, 더 넓은 영역을 대표하는 값도 준비합니다.
2D texture의 mipmap처럼 작은 영역은 세밀한 수준에서, 큰 영역은 거친 수준에서 조회할 수 있습니다.



The Tomorrow Children · Q-Games, CEDEC 2014

그림은 실제 게임의 voxel cascade 시각화입니다.
카메라 주변은 작은 voxel로 표현하고,
먼 영역은 큰 voxel로 표현하여
넓은 공간을 제한된 메모리에 저장합니다.

Voxel 정보를 저장하는 구조는 구현마다 다를 수 있습니다.
책에서는 Sparse Voxel Octree를 소개하며,
그림의 게임에서는 캐스케이드 3D 텍스처를 사용합니다.

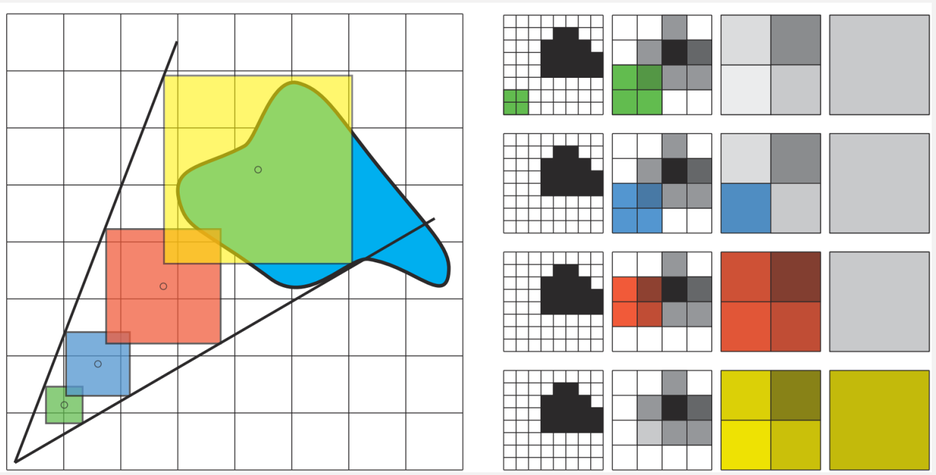
복셀 기반 방법(Voxel-Based Methods)

Voxel Cone Tracing으로 간접광 구하기

표면에 들어오는 간접광을 구하려면 주변의 여러 방향에서 빛이 얼마나 도달하는지 알아야 합니다.
많은 ray를 추적하는 대신, 일정한 방향 범위를 하나의 cone으로 묶어 근사하는 방법이 Voxel Cone Tracing입니다.

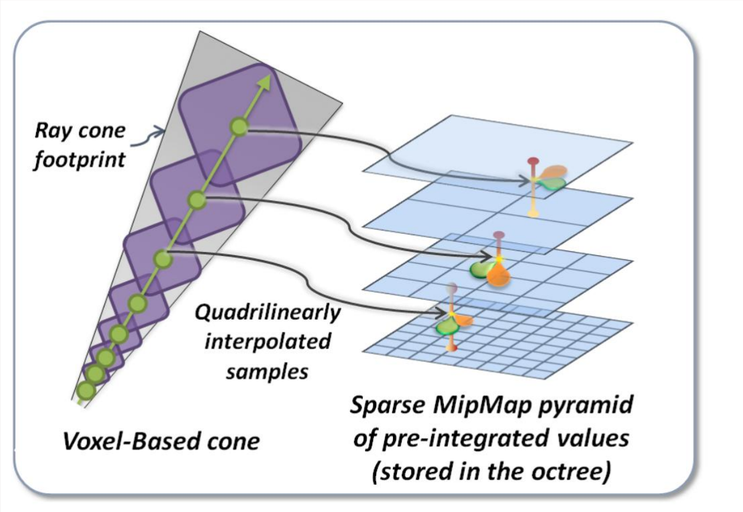
Cone의 축을 따라 여러 위치를 조회하며, 각 위치에서 cone의 단면 크기에 맞는 해상도를 선택합니다.
가까운 위치는 세밀한 voxel을, 멀어져 cone이 넓어지면 여러 voxel을 모아둔 거친 수준을 사용합니다.
각 조회에서는 시작점을 향하는 반사광과 지오메트리 점유율을 얻고, 앞쪽 차폐에 따라 뒤쪽 빛의 기여를 줄이며 누적합니다.
Diffuse GI는 표면의 반구 방향으로 여러 cone을 사용하고, 표면 방향에 따른 가중치를 주어 결과를 합산합니다.
Cone을 많이 사용할수록 품질과 비용이 함께 높아집니다. Cone 내부의 모든 지오메트리를 정확히 검사하는 방식은 아닙니다.

그림 11.34 · 조회 범위에 따른 해상도 선택



* Cone의 단면이 커질수록 넓은 영역을 대표하는 값을 사용하여 근사합니다.

Crassin et al. (2011) · 계층별 보간 조회



복셀 기반 방법(Voxel-Based Methods)

조회 비용과 실제 구현에서의 개선

Voxel Cone Tracing은 지오메트리와 반사광을 함께 조회하므로, 간접광 전달과 차폐를 하나의 공간 표현에서 처리할 수 있습니다.
다만 넓은 영역의 정보를 묶는 근사이므로, 낮은 해상도에서는 빛이 새거나 차폐가 부정확해지는 문제가 남습니다.

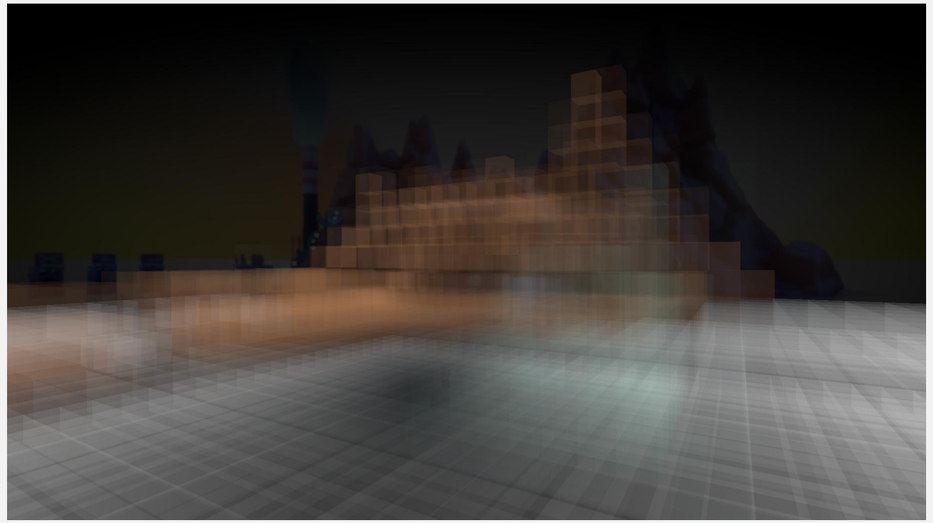
Sparse Voxel Octree는 저장 공간을 줄이는 데 유리하지만, 원하는 위치까지 노드를 차례로 따라가는 조회 비용이 큽니다.
현재 노드를 읽어야 다음 노드의 주소를 알 수 있으므로, 하나의 탐색 경로에서는 메모리 조회가 연속적으로 이어집니다.
캐스케이드 3D 텍스처를 사용하면 여러 해상도를 유지하면서, 노드를 따라가는 탐색을 일반적인 texture 조회로 바꿀 수 있습니다.
그림은 The Tomorrow Children의 게임 화면과 voxel cascade 디버그 화면입니다. 카메라가 이동하면 세밀하게 표현할 영역도 바뀝니다.
필요한 지오메트리를 그때그때 voxelization하며, 조회 성능과 해상도뿐 아니라 데이터를 갱신하는 비용도 함께 고려해야 합니다.

The Tomorrow Children · 게임 화면



Q-Games, CEDEC 2014 · 15, 18페이지

Voxel cascade · 디버그 화면



리얼타임 렌더링 4판 · 11장 전역 조명 · 5차시

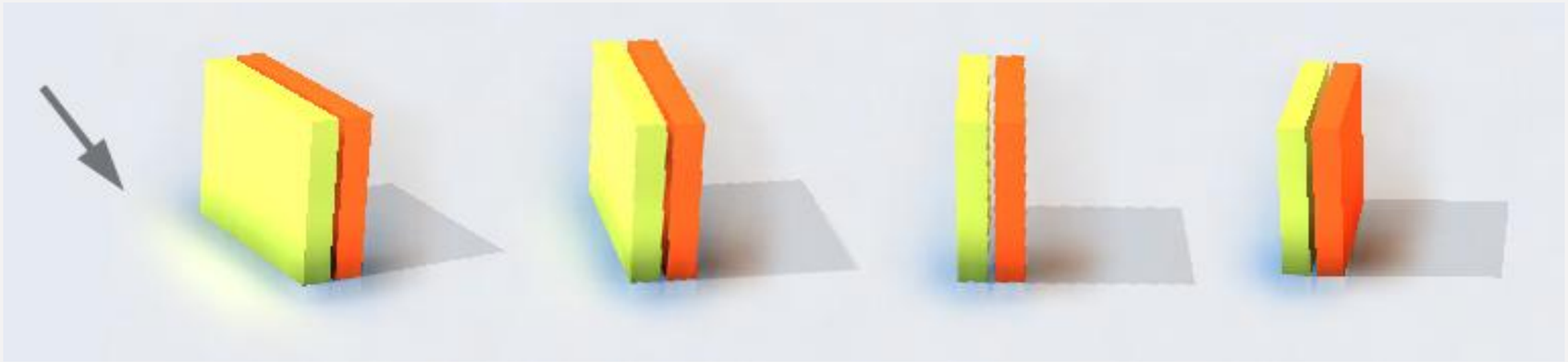
화면 공간 방법(Screen-Space Methods)

화면 정보로 근사하는 간접광

Screen-Space Methods는 현재 화면에 보이는 표면의 정보를 이용하여 간접광을 근사합니다.
SSAO에서 주변의 차폐를 추정했다면, 여기서는 표면에서 반사된 빛도 이용하여 color bleeding 등의 효과를 표현합니다.

하지만 화면 밖이나 다른 물체에 가려진 표면의 정보는 사용할 수 없습니다.
특히 강한 직접광을 받는 넓은 표면은 간접광에 크게 기여하지만, 표면 전체를 화면에 담기는 어렵습니다.
따라서 장면과 조명이 그대로여도 카메라 구도에 따라 계산에 사용하는 정보가 달라져, 간접광의 양과 색이 흔들릴 수 있습니다.
SSAO에서도 정보 부족에 따른 오류가 발생하지만, 간접광에서는 주변을 밝히는 빛의 기여가 변하므로 문제가 더 두드러집니다.
그림에서는 노란 표면이 화면에서 차지하는 면적이 줄어들면서, 바닥으로 전달되는 노란 간접광도 약해집니다.

화면에 보이는 반사면과 간접광 · Ritschel (2009), 그림 4.4



이러한 한계 때문에 다른 GI 방법과 결합하여, 기본 해상도로 표현하기 어려운 근거리의 세부 효과를 보완하는 데 사용합니다.
Quantum Break에서는 큰 규모의 GI를 irradiance volume으로 처리하고, 짧은 거리의 반사광을 screen-space 방식으로 보완합니다.

* Screen-Space 처리의 구체적인 작동 원리는 이후 Specular GI의 Screen-Space Reflection에서 다루며, 여기서는 활용과 한계만 설명합니다.

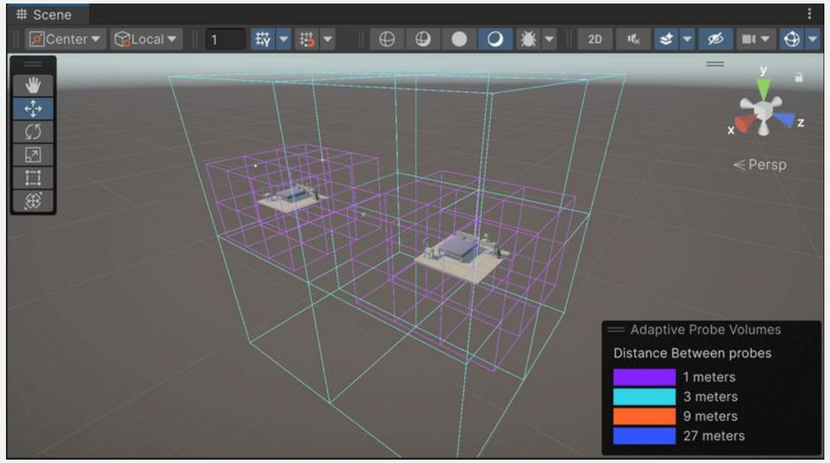
실제 엔진의 확산 GI(Diffuse GI in Engines)

Lightmap과 공간에 저장하는 사전 계산 GI

Unity의 Lightmapper와 Unreal의 Lightmass는 간접광을 미리 계산하여, 표면의 Lightmap이나 공간의 조명 샘플에 저장합니다.
Unity APV는 주변 기하 구조에 따라 probe를 자동으로 배치하며, 복잡한 곳은 촘촘하게, 빈 공간은 성기게 구성합니다.
렌더링할 때에는 주변 probe의 조명을 픽셀별로 보간합니다. 그림은 해상도별 brick과 실제 장면에 배치한 probe입니다.
Unreal의 Volumetric Lightmap도 공간의 조명 샘플을 사용하며, 기존 Indirect Lighting Cache보다 세밀하게 조명을 보간합니다.

다만 움직이는 물체가 저장된 조명을 받는 것과, 그 물체의 움직임이 주변의 간접광까지 바꾸는 것은 다른 문제입니다.

APV · 해상도별 brick 배치



Oasis · 실제 장면의 APV probe



주요 대상: 사전 계산 GI는 PC·콘솔·모바일에서 사용하며, APV는 URP와 HDRP에서 제공합니다.
Unreal VLM의 모바일 경로는 CPU에서 객체의 bounds 중심을 보간하므로, PC·콘솔의 픽셀별 보간과 구분합니다.

* Unity 6.6에는 Lightmap·Light Probe·APV 데이터를 생성하는 Compute Light Baker가 추가되었습니다.

Unity APV 공식 설명 / APV · Unity GDC 2023 영상

실제 엔진의 확산 GI(Diffuse GI in Engines)

전달 관계를 사전 계산하는 동적 GI · Enlighten

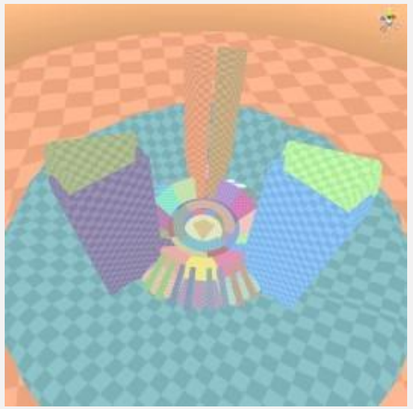
Enlighten은 조명의 최종 결과를 고정하여 저장하는 대신, 정적인 표면 사이에서 빛이 전달되는 관계를 미리 계산합니다.
그림의 cluster는 간접광 계산에 사용하는 표면의 묶음입니다. 표면 사이의 가시성과 전달 관계를 사전 계산해 둡니다.
광원의 색이나 밝기가 바뀌면 이 관계를 이용하여 간접광을 다시 계산하고, Realtime Lightmap과 Light Probe로 적용합니다.

다만 벽을 옮기거나 문을 여는 것처럼 표면 사이의 관계가 달라지는 변화까지 그대로 처리하지는 못합니다.
조명 갱신도 여러 프레임에 걸쳐 이루어지므로, 태양 이동처럼 천천히 변하는 조명에 적합합니다.

Courtyard · Enlighten 적용 장면



간접광 계산에 사용하는 cluster



표면을 묶어 반사광의 전달을 계산

주요 대상: 중급 이상 PC·콘솔. 일부 고성능 모바일에서는 장면 규모와 Realtime Lightmap 해상도를 낮춰 사용합니다.
CPU에서 조명을 갱신하므로, 조명 변화에 대한 반응 속도와 계산 비용을 함께 조절해야 합니다.

* Unity 6.7 LTS는 Enlighten Realtime GI를 지원하는 마지막 버전으로 안내되어 있습니다.

Enlighten · Unity 기술 설명 / Courtyard · Unity 공식 영상

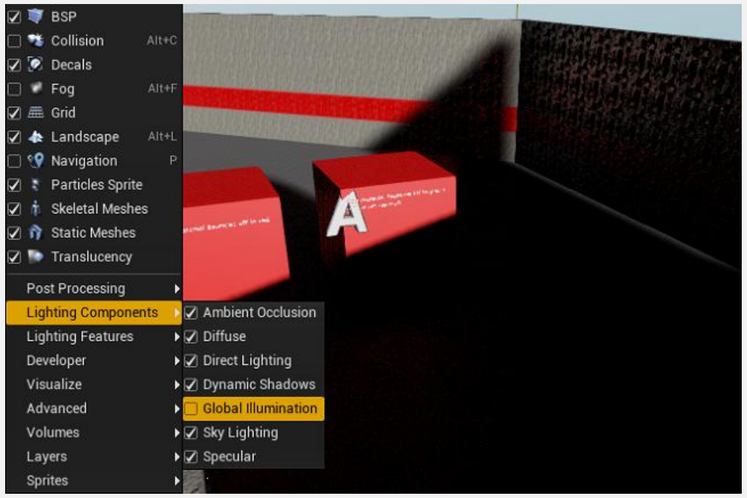
리얼타임 렌더링 4판 · 11장 전역 조명 · 5차시

실제 엔진의 확산 GI(Diffuse GI in Engines)

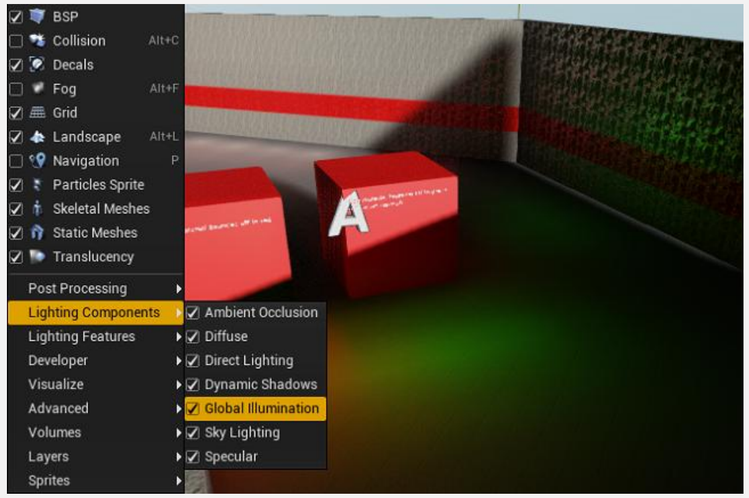
공간 정보를 갱신하는 동적 GI · LPV와 SVOGI

Unreal의 LPV는 RSM에서 얻은 반사광을 3D 격자에 주입하고, 이웃 cell로 전파하여 간접광을 갱신합니다.
그림에서는 GI를 켰을 때 붉은 물체와 초록 벽에서 반사된 빛이 바닥에 더해집니다.
낮은 해상도에서는 light bleeding과 세부 조명 손실이 남으며, 넓은 범위를 같은 해상도로 처리하면 메모리와 계산 비용이 증가합니다.
CRYENGINE의 SVOGI는 장면을 voxel로 표현하고, voxel을 따라 광선을 추적하여 간접광과 차폐를 구합니다.
같은 공간 표현을 사용하더라도, LPV의 cell 간 전파와 Voxel 방법의 광선 추적은 구분할 수 있습니다.

Unreal LPV · GI OFF



Unreal LPV · GI ON



주요 대상: Unreal LPV는 UE4 시기의 DX11 compute shader 지원 환경에서 제공한 기능이며, 현재는 deprecated 상태입니다.
SVOGI는 PC·콘솔 적용 사례가 있으며, Crysis Remastered의 PC·PS4·Xbox One 출시 자료에서도 사용을 확인할 수 있습니다.

Unreal LPV 공식 문서 / SVOGI · CRYENGINE 공식 영상

실제 엔진의 확산 GI(Diffuse GI in Engines)

캐시와 광선 추적을 활용하는 동적 GI · SCGI와 Lumen

Unity SCGI는 표면의 위치와 방향을 기준으로 patch를 구성하고, 각 patch에서 광선을 추적하여 조명을 계산합니다.
저장한 결과를 보간하여 픽셀의 간접광을 구합니다. 그림의 색 구획은 조명 세기가 아니라 patch를 구분한 모습입니다.
조명을 미리 bake하지 않고 런타임에 갱신하므로, 광원뿐 아니라 기하 구조의 변화에도 대응할 수 있습니다.

Unreal Lumen은 화면 공간 탐색과 장면 공간의 광선 추적을 결합합니다. Surface Cache에 표면의 재질과 조명 정보를 유지합니다.
광선이 표면에 도달했을 때 캐시를 조회하며, 직접광·간접광을 여러 프레임에 나누어 갱신하여 반복 계산을 줄입니다.
Lumen의 두 그림은 같은 장면의 직접광과 간접광 성분입니다. 두 엔진의 캐시 구성과 조회 방식은 서로 다릅니다.

* SCGI는 확산 GI를 처리합니다. Lumen은 정반사도 함께 다루지만, 해당 확장 내용은 이후 Specular GI에서 살펴봅니다.

SCGI · 실제 장면



SCGI · 표면 patch



Lumen · Direct Lighting



Lumen · Indirect Lighting



주요 대상: SCGI는 URP의 PC·콘솔·고성능 모바일 대상이며, Unity 6.7 Beta에서 제공합니다. 하드웨어 RT가 없으면 compute로 처리합니다.
Lumen은 고성능 PC·PS5·Xbox Series X/S·Switch 2가 주요 대상이며, 최신 문서에는 Android Vulkan 지원도 안내되어 있습니다.

SCGI · 개발팀 가이드 / Unite Seoul 2026 공식 영상
Lumen · 공식 기술 문서 / Lumen · Inside Unreal 영상

리얼타임 렌더링 4판 · 11장 전역 조명 · 5차시

오늘의 순서 - 6차시

00 반사 전역 조명 — 11.6

01 통합적 접근 방식 — 11.7

* 오늘 끝을 보는걸로...

정반사 전역 조명 (Specular Global Illumination)

11.5 파트에서는 빛이 눈에 들어오기 직전에 Diffuse 표면에서 반사되는 경우를 다루었습니다.
넓은 방향의 빛이 함께 반영되므로, 방향에 따른 세부적인 차이가 최종 결과에서는 비교적 부드럽게 나타납니다.
이에 따라 적은 수의 계수나 낮은 해상도의 표현으로 간접 조명을 근사할 수 있었습니다.

이번 장에서는 빛이 눈에 들어오기 직전에 Specular 표면에서 반사되는 경우를 다룹니다.
표면이 매끄러울수록 좁은 방향의 조명이 결과에 크게 영향을 주며, 시점이 움직이면 참조하는 방향도 달라집니다.
주변 물체의 형태나 밝기 변화를 반사상에 표현하려면, 더 세밀한 방향 정보를 보존해야 합니다.

대신 정반사는 반사 방향 주변의 좁은 범위만 참조한다는 특징이 있습니다.

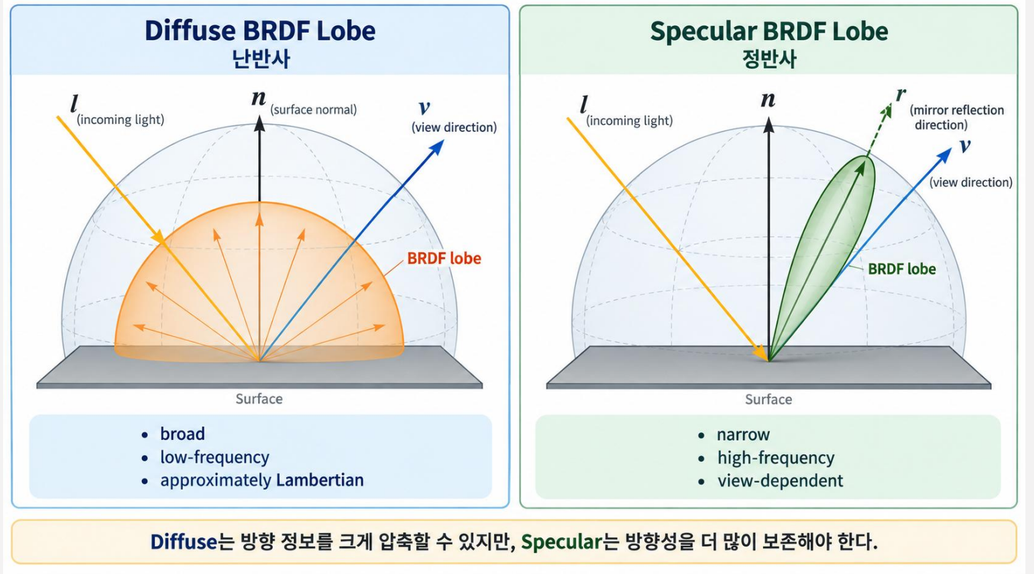
기존 방법들도 남아 있는 방향 정보에 따라 활용할 수 있습니다.

AHD / HL2는 대표 방향으로 간접광의 하이라이트를 근사합니다.
SH / Spherical Gaussian으로 방향별 radiance를 저장하면,
비교적 거친 표면의 정반사에도 활용할 수 있습니다.
Voxel Cone Tracing도 반사 방향을 따라 cone을 조회할 수 있습니다.

그러나 위의 방법만으로는 품질 높은 거울이나
물에 비친 모습등의 선명한 반사상을 표현하기 어렵습니다.

11.6 파트에서는 환경맵, voxel, 평면 반사와 화면 공간 반사로
정반사 전역 조명을 처리하는 방법을 살펴봅니다.

11.6 Specular Global Illumination



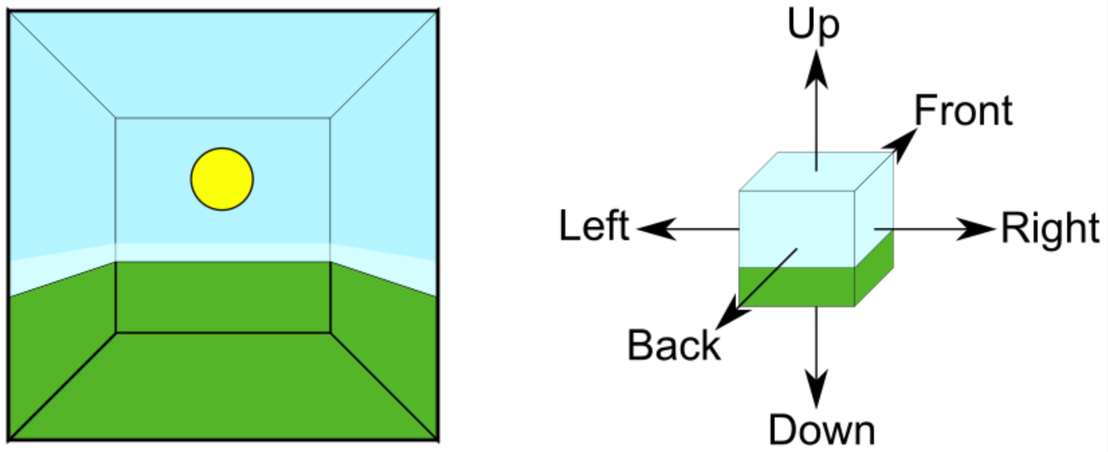
국소 환경 맵 (Localized Environment Maps)

환경맵은 주변에서 들어오는 빛을 방향별로 저장한 텍스처입니다.
Cubemap은 이를 표현하는 방법 중 하나로, 한 위치에서 여섯 방향을 바라본 모습을 6장의 텍스처에 저장합니다.
표면에서는 시선과 법선으로 반사 방향을 구하고, 그 방향의 환경맵을 조회하여 주변 환경의 반사를 표현할 수 있습니다.

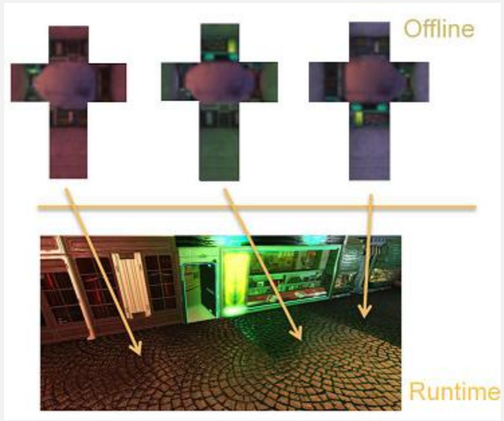
장면 곳곳의 특정 위치에서 환경맵을 생성하여 사용하는 것을 Reflection Probe라고 합니다.
그림은 Cubemap의 내부 구조와, 여러 위치에서 만든 Cubemap을 실제 장면에 적용하는 과정을 보여줍니다.

이 방법은 공간상의 조명 샘플을 촘촘하게 배치하는 대신, 각 위치에서의 방향 정보를 세밀하게 저장합니다.
선명한 반사를 적은 비용으로 근사할 수 있지만, 저장한 위치와 실제 사용하는 위치의 차이에서 오차가 발생합니다.

Cubemap · 방향별로 저장한 주변 모습



국소 Cubemap · 사전 생성과 런타임 적용



Unity · Reflection Probes / Cubemap

리얼타임 렌더링 4판 · 11장 전역 조명 · 6차시

국소 환경 맵 (Localized Environment Maps)

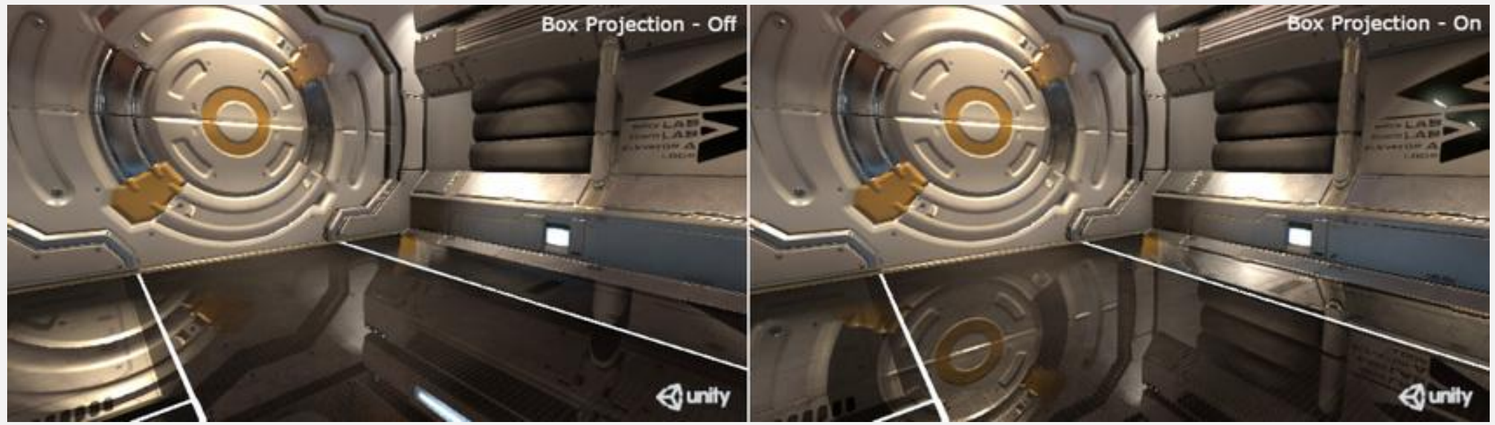
환경맵을 저장한 위치와 사용하는 위치의 차이

환경맵에 저장된 주변 모습은 probe가 놓인 위치에서 바라본 결과입니다.
다른 위치의 표면에서 같은 환경맵을 사용하면, 실제로 보여야 하는 주변 모습이나 조명과 차이가 발생할 수 있습니다.
보조 그림은 녹색 광원과 바닥의 반사광을 표시한 예시로, 위치마다 달라지는 조명을 정확히 재현하기 어렵다는 것을 보여줍니다.

또한 반사 방향만으로 환경맵을 조회하면, 주변 환경이 무한히 멀리 있는 것처럼 처리합니다.
멀리 있는 하늘이나 배경에는 적절한 근사이지만, 가까운 벽이나 물체가 있는 실내에서는 위치 차이를 무시하기 어렵습니다.
실제로는 표면이 벽에 가까워질수록 벽의 반사가 더 크게 보여야 하지만, 방향만으로 조회하면 이러한 거리 변화가 반영되지 않습니다.

아래 비교의 왼쪽에서는 바닥에 반사된 문의 크기와 위치가 어긋나며, 오른쪽에서는 공간의 크기를 고려하여 이를 보정합니다.
어느 위치의 환경맵을 적용할지와, 위치 차이에 따른 조회 방향을 어떻게 보정할지를 함께 고려해야 합니다.

Unity · Box Projection OFF / ON



[Unity · Box Projection 공식 설명]

국소 Cubemap의 지역별 조명 오차



리얼타임 렌더링 4판 · 11장 전역 조명 · 6차시

국소 환경 맵 (Localized Environment Maps)

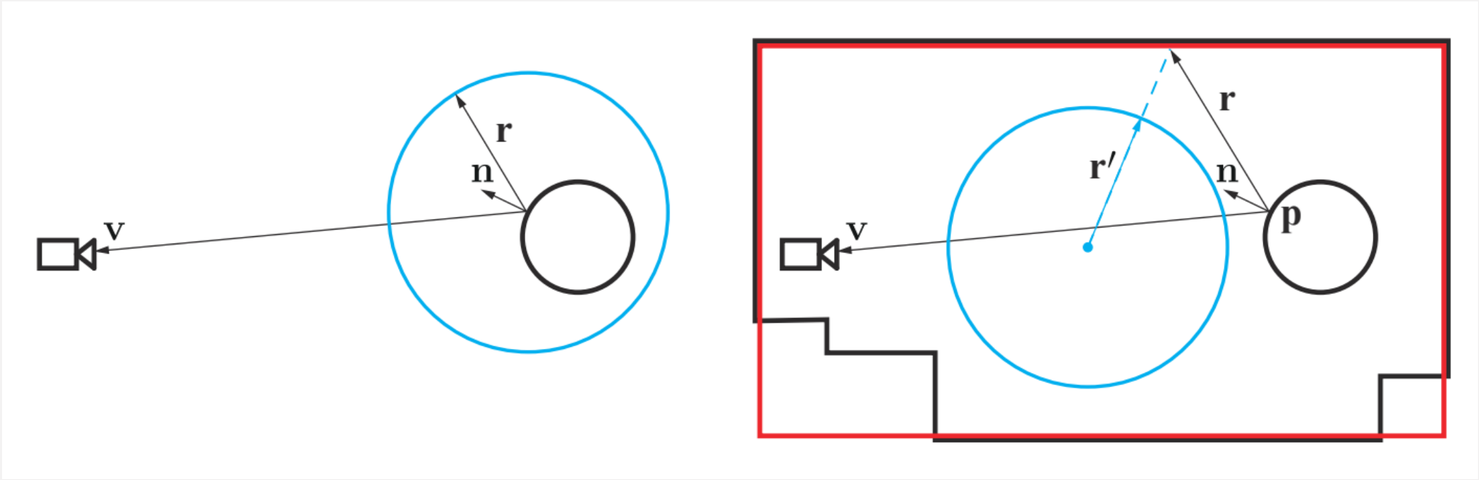
Reflection Proxy와 Parallax Correction (시차 보정)

환경맵을 사용하는 위치의 차이를 반영하기 위해, 주변 공간을 box나 sphere 같은 단순한 형태로 근사합니다.
반사 방향과의 교차점을 구하는 데 사용하는 이 형태를 Reflection Proxy라고 합니다.

먼저 셰이딩 지점에서 반사 방향으로 진행하여 proxy와 만나는 위치를 구합니다.
그다음 probe 중심에서 이 교차점을 바라보는 방향을 구하고, 이 방향으로 환경맵을 조회합니다.
그림의 왼쪽에서는 반사 방향 r을 그대로 사용하며, 오른쪽에서는 빨간 box와의 교차점으로 조회 방향 r′을 구합니다.

실제 장면 전체를 추적하는 대신 단순한 형태와의 교차만 계산하므로, 적은 비용으로 위치에 따른 반사 오차를 줄일 수 있습니다.
다만 proxy의 형태와 크기는 환경맵에 담긴 주변 공간을 적절히 근사해야 합니다.

그림 11.37 · 기존 환경맵 조회 / Proxy를 이용한 조회 방향 보정



여러 probe가 같은 영역을 덮는 경우에는 우선순위를 정하거나 결과를 혼합하여, 환경맵 사이의 전환을 완화할 수 있습니다.

국소 환경 맵 (Localized Environment Maps)

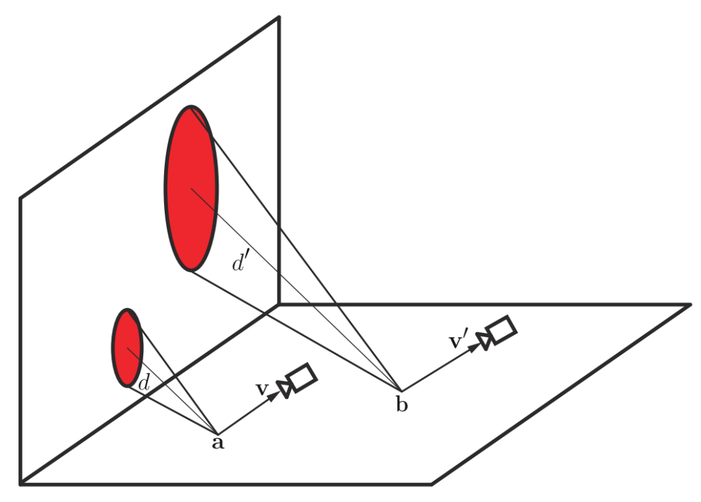
거칠기와 주변 구조까지의 거리

매끄러운 표면은 반사 방향 주변의 좁은 범위를 참조하지만, 거친 표면은 더 넓은 방향에서 들어오는 빛을 함께 반영합니다.
보조 그림의 구와 상자에서도 표면의 smoothness에 따라 반사상이 선명하거나 흐릿하게 나타나는 것을 확인할 수 있습니다.

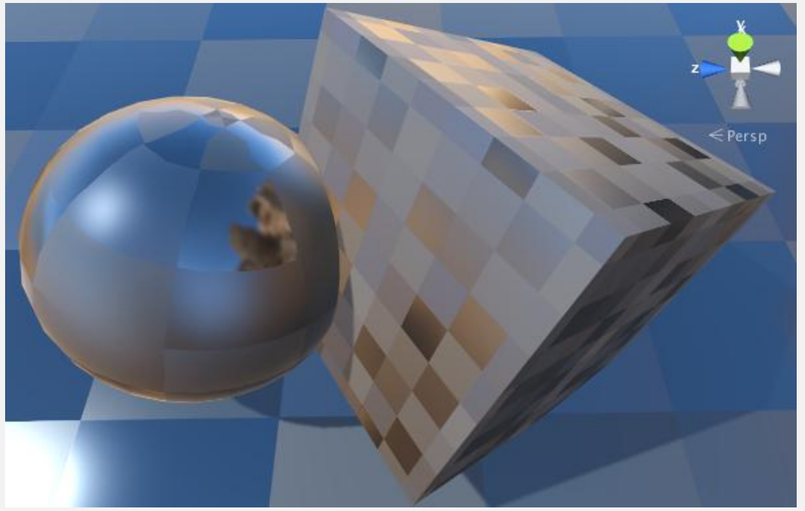
이를 매번 여러 방향에서 샘플링하는 대신, 미리 필터링한 환경맵을 사용할 수 있습니다.
mipmap에는 점점 더 넓은 방향 범위의 조명을 합친 결과를 저장하여, 거친 표면의 부드러운 반사를 근사합니다.

국소 환경에서는 표면의 거칠기뿐 아니라 주변 구조까지의 거리도 영향을 줍니다.
그림의 a와 b는 같은 BRDF와 시선 방향을 사용하지만, 먼 벽에서는 같은 로브가 더 넓은 영역을 덮습니다.
따라서 거칠기와 proxy까지의 거리를 함께 고려하여, 필터링한 환경맵의 조회 수준을 조절할 수 있습니다.

그림 11.38 · 거리에 따라 달라지는 참조 영역



표면의 smoothness에 따른 반사 선명도



* 위치에 따라 참조 영역의 크기와 형태가 달라지므로, 미리 필터링한 환경맵만으로 완전히 정확하게 처리하기는 어렵습니다.

국소 환경 맵 (Localized Environment Maps)

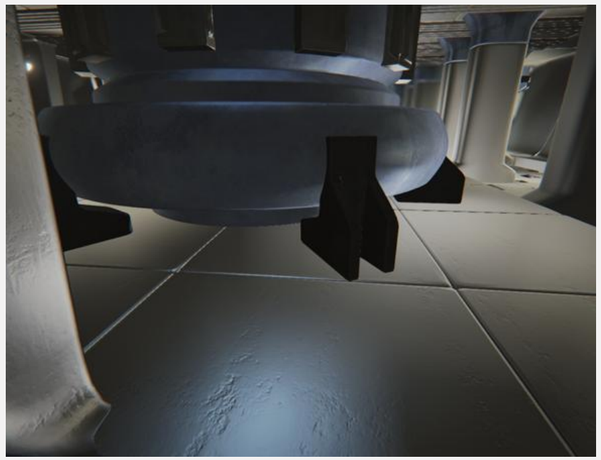
국소 환경맵의 한계와 차폐 보완

Reflection Proxy는 주변 공간을 단순하게 근사하므로, 실제 기하 구조와 정확히 일치하는 경우는 드뭅니다.
형태가 맞지 않는 곳에서는 반사상이 늘어나거나 위치가 어긋날 수 있으며, 선명한 반사일수록 오차가 두드러집니다.
또한 빛을 가려야 하는 벽이나 물체를 proxy가 표현하지 못하면, 가려진 방향의 밝은 조명을 조회할 수 있습니다.
위치에 따른 반사 방향을 보정하더라도, 세부적인 차폐까지 해결되는 것은 아닙니다.

그림은 거친 표면에 Cubemap 반사만 사용한 결과와, Lightmap 정보를 함께 사용한 결과를 비교합니다.
각 표면 위치의 간접 조명과 지역별 차폐 정보를 이용하면, 과도하게 밝아진 반사를 완화할 수 있습니다.

방향성 폐색으로 차폐를 보완하거나, probe에 깊이 정보를 저장하고 광선을 추적하는 방법도 있습니다.
더 정확한 공간 정보를 사용할수록 오차를 줄일 수 있지만, 저장량과 조회 비용도 증가합니다.

지역별 차폐가 부족한 Cubemap 반사



Lightmap 정보를 이용해 보완한 반사



11.6.2 파트에서는 주변 조명이나 장면이 변할 때 환경맵을 갱신하는 방법을 다룹니다.

Unreal · Reflection Environment 공식 설명

환경맵의 동적 갱신 (Dynamic Update of Environment Maps)

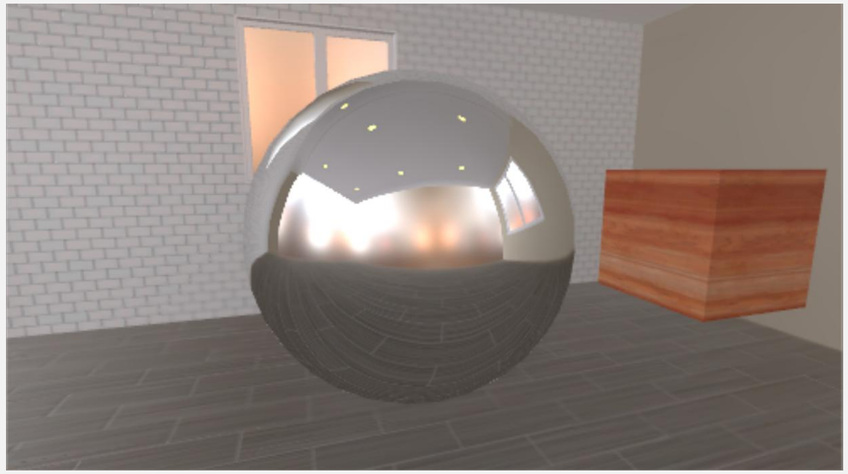
환경맵을 실행 중에 갱신해야 하는 이유

국소 환경맵에는 probe 위치에서 바라본 주변 장면의 조명이 저장됩니다.
이를 미리 계산해 두면 실행 중에는 적은 비용으로 조회할 수 있지만, 저장한 이후의 변화는 반사상에 반영되지 않습니다.

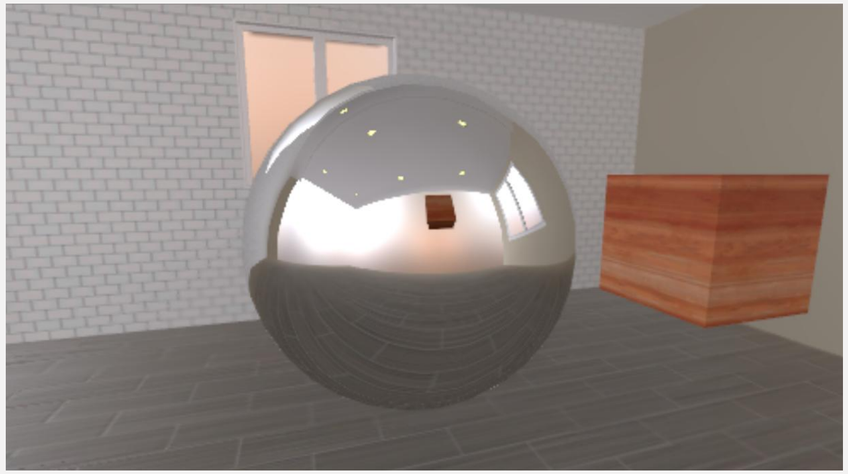
시간대에 따라 태양과 하늘의 조명이 바뀌거나, 주변 물체가 움직이면 반사에 필요한 정보도 달라집니다.
이러한 변화를 반영하려면 실행 중에 환경맵을 갱신해야 합니다.
여러 시간대와 장면 상태를 모두 미리 준비하는 방법은 제작 시간과 저장 용량이 크게 늘어날 수 있습니다.

환경맵을 갱신할 때에는 주변 장면을 다시 렌더링하고, 거칠기별 반사에 사용할 수 있도록 필터링해야 합니다.
특히 Cubemap은 여섯 방향의 장면을 담으므로, 많은 probe를 매 프레임 모두 갱신하면 비용이 커집니다.
따라서 어떤 probe를 언제 갱신할지 정하고, 한 프레임에서 처리할 작업량을 제한해야 합니다.

미리 생성한 환경맵 · 동적 물체가 빠진 반사



실행 중 캡처한 환경맵 · 동적 물체를 포함한 반사



Unity · Configuring Reflection Probes

리얼타임 렌더링 4판 · 11장 전역 조명 · 6차시

환경맵의 동적 갱신 (Dynamic Update of Environment Maps)

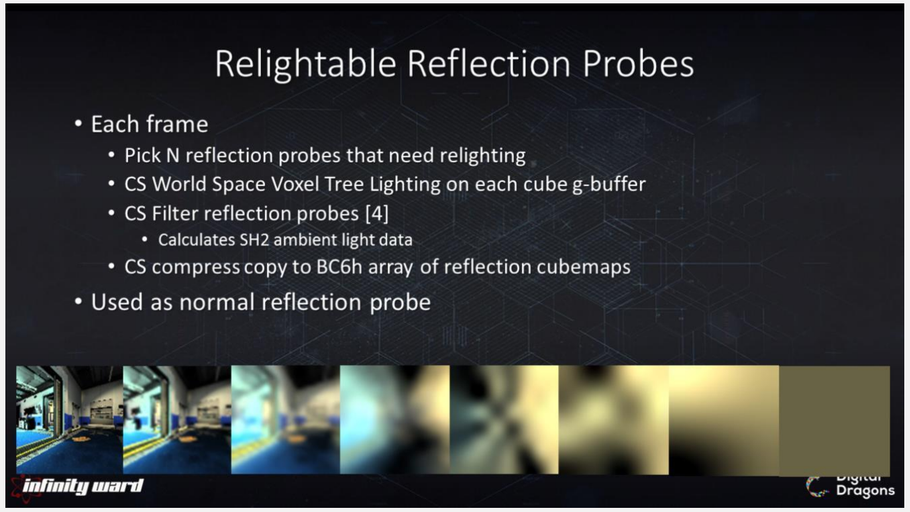
환경맵 갱신을 여러 프레임에 분산

화면에 사용하는 모든 probe를 매 프레임 갱신할 필요는 없습니다.
천천히 변하는 주변 조명은 낮은 빈도로 갱신하고, 빠르게 움직이는 물체의 반사는 화면 공간 반사 같은 다른 기법으로 보완할 수 있습니다.

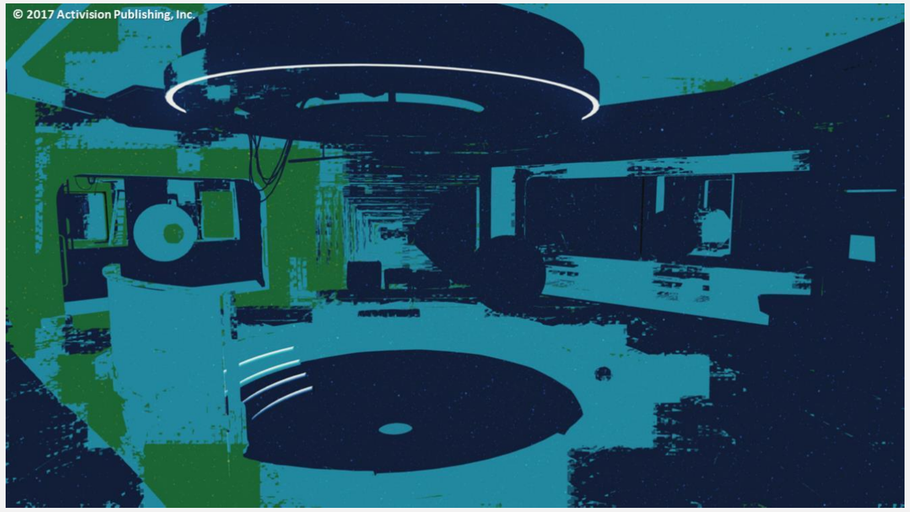
먼저 환경맵 갱신에 사용할 프레임 시간 예산을 정하고, 그 범위에서 일부 probe만 처리합니다.
카메라와의 거리나 마지막 갱신 이후 지난 시간 등을 기준으로 갱신 순서를 정할 수 있습니다.
장면을 불러올 때 필요한 probe를 먼저 준비하고, 나머지는 시야에 들어오는 순서에 따라 갱신할 수도 있습니다.

하나의 probe도 Cubemap의 여섯 면을 한꺼번에 렌더링하는 대신, 매 프레임 한 면씩 나누어 처리할 수 있습니다.
한 프레임에 집중되는 비용은 줄어들지만, 반사가 장면의 변화를 뒤늦게 따라가거나 면 사이에 시간 차이가 나타날 수 있습니다.

매 프레임 일부 probe를 선택하여 갱신



장면의 위치에 따라 구분한 probe 영역



Activision · Rendering of Call of Duty: Infinite Warfare

리얼타임 렌더링 4판 · 11장 전역 조명 · 6차시

환경맵의 동적 갱신 (Dynamic Update of Environment Maps)

표면 정보 재사용과 빠른 필터링

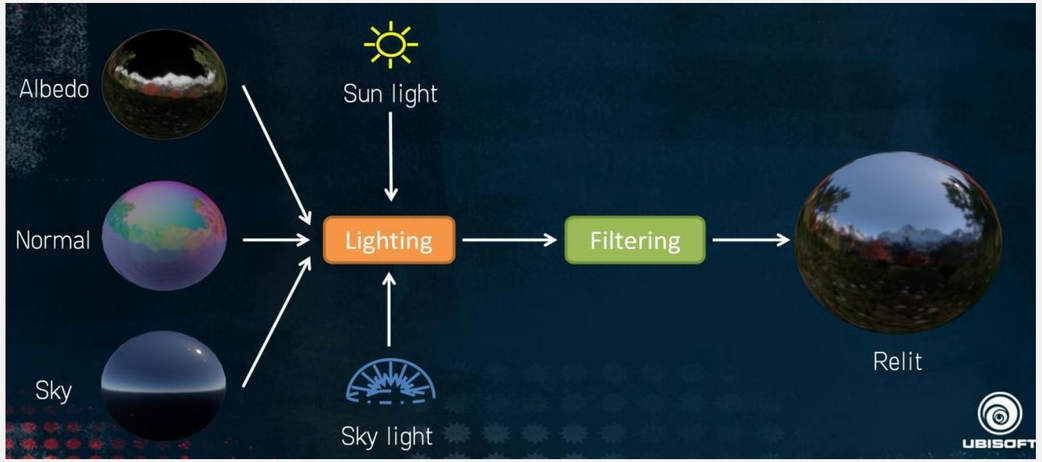
주변 기하 구조가 고정되어 있다면, 조명이 바뀔 때마다 장면 전체를 다시 렌더링하는 비용을 줄일 수 있습니다.
probe에서 본 표면의 색과 법선, 깊이 등을 G-buffer로 미리 저장하고, 실행 중에는 이 정보로 조명을 다시 계산합니다.
이렇게 하면 기하 구조를 반복해서 처리하지 않고도, 시간대나 광원 변화에 맞춰 환경맵을 갱신할 수 있습니다.

* G-buffer는 조명 계산에 필요한 표면의 깊이와 법선, 재질색 등의 정보를 픽셀별로 저장한 텍스처들의 집합입니다.
* G-buffer에 대한 세부 내용은 20-1, Deferred Shading파트에서 다루게 됩니다.

조명을 다시 계산한 환경맵은 거칠기별 반사에 사용할 수 있도록 필터링해야 합니다.
먼저 저렴한 필터로 축소한 mip chain을 만들고, 이 데이터를 중요도 샘플링에 활용할 수 있습니다.
이를 통해 적은 수의 텍스처 조회로 넓은 방향 범위의 빛을 근사할 수 있습니다.
HDR 환경맵은 BC6H 등으로 압축하여 메모리 사용량과 조회할 때의 데이터 전송량을 줄일 수 있습니다.

다만 미리 저장한 G-buffer는 당시의 기하 구조를 기준으로 하므로, 움직이거나 새로 생긴 물체는 별도로 반영해야 합니다.

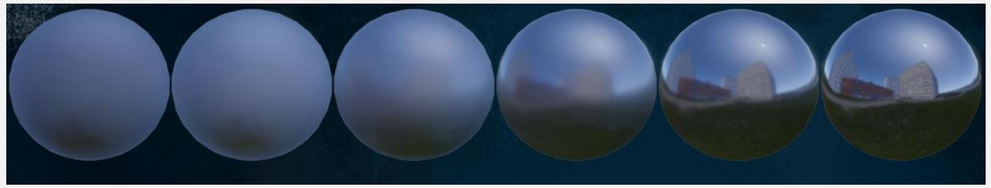
저장한 표면 정보로 조명을 다시 계산하는 과정



움직이는 물체는 별도로 렌더링해 추가하거나, 다른 반사 기법으로 보완할 수 있습니다.

Ubisoft · Rendering the World of Far Cry 4

거칠기별 반사를 위한 필터링 결과



거친 표면일수록 더 넓은 방향의 조명을 반영합니다.

복셀 기반 방법 (Voxel-Based Methods)

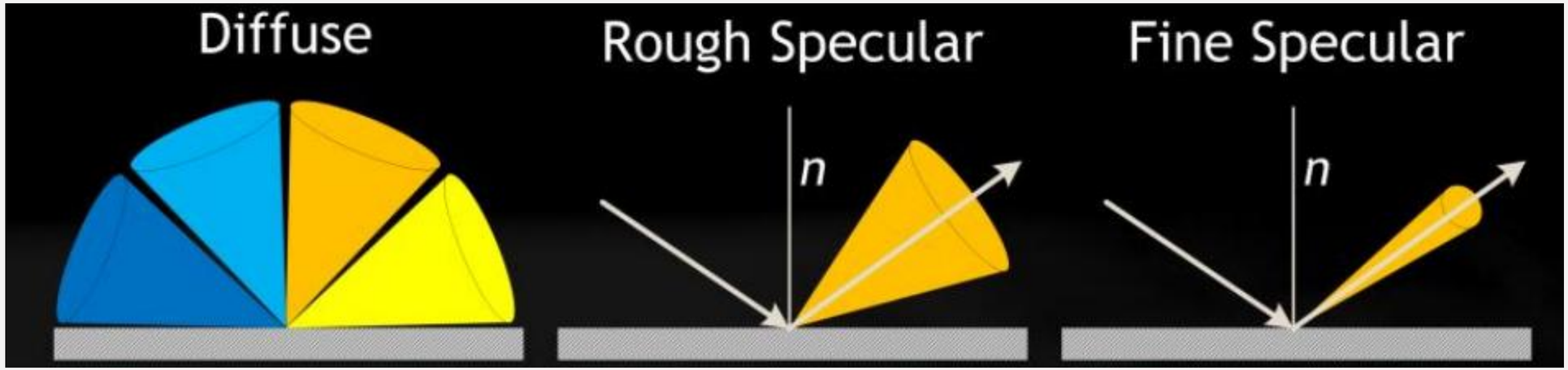
Voxel Cone Tracing으로 정반사 구하기

국소 환경맵은 적은 비용으로 반사를 표현할 수 있지만, probe의 배치 간격이나 proxy의 단순한 형태 때문에 공간적인 오차가 발생할 수 있습니다.
이러한 한계를 보완하기 위해, 11.5.7에서 다룬 Voxel Cone Tracing을 정반사에도 적용할 수 있습니다.

장면의 지오메트리와 반사광을 저장한 voxel을 이용하고, 셰이딩 지점에서 반사 방향을 중심으로 cone을 추적합니다.
하나의 cone은 자신이 덮는 방향 범위에서 들어오는 평균적인 radiance를 근사합니다.
Cone을 따라 이동하며 단면의 크기에 맞는 해상도를 조회하고, 앞쪽 차폐를 고려하여 빛을 누적합니다.

Diffuse GI는 표면의 반구 방향으로 여러 cone을 사용하여 빛을 모읍니다.
반면 정반사는 반사 방향 주변의 빛이 주로 필요하므로, 반사 로브가 좁은 glossy 표면에서는 하나의 cone만으로도 근사할 수 있습니다.
따라서 여러 cone을 사용하는 확산 GI보다 적은 추적 횟수로 정반사 성분을 구할 수 있습니다.

확산 반사와 정반사에서의 cone 조회 범위



NVIDIA · Voxel Global Illumination, GDC 2015

복셀 기반 방법 (Voxel-Based Methods)

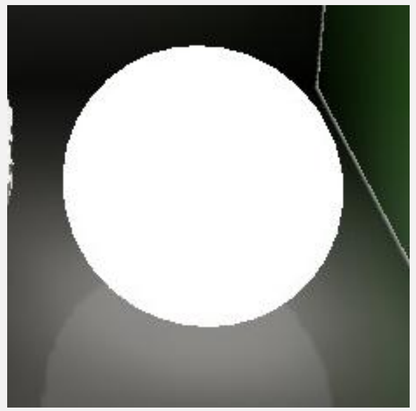
좁은 cone에서 드러나는 voxel 해상도의 한계

표면이 거칠수록 반사 로브가 넓어지므로, 넓은 cone으로 여러 방향에서 들어오는 빛을 반영합니다.
거친 정반사에 여러 cone이 필요할 때에는, 흐릿한 반사상을 국소 환경맵으로 대신하여 비용을 줄일 수 있습니다.

반대로 거울처럼 매끄러운 표면에서는 cone이 하나의 ray에 가까울 정도로 좁아집니다.
하지만 cone이 좁아져도, voxel로 저장하면서 잃어버린 표면의 세부 형상이 복원되는 것은 아닙니다.
넓은 cone에서는 여러 voxel의 값이 함께 반영되어 경계가 흐려지지만, 좁은 cone에서는 각진 경계가 반사상에 드러날 수 있습니다.

즉, 조회하는 방향 범위는 좁아져도 공간 표현의 해상도는 voxel의 크기로 제한됩니다.
표면의 텍스처가 이러한 오차를 가려주기도 하지만, 선명한 거울 반사를 정확히 표현하는 데에는 한계가 있습니다.
반사면이 평면이라면, 반사된 시점에서 장면을 다시 렌더링하여 더 정확한 반사상을 얻는 방법을 사용할 수 있습니다.

원래 구의 윤곽 · Mesh



저장한 윤곽 · Voxel 디버그



반사상에 남은 voxel 경계 · NVIDIA VXGI 사례



왼쪽은 저장된 형상의 차이이며, 오른쪽은 다른 장면의 반사 오류 사례입니다.

11.6.4 파트에서는 평면을 기준으로 장면을 다시 렌더링하는 평면 반사를 다룹니다.

OGRE · Voxel Cone Tracing / NVIDIA · VXGI, GDC 2015

평면 반사 (Planar Reflections)

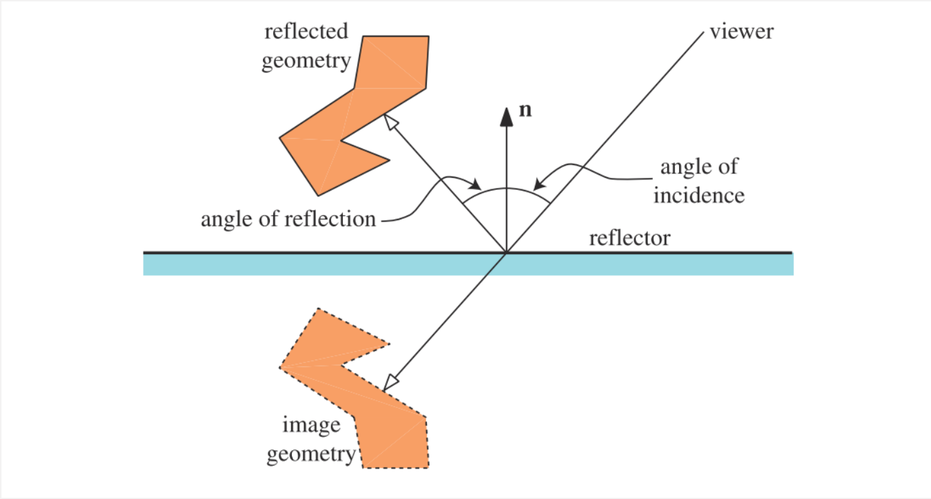
반사된 시점에서 장면을 다시 렌더링하기

Voxel로 근사한 형상을 조회하는 대신, 원래 장면의 지오메트리를 다시 렌더링하여 반사상을 만들 수도 있습니다.
거울이나 잔잔한 수면처럼 반사면을 하나의 평면으로 표현할 수 있을 때 사용하는 방법이 평면 반사입니다.

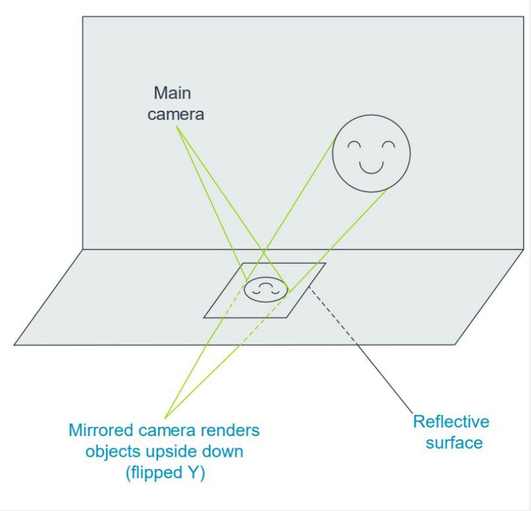
평면에서 빛이 반사될 때에는, 법선을 기준으로 입사각과 반사각이 같아집니다.
이 관계를 이용하면, 반사면 너머의 대칭 위치에 물체의 상이 있는 것으로 해석할 수 있습니다.

실제로는 원래 카메라를 평면 너머의 대칭 위치로 옮기고, 시선 방향도 반사한 가상 카메라를 사용합니다.
이 가상 카메라에서 원래의 지오메트리와 광원을 이용해 장면을 다시 렌더링하여, 반사된 시점에서 보이는 모습을 구합니다.
그 결과를 별도의 텍스처에 저장하고, 반사면의 각 위치에 대응하는 부분을 조회하여 표면에 적용합니다.

그림 11.39 · 평면 너머의 대칭 위치에 생기는 상



반사 영상을 만드는 가상 카메라



ARM · Local Cubemap Rendering Techniques

평면 반사 (Planar Reflections)

반사면을 기준으로 한 클리핑과 렌더링 비용

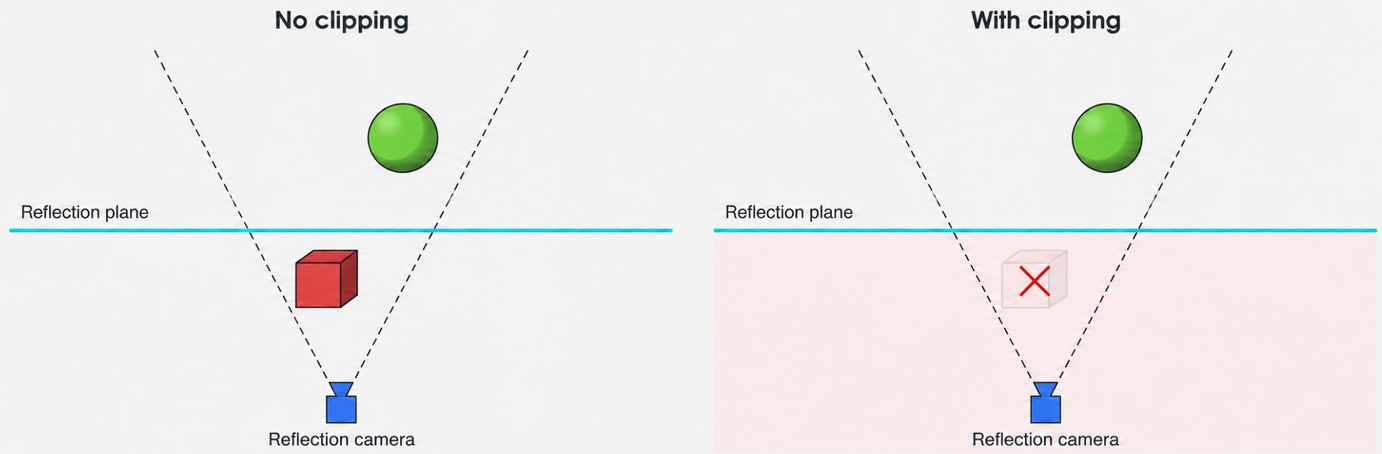
반사 카메라는 반사면의 반대쪽에 있으므로, 그대로 렌더링하면 반사에 포함되지 않아야 할 물체까지 보일 수 있습니다.
예를 들어 수면 위의 장면을 반사할 때에는, 수면 아래의 물체가 반사 영상에 들어가지 않도록 해야 합니다.

이를 위해 반사면과 일치하는 clipping plane을 두고, 평면의 어느 쪽에 있는지에 따라 지오메트리를 남기거나 제거합니다.
반사 영상을 만들 때에는 원래 카메라와 같은 쪽에 있는 장면을 남기고, 반사면 뒤쪽의 장면을 제외합니다.

생성한 반사 텍스처를 필터링하면, 선명한 거울 반사뿐 아니라 거친 표면의 부드러운 반사도 근사할 수 있습니다.

다만 본래 화면과 별도로 장면을 다시 렌더링해야 하며, 서로 다른 반사 평면에는 각각의 반사 영상이 필요합니다.
따라서 반사면의 수와 반사 영상의 해상도를 제한하여, 추가 렌더링 비용을 조절해야 합니다.

반사면과 일치하는 clipping plane · 반사 영상에 포함할 영역과 제외할 영역



Epic · Planar Reflections

화면 공간 반사 (Screen-Space Reflections)

이미 렌더링한 화면 정보를 이용하는 반사

평면 반사처럼 다른 시점에서 장면을 다시 렌더링하는 대신, 현재 화면에 저장된 정보를 이용하여 반사를 구할 수도 있습니다.
이처럼 화면 공간의 깊이와 색 정보를 이용하여 정반사 효과를 만드는 방법을 화면 공간 반사(Screen-Space Reflections, SSR)라고 합니다.

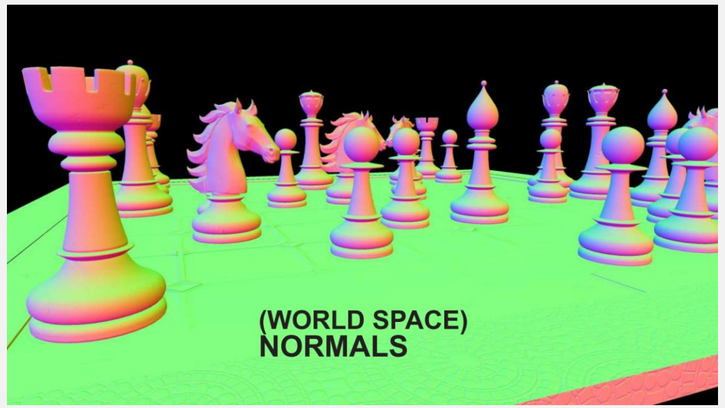
깊이 버퍼를 이용하면 화면에 보이는 표면의 위치를 구할 수 있으며, 법선 정보와 시선 방향을 이용하면 반사 방향을 계산할 수 있습니다.
이 방향으로 다른 표면을 찾아낸 뒤, 해당 위치의 색을 읽어 반사에 들어오는 빛으로 사용합니다.

확산 반사는 표면 위쪽의 넓은 반구에서 들어오는 빛이 필요하지만, 선명한 정반사는 반사 방향 주변의 좁은 범위가 중요합니다.
따라서 필요한 방향의 표면이 현재 화면에 포함되어 있다면, 화면 정보만으로도 유용한 반사 결과를 얻을 수 있습니다.

깊이 버퍼 · 표면의 위치



법선 버퍼 · 반사 방향 계산



색 버퍼 · 교차 위치의 빛 조회



AMD GPUOpen · FidelityFX SSSR Technology

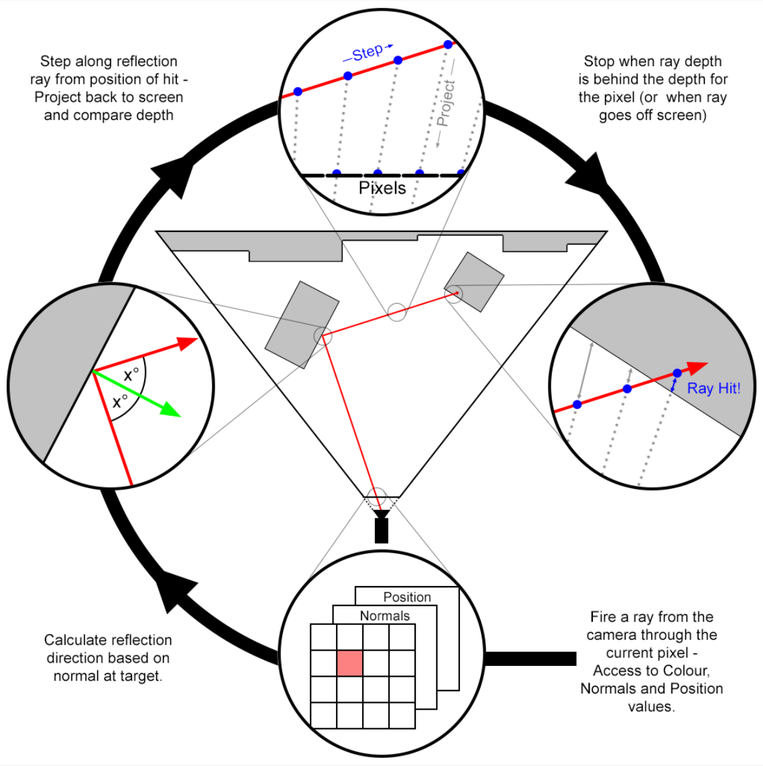
리얼타임 렌더링 4판 · 11장 전역 조명 · 6차시

화면 공간 반사 (Screen-Space Reflections)

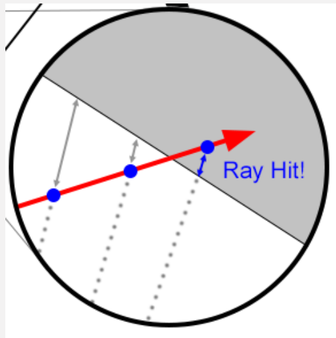
깊이 버퍼에서 반사 광선의 교차점 찾기

기본적인 SSR은 표면에서 반사 방향으로 광선을 진행시키면서, 광선 위의 점을 화면에 투영합니다.
같은 픽셀에 기록된 표면의 깊이와 광선 위의 점의 깊이를 비교하여, 그 점이 표면 앞쪽에 있는지 뒤쪽에 있는지 확인합니다.
앞쪽에서 뒤쪽으로 넘어간 구간을 교차 후보로 판단한 뒤, 이분 탐색으로 범위를 좁혀 반사에 사용할 화면 위치를 구합니다.

반사 방향 계산과 화면 투영, 깊이 비교의 전체 과정



표면 앞쪽에서 뒤쪽으로 넘어가는 구간



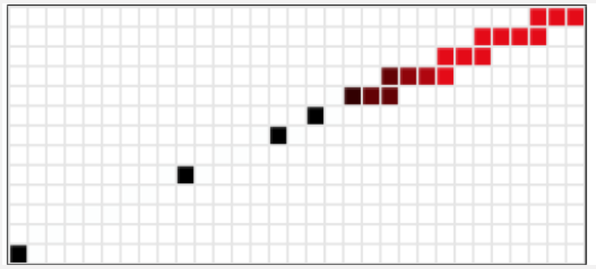
빨간 선은 반사 광선이며, 파란 점은
깊이를 비교하는 지점입니다.

흰 영역은 표면 앞쪽, 회색 영역은 물체 내부입니다.
마지막 점이 회색 영역으로 들어갔으므로,
앞선 점과의 사이에서 교차 후보를 찾습니다.

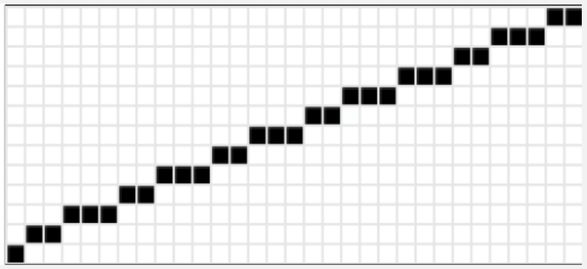
일정한 월드 공간 보폭과 화면 공간의 DDA 비교

월드 공간의 일정한 보폭은 화면에서 픽셀을 건너뛰거나 중복 조회할 수 있습니다.
DDA(Digital Differential Analyzer)는 화면 픽셀을 순서대로 검사하여 이를 줄입니다.

일정한 월드 공간 보폭 (빨강: 중복 조회)



화면 픽셀을 순서대로 검사



Matthew Shaw · SSR 도해 / McGuire & Mara · Screen-Space Ray Tracing

화면 공간 반사 (Screen-Space Reflections)

거친 표면의 반사와 확률적 샘플링

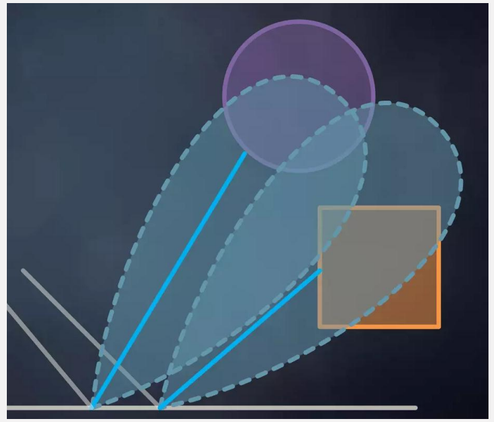
반사 방향으로 광선 하나만 추적하는 기본 방식은 선명한 거울 반사에 적합합니다.
거친 표면의 반사는 여러 방향에서 들어오는 빛이 섞이므로, 단일 방향의 조회만으로는 충분히 표현하기 어렵습니다.

간단한 방법은 반사 결과를 별도의 버퍼에 저장한 뒤, 서로 다른 정도로 흐려진 반사 영상을 만드는 것입니다.
거칠기에 따라 적절한 영상을 조회할 수 있지만, 화면에서의 필터링은 표면의 방향과 깊이 경계를 충분히 반영하지 못합니다.

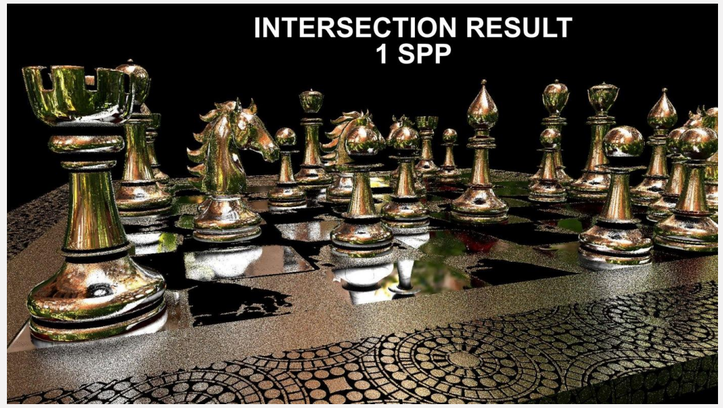
확률적 SSR은 BRDF에 따라 기여도가 높은 방향을 더 자주 선택하고, 그 방향으로 광선을 추적합니다.
거칠기에 따른 반사 방향의 분포를 샘플링하여, 여러 방향에서 들어오는 빛의 합을 근사하는 방법입니다.

적은 수의 광선에서 생기는 노이즈를 줄이기 위해, 주변 픽셀의 교차 결과를 가중치를 조정하여 재사용하고 이전 프레임의 결과도 누적합니다.
다만 주변 픽셀에서 보이는 표면이 비슷하다는 가정이 필요하므로, 가림 관계나 화면 내용이 달라지는 곳에서는 재사용에 주의해야 합니다.

BRDF에 따른 방향 선택



1 spp · 필터링 전 반사 결과



디노이징 후 반사 결과



Stachowiak · Stochastic SSR / AMD · FidelityFX SSSR

리얼타임 렌더링 4판 · 11장 전역 조명 · 6차시

화면 공간 반사 (Screen-Space Reflections)

계층 깊이 버퍼(Hi-Z)를 이용한 탐색 가속

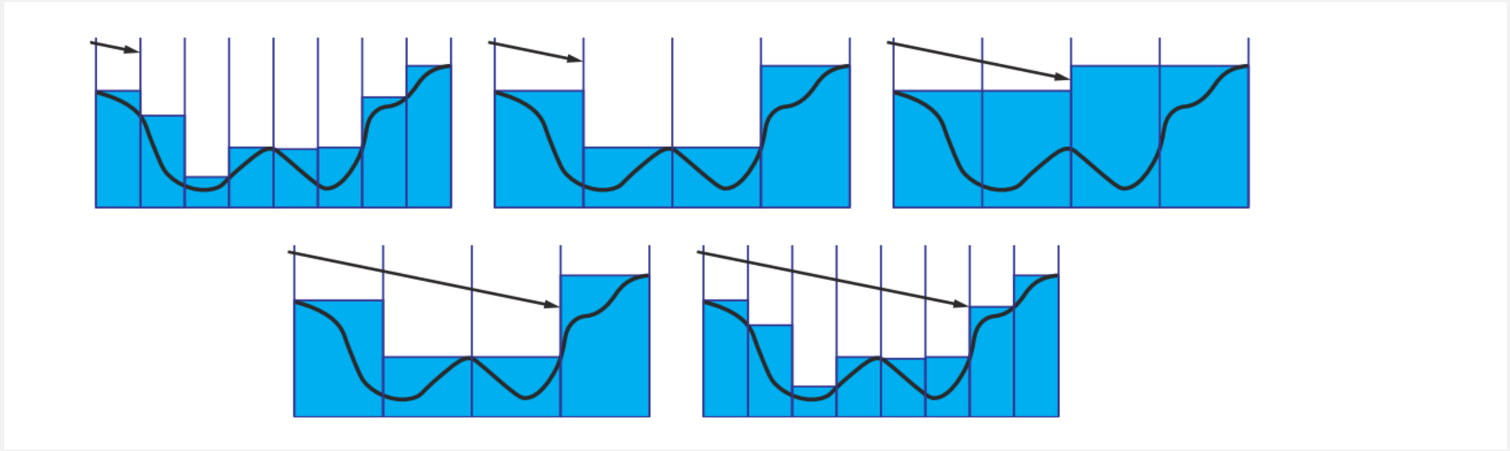
SSR은 광선을 따라 깊이 버퍼를 반복해서 조회하므로, 탐색 구간이 길어질수록 계산과 메모리 접근 비용이 커집니다.
이를 줄이기 위해 여러 해상도의 깊이 버퍼로 구성한 Hi-Z(Hierarchical Z-buffer), 즉 계층 깊이 버퍼를 사용할 수 있습니다.

교재에서 설명하는 방식은 해상도를 단계마다 절반으로 줄이고, 각 영역에서 가장 가까운 표면의 깊이를 저장합니다.
이렇게 만든 깊이 피라미드(depth pyramid)를 이용하면, 넓은 영역을 한 번에 검사할 수 있습니다.

교차하지 않는 영역은 더 낮은 해상도에서 큰 보폭으로 건너뛰고, 교차 가능성이 나타나면 더 높은 해상도로 내려가 확인합니다.
이 과정을 반복하여, 최종적으로 원래 해상도에서 교차점을 찾습니다.

광선 추적을 생략하고 반사 프록시와의 교차 위치에서 화면의 색을 조회하거나,
평면 반사를 가정하여 화면의 각 픽셀을 반사될 위치에 기록하는 방법도 있습니다.

그림 11.41 · 교차 가능성에 따른 깊이 계층의 단계 조절



Real-Time Rendering · 그림 11.41 / AMD · FidelityFX SSSR

낮은 해상도의 깊이 버퍼



여러 픽셀을 묶어 넓은 영역을 검사합니다.

화면 공간 반사 (Screen-Space Reflections)

화면 정보의 한계와 다른 반사 기법을 이용한 보완

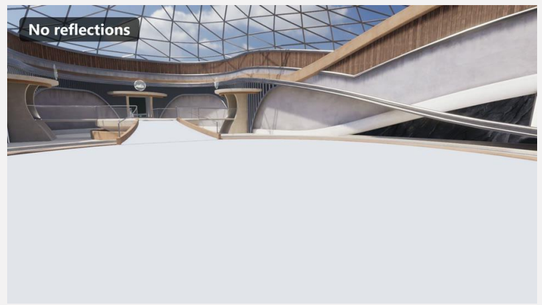
SSR에서 사용할 수 있는 정보는 현재 카메라에 보이는 표면으로 제한됩니다.
반사 광선이 화면 밖으로 나가거나, 다른 물체에 가려진 표면과 뒷면을 향하면 필요한 깊이와 색을 얻을 수 없습니다.

일반적인 깊이 버퍼는 각 픽셀에 가장 앞쪽 표면의 깊이 하나만 저장하므로, 물체의 두께와 뒤쪽 형상도 알 수 없습니다.
따라서 광선이 기록된 표면 뒤로 지나갔다는 사실만으로는, 실제로 그 물체와 교차했는지 정확히 판단하기 어렵습니다.

공간 필터로 주변의 유효한 결과를 이용하거나, 여러 층의 표면 정보를 저장하는 deep G-buffer로 가려진 표면 정보를 일부 보완할 수 있습니다.
다만 필터링만으로 화면에 존재하지 않는 정보를 모두 복원할 수는 없습니다.

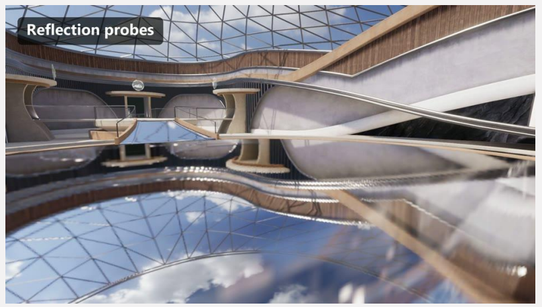
Unity URP에서도 SSR의 결과를 우선 사용하고, 유효한 결과를 얻지 못한 부분은 지역 반사 프로브로 보완합니다.
지역 프로브로도 보완하지 못한 부분에는 전역 반사 프로브를 사용하며, 기본 환경에서는 스카이박스의 반사 정보를 이용합니다.

반사 없음



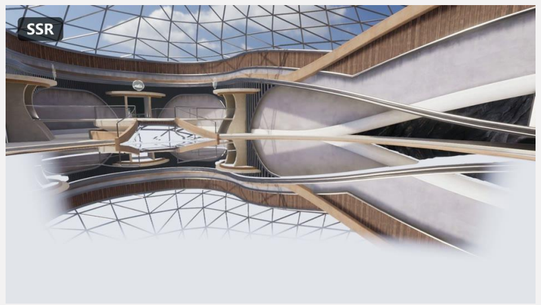
반사 효과를 끈
기준 화면입니다.

반사 프로브



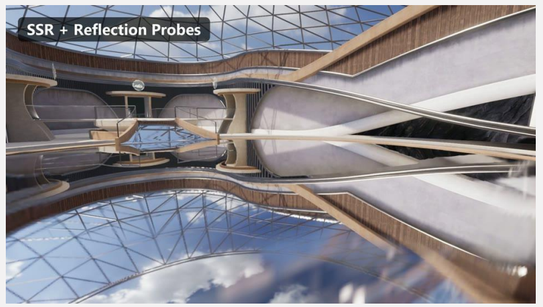
프로브에 저장된 환경을
반사에 사용합니다.

SSR 단독



화면 가장자리와 전경에
반사 정보가 누락됩니다.

SSR + 반사 프로브



SSR의 누락 영역을
반사 프로브로 보완합니다.

오른쪽 결합 화면 클릭: 전체 비교 영상 재생

Unity · Preview of Screen Space Reflections for URP

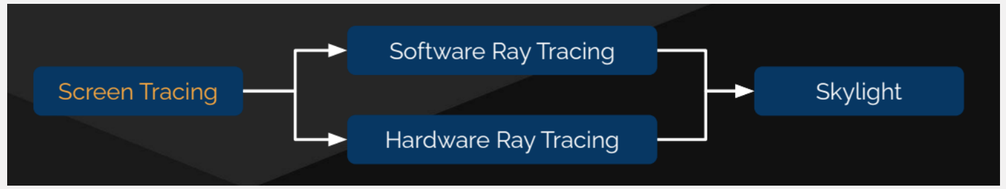
리얼타임 렌더링 4판 · 11장 전역 조명 · 6차시

Lumen의 정반사 (Lumen Reflections)

화면·장면 공간의 혼합 추적과 반사 조명 평가

Lumen은 SSR과 같은 화면 공간 추적으로 반사 광선을 먼저 검사하고, 유효한 결과를 얻지 못하면 장면 공간의 추적으로 보완합니다.
장면 추적에는 부호 있는 거리장(SDF)을 이용하는 소프트웨어 방식이나, 삼각형과 교차를 검사하는 하드웨어 방식을 사용합니다.

공식 발표 · Hybrid ray tracing pipeline



소프트웨어와 하드웨어 추적은 선택 가능한 대안입니다.

교차 위치에서 조명 구하기 · AI 개념도



장면에서 교차 위치를 찾은 뒤에는, 그 표면에서 반사 방향으로 들어오는 빛을 구해야 합니다.
기본적으로는 표면의 재질과 직접광·간접광 계산 결과를 저장한 Surface Cache에서 조명을 조회하여 비용을 줄입니다.

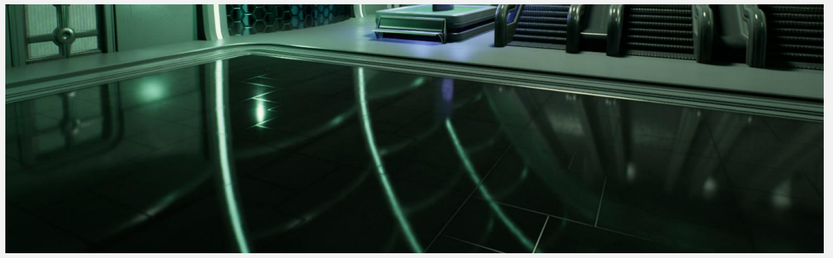
하드웨어 광선 추적에서는 교차 위치에서 재질과 조명을 평가하는 Hit Lighting을 선택할 수 있습니다.
정반사 하이라이트 등을 더 잘 표현할 수 있지만, 교차 위치에서 조명을 계산하는 비용이 추가됩니다.

Surface Cache · 저장된 조명 조회



동일 장면의 반사 영역 비교 · Lumen ON/OFF 비교가 아닙니다.

Hit Lighting · 교차점의 조명 평가



리얼타임 렌더링 4판 · 11장 전역 조명 · 6차시

Epic Games · Lumen, SIGGRAPH 2022 / Technical Details

통합적 접근 방식 (Unified Approaches)

광선 추적의 역할과 실시간 제약

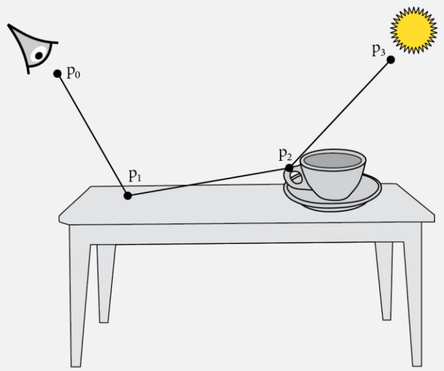
지금까지 다룬 기법들은 확산 간접광과 정반사 등을 서로 다른 방식으로 근사합니다. 각각의 가정과 사용하는 정보가 다르므로, 이를 조합하더라도 장면에 따라 결과가 어긋나거나 일부 조명이 누락될 수 있습니다.

광선 추적은 광선이 어떤 표면에 도달하는지 검사하는 수단입니다. 패스 트레이싱은 표면에서 다음 방향을 샘플링하고 추적을 반복하여, 확산 반사와 정반사 등을 공통된 광 전달 과정으로 계산합니다.

교재는 당시의 성능 조건에서 게임에 패스 트레이싱을 적용하기 어렵다고 설명합니다. 현재는 전용 교차 검사 하드웨어와 디노이징 등을 활용하는 실시간 구현도 있지만, 충분한 수의 경로를 계산하는 비용은 여전히 큽니다.

60fps에서 한 프레임 전체에 사용할 수 있는 시간은 약 16.7ms입니다. 경로마다 교차 검사와 재질·조명 계산이 필요하고, 광선의 진행이 달라지면 GPU의 병렬 처리 효율도 낮아질 수 있습니다. 계산량을 줄이기 위해 샘플 수를 제한하면 결과에 노이즈가 나타납니다.

빛의 경로 · PBRT Fig. 13.5



카메라와 광원을 연결하는 경로의 한 예입니다.

같은 장면의 패스 트레이싱 결과

8 spp · 적은 샘플



8192 spp · 많은 샘플



리얼타임 렌더링 4판 · 11장 전역 조명 · 6차시

* spp (samples per pixel)는 픽셀당 샘플 수입니다. 8 spp는 각 픽셀의 값을 8개 샘플로 추정한다는 뜻입니다.

PBRT 4e · Path Tracing / Fig. 13.4, 13.5

통합적 접근 방식 (Unified Approaches)

래스터화와 디노이징을 이용한 비용 절감

하이브리드 렌더링은 카메라에 직접 보이는 표면을 래스터화로 구하고, 반사나 간접광처럼 추가적인 가시성 검사가 필요한 부분에 광선 추적을 적용합니다. 효과마다 필요한 계산을 나누어 수행하여 프레임의 비용을 조절합니다.

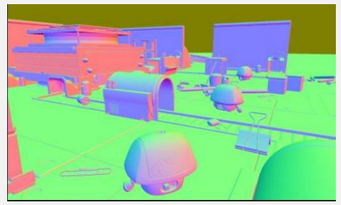
광선 추적에 사용할 수 있는 샘플 수는 제한적이므로, 적은 샘플로 계산한 조명에는 노이즈가 발생합니다. 작은 밝은 광원처럼 경로가 드물게 도달하는 기여는 안정적으로 추정하기 더 어렵습니다.

디노이징은 이전 프레임의 결과를 현재 표면에 대응시켜 누적하고, 주변 픽셀의 정보를 함께 사용하여 노이즈를 줄입니다. 깊이와 법선 등의 정보를 이용하면 서로 다른 표면의 결과가 섞이는 것을 억제할 수 있습니다.

하지만 새로 드러난 영역에는 과거 정보가 없고, 물체나 조명이 빠르게 변하면 잔상이 남을 수 있습니다. 강한 필터링은 세부 그림자나 반사를 흐리게 만들 수 있으므로, 샘플 수와 갱신 속도, 복원 품질 사이의 절충이 필요합니다.

효과별로 나누어 계산하는 하이브리드 구현

표면 정보 · 래스터화



확산 GI · 광선 추적



정반사



EA SEED · PICA PICA (2018)
보이는 표면과 각 조명 효과에 사용할 기법을 나누어 구성합니다.
이 사례의 반사는 광선 추적 또는 Compute 방식을 사용합니다.

적은 샘플에서 조명 복원하기 · SVGF (Spatiotemporal Variance-Guided Filtering)



왼쪽 아래 1 spp · 중앙 SVGF · 오른쪽 위 2048 spp

* spp (samples per pixel)는 픽셀당 샘플 수입니다. 1 spp 입력도 이전 프레임의 결과를 누적하여 복원합니다.

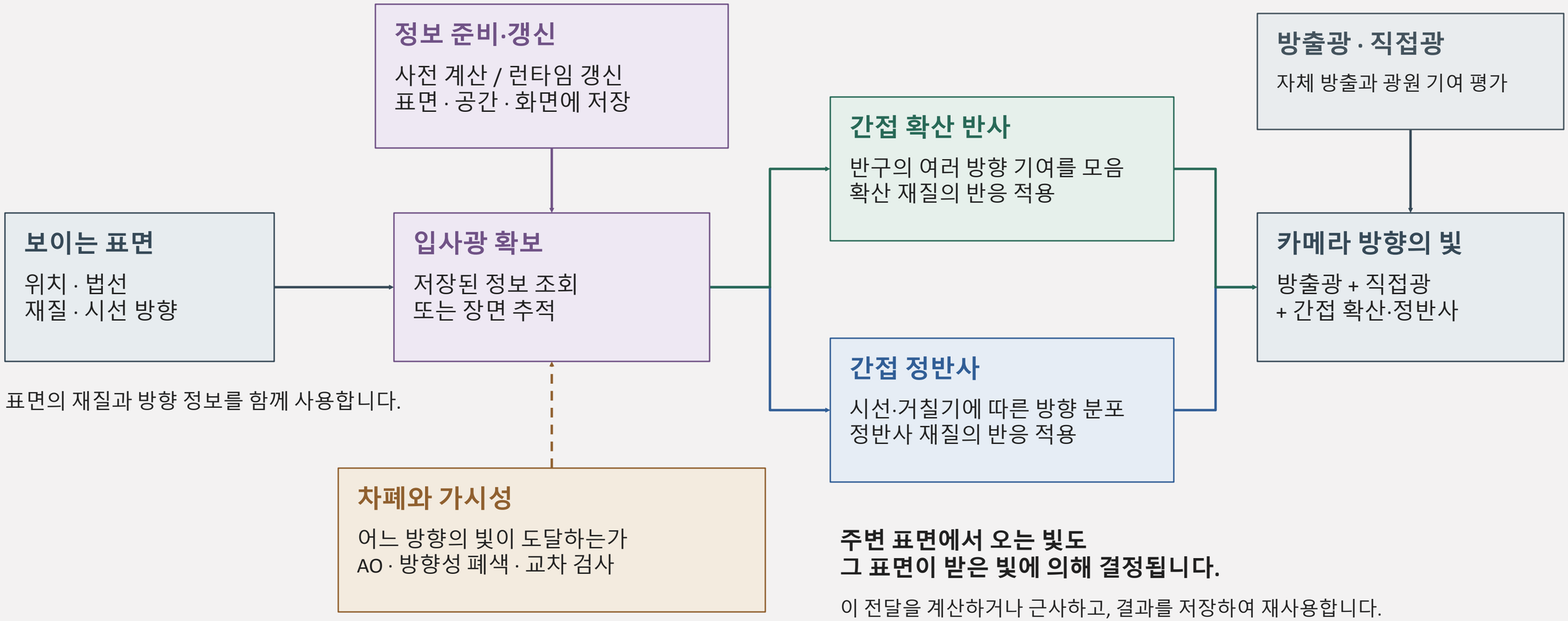
EA SEED · PICA PICA (2018)

Schied et al. · SVGF (HPG 2017)

리얼타임 렌더링 4판 · 11장 전역 조명 · 6차시

전역 조명 전체 정리 (Global Illumination)

주변에서 들어오는 빛을 구하고, 재질에 따라 카메라 방향으로 반사하기



표면의 재질과 방향 정보를 함께 사용합니다.

주변 표면에서 오는 빛도
그 표면이 받은 빛에 의해 결정됩니다.

이 전달을 계산하거나 근사하고, 결과를 저장하여 재사용합니다.

* 계산 역할을 정리한 개념도입니다. 기법에 따라 일부 역할은 사전 계산·조회 과정에 포함됩니다.

11.1 렌더링 방정식 · 11.3~11.4 차폐 · 11.5 확산 GI · 11.6 정반사 GI

전역 조명 전체 정리 (Global Illumination)

무엇을 계산하고, 어디에 저장하며, 언제 갱신하는가

빛의 전달 · 11.1~11.2

렌더링 방정식: 표면 사이의 광 전달
Radiosity: 확산 표면의 에너지 교환
Ray Tracing / Path Tracing: 교차·경로 샘플링

Radiosity·Path Tracing의 계산 결과도
미리 저장하여 런타임에 조회할 수 있습니다.

차폐 근사 · 11.3~11.4

AO·Obscurance: 주변 차폐 정도
Bent Normal: 가려지지 않은 평균 방향
방향성 폐색: 방향별 가시성
사전 계산 · SSAO/HBAO · SDF/Voxel

차폐 정보만으로 주변 표면의 반사광까지 구하지는 못합니다.

간접광을 실시간으로

계산 · 조회 · 조합하기

확산 간접광 · 11.5

사전 계산: 표면 조명·방향성 표현
전달 관계: PRT·Enlighten
저장·조회: Lightmap·공간 Probe/Volume·SH
런타임: VPL/RSM·LPV·Voxel·SSGI

RSM의 반사광 정보는 VPL 계산이나 LPV 주입에 활용됩니다.

정반사 간접광 · 11.6

국소 환경맵: 저장·조회, 시차 보정·동적 갱신
Voxel: 공간에 저장한 반사광 조회
평면 반사: 반사된 시점에서 렌더링
SSR: 화면 깊이·색을 이용한 조회

조합과 재사용 · 11.7

하이브리드 · Fallback · 캐시 · 시간 누적 · 디노이징
Unity: SCGI + SSR·반사 프로브
Lumen: 화면·장면 추적 + 조명 캐시

가지 사이의 공통 표현

Voxel: 차폐·확산·정반사
화면: SSAO·SSGI·SSR
재사용: 사전 계산·캐시·시간 누적

* Probe/Volume은 저장 위치·구조, SH는 방향 분포의 표현입니다.